

中图分类号：TP317

论文编号：10006SY2306142

北京航空航天大学
硕士学位论文

面向云边端协同的跨域分布式训练通信优化研究

作者姓名	李云潼
学科专业	计算机科学与技术
指导教师	肖利民 教授
培养学院	计算机学院

Research on Communication Optimization for Cross-Domain Distributed Training Toward Cloud-Edge-Device Collaboration

A Dissertation Submitted for the Degree of Master of Engineering

Candidate: Yuntong Li

Supervisor: Prof. Limin Xiao

School of Computer Science and Engineering
Beihang University, Beijing, China

中图分类号：TP317

论文编号：10006SY2306142

硕 士 学 位 论 文

面向云边端协同的跨域分布式训练通信优化研究

作者姓名	李云潼	申请学位级别	工学硕士
指导教师姓名	肖利民	职 称	教授
学科专业	计算机科学与技术	研究方向	模型分布式训练
一级学科	计算机科学与技术	学科方向	计算机体系结构
学习时间自	2023 年 09 月 01 日	至	2026 年 05 月 29 日
论文提交日期	2026 年 05 月 08 日	论文答辩日期	2026 年 05 月 25 日
学位授予单位	北京航空航天大学	学位授予日期	年 月 日

关于学位论文的独创性声明

本人郑重声明：所呈交的论文是本人在指导教师指导下独立进行研究工作所取得的成果，论文中有关资料和数据是实事求是的。尽我所知，除文中已经加以标注和致谢外，本论文不包含其他人已经发表或撰写过的研究成果，也不包含本人或他人为获得北京航空航天大学或其它教育机构的学位或学历证书而使用过的材料。与我一同工作的同志对研究所做的任何贡献均已在论文中作出了明确的说明。

若有不实之处，本人愿意承担相关法律责任。

学位论文作者签名：_____ 日期：_____ 年 _____ 月 _____ 日

学位论文使用授权书

本人完全同意北京航空航天大学有权使用本学位论文（包括但不限于其印刷版和电子版），使用方式包括但不限于：保留学位论文，按规定向国家有关部门（机构）送交学位论文，以学术交流为目的赠送和交换学位论文，允许学位论文被查阅、借阅和复印，将学位论文的全部或部分内容编入有关数据库进行检索，采用影印、缩印或其他复制手段保存学位论文。

保密学位论文在解密后的使用授权同上。

学位论文作者签名：_____ 日期：_____ 年 _____ 月 _____ 日

指导教师签名：_____ 日期：_____ 年 _____ 月 _____ 日

摘 要

分布式训练技术已成为支撑大规模模型训练的核心基础。随着模型参数量与训练数据的指数级增长，单机单卡乃至同构集群已难以满足高效训练的资源与存储需求。为了缓解单一数据中心的算力资源压力并避免海量数据跨地域搬移，训练数据与算力资源正在从“单一数据中心”向“云-边-端协同”的跨域形态演进。然而，在跨域分布式场景下分布式训练系统面临着跨域链路异构、域间网络带宽低、延时高且抖动显著的问题。导致在跨域分布式训练场景下，跨域并行训练中的通信操作极易成为系统的性能瓶颈，使得算力资源大量闲置。

针对上述通信瓶颈挑战，本文以“降低跨域同步在训练关键路径中的暴露开销”为核心目标，从通信数据规模压缩、集合通信调度重排、计算-通信深度重叠三个层面展开系统性研究，提出一套面向云边端跨域训练的协同通信优化方案：

(1) 通信数据规模优化层面：针对跨域受限链路下数据交换载荷过大导致的训练吞吐受限问题，提出分布式优化器低比特量化方法。首先，通过 k -bit 随机舍入量化分布式压缩机制，将连续浮点梯度映射为低比特表示的有限离散数值；其次，通过带有残差补偿的量化同步与聚合算法，缓解位宽降低带来的累积噪声。该方法从源头降低跨域同步的数据体积，在受限网络下将单次通信量最高压缩至原规模的 $1/32$ ，在保障收敛稳定性的同时，在 100 Mb/s 低带宽场景下将端到端吞吐量提升了 2.43 倍。

(2) 集合通信调度优化层面：针对跨域链路异构且域间网络带宽低导致集合通信执行效率低的问题，提出基于流水线的层次化梯度通信调度策略。首先，将全局全归约通信分解为域内归约、跨域传输与域内广播三个阶段，降低域间慢链路上的通信压力；其次，引入张量分块级流水线机制，使三阶段按张量块并发接力执行，充分利用域内高速链路掩盖跨域传输等待。该方法优化了跨域异构集群间数据聚合的调度重叠率，使跨域梯度同步通信耗时降低 42.3% ，进而使端到端训练吞吐量提升 1.35 倍。

(3) 计算通信重叠层面：针对跨域训练在单次迭代末尾极易产生不可掩盖的阻塞式同步尾延迟问题，提出面向通信受限场景下的混合并行通信掩盖策略。首先，在反向传播阶段提前触发非阻塞集合通信，为梯度同步预留更长的计算掩盖窗口；其次，利用张量缩放策略预测尚未计算完成的梯度数据并将其纳入跨域同步，降低提前触发导致的梯度尺度偏差；最后，利用轻量级误差反馈，将预测梯度与完整梯度之间的偏差延迟补偿

至下一迭代。该方法在维持“每迭代一次全局同步”语义的前提下，将跨域 All-Reduce 从迭代尾部关键路径中移除，使端到端训练吞吐量提升至未优化基线的 1.87 倍。

本文的系统集成方案在多节点 GPU 集群中进行了验证，依托网络控制复现真实的跨域受限链路条件，并将上述三个层面的机制整合为可协同运行的通信优化框架。端到端评估表明，本优化框架能够将面向云边端协同的模型训练吞吐量提升至未优化基线的 3.85 至 6.41 倍；同时，模型收敛精度与损失曲线与传统全同步训练保持高度一致，在维持模型质量的前提下显著降低跨域同步瓶颈。

关键词：分布式训练，云边端协同，跨域通信，通信优化，大语言模型

Abstract

Distributed training is the fundamental infrastructure for scaling large language models. To alleviate GPU resource pressure in single datacenters and avoid migrating massive data across regions, resources are shifting towards a “cloud-edge-device” collaborative paradigm. However, training systems under this paradigm face severe network heterogeneity: while intra-domain links offer high bandwidth and low latency, cross-domain links suffer from significantly lower bandwidth, higher latency, and larger jitter. Consequently, the blocking gradient synchronization operations across domain clusters become a severe performance bottleneck, underutilizing available computation resources.

To address this bottleneck, this thesis aims to reduce the exposed cost of cross-domain synchronization on the training critical path. It conducts systematic research across three interrelated aspects: transmission-volume reduction, topology-aware collective scheduling, and computation-communication overlap. The main contributions are as follows:

(1) **Communication Volume Optimization:** We propose a k -bit stochastic-rounding compression mechanism ($b \in \{1, 2, 4, 8, 16\}$) with bit packing, quantized aggregation, and residual compensation. It reduces the payload of each cross-domain synchronization by up to 32x and improves end-to-end throughput by 2.43x under a 100 Mb/s constrained link while preserving convergence stability.

(2) **Collective Communication Scheduling:** We propose a pipelined hierarchical All-Reduce scheduler for the asymmetric “fast intra-domain, slow inter-domain” topology. The method decomposes synchronization into intra-domain reduction, inter-domain transfer, and intra-domain broadcast, and further pipelines tensor chunks with dual-stream concurrency. It reduces cross-domain gradient synchronization time by 42.3% and improves end-to-end training throughput by 1.35x.

(3) **Computation-Communication Overlap:** We propose a prefix-triggered non-blocking All-Reduce mechanism for hybrid DP+PP training. It starts cross-domain synchronization before the iteration tail, scales incomplete prefix gradients, and uses residual error feedback to compensate prediction deviations in later iterations. Under intact “one global synchronization

per iteration” semantics, it removes the exposed All-Reduce waiting tail and improves end-to-end throughput to 1.87x of the unoptimized baseline.

End-to-end evaluations on multi-node GPU clusters under emulated cross-domain link constraints validate the integrated framework. Experiments demonstrate a 3.85 ~ 6.41x throughput improvement over the standard synchronous baseline while preserving convergence trajectories close to full synchronization, demonstrating efficient and robust large-model training for cross-domain collaboration.

Keywords: Distributed Training, Cloud-Edge-Device Collaboration, Cross-Domain Communication, Communication Optimization, Large Language Models

目 录

第一章 绪论	1
1.1 研究背景	1
1.1.1 大语言模型的规模演进与分布式训练需求	1
1.1.2 云边端协同训练的兴起	1
1.1.3 跨域通信的特征	2
1.2 云边端协同分布式训练的通信挑战	3
1.3 研究目标与内容	4
1.3.1 研究目标	4
1.3.2 研究内容	6
1.4 本文主要工作与创新点	8
1.5 论文结构安排	9
第二章 国内外相关研究	11
2.1 分布式训练并行策略与系统框架	11
2.1.1 数据并行与参数/状态分片系统	11
2.1.2 模型并行与细粒度张量切分机制	11
2.1.3 流水线并行与流水调度策略优化	12
2.2 跨域与跨数据中心训练	13
2.3 梯度压缩与通信量优化	14
2.3.1 量化与低比特表示	14
2.3.2 稀疏化与误差反馈	14
2.3.3 低秩与混合压缩	15
2.4 集合通信优化与层次化调度	15
2.5 通信重叠与同步尾部优化	16
2.6 云边端协同训练范式与研究空白	17
2.6.1 云边端协同训练问题	17
2.6.2 云边端场景下的跨域训练策略的选择	18
2.7 本章小结	19
第三章 分布式优化器低比特量化方法	20
3.1 引言	20

3.1.1 与云边端协同的关联	20
3.1.2 分布式训练中的通信瓶颈	20
3.1.3 现有梯度压缩方法的局限性	21
3.2 问题分析与研究思路	21
3.2.1 跨域训练中梯度通信的困境	22
3.2.2 现有量化方法的不足与启示	22
3.2.3 本文核心研究问题	23
3.2.4 算法核心思想与解决方案	23
3.2.5 数学表达	24
3.3 详细方案设计与实现	24
3.3.1 量化过程	24
3.3.2 位打包优化	25
3.3.3 反量化与聚合	26
3.3.4 集成到分布式训练框架	27
3.4 实验与验证	29
3.4.1 实验设置	30
3.4.2 实验结果与分析	32
3.5 本章小结	38
第四章 基于流水线的层次化梯度通信调度策略	39
4.1 引言	39
4.1.1 异构集群通信特征	39
4.1.2 传统同步算法的局限	40
4.2 问题分析与研究思路	40
4.2.1 层次化通信拓扑	41
4.2.2 流水线分块策略	42
4.2.3 双流并行机制	43
4.3 详细方案设计与实现	43
4.3.1 阶段一：预热阶段	44
4.3.2 阶段二：流水线主循环	44
4.3.3 阶段三：冷却阶段	45
4.4 理论性能与开销分析	46
4.4.1 时间复杂度	46

4.4.2 空间复杂度	46
4.5 系统集成与工程优化	46
4.6 实验与验证	47
4.6.1 实验环境	48
4.6.2 通信性能对比	49
4.6.3 端到端训练加速	49
4.6.4 块大小敏感性分析	50
4.6.5 带宽不对称性影响	51
4.6.6 低质网络全面实验设计	52
4.7 本章小结	55
第五章 通信受限场景下的混合并行通信掩盖策略	56
5.1 引言	56
5.1.1 并行策略骨架的选择	56
5.1.2 同步尾部成为主要瓶颈	58
5.2 问题分析与研究思路	58
5.3 详细方案设计与实现	60
5.4 前缀触发的异步梯度同步	60
5.4.1 梯度 All-Reduce 的触发时机	61
5.4.2 截断点 τ 的选择规则	61
5.5 预测误差修正	62
5.5.1 前缀梯度与完整梯度	62
5.5.2 发送缓冲与误差更新	62
5.6 工程实现与系统集成	63
5.6.1 与 1F1B 流水线的集成位置	63
5.6.2 通信流与计算流的隔离	63
5.6.3 误差缓冲的存储与开销	64
5.7 理论分析	64
5.8 实验与验证	64
5.8.1 实验设置	64
5.8.2 端到端吞吐	65
5.8.3 收敛曲线与消融	66

5.8.4 网络敏感性分析	68
5.8.5 截断点敏感性分析	69
5.8.6 实验结论	70
5.9 本章小结	71
第六章 面向云边端协同的跨域分布式训练通信优化系统	72
6.1 引言	72
6.2 问题分析与研究思路	72
6.2.1 统一符号与状态记号	73
6.3 详细方案设计与实现	73
6.3.1 模块接口与状态变量约定	74
6.3.2 运行时决策与回退机制	74
6.3.3 工程实现细节与开销讨论	75
6.4 实验与验证	76
6.4.1 系统级实验设置	76
6.4.2 测试目标与判据	77
6.4.3 测试流程	77
6.4.4 测试结果汇总	78
6.4.5 不同带宽下的横向方法对比	79
6.4.6 Batch Size 敏感性分析	80
6.4.7 适用范围与失效模式	82
6.4.8 可复现性与部署建议	83
6.5 本章小结	83
总结与展望	84
7.1 论文工作总结	84
7.2 下一步工作展望	85
参考文献	87
攻读硕士学位期间取得的成果	92
9.1 攻读硕士学位期间取得的创新成果	92
9.2 攻读硕士学位期间参与的主要科研工作	92
致谢	93

插图清单

图 1.1 研究目标与三项研究内容之间的关系	5
图 3.1 大规模分布式训练中的通信瓶颈	21
图 3.2 k-bit 随机舍入量化核心工作流程	23
图 3.3 位打包与解包过程示意图	26
图 3.4 k-bit 与主流压缩方法的端到端性能对比	33
图 3.5 k-bit 在多模型端到端训练中的收益	34
图 3.6 FP32 与 4-bit 在代表性网络条件下的 validation loss 收敛曲线	36
图 3.7 k-bit 量化方法的收敛性与训练加速比验证（以 LLaMA-7B 为例）	36
图 4.1 异构带宽不对称性导致的通信瓶颈	39
图 4.2 层次化 All-Reduce 通信拓扑与数据流向	42
图 4.3 流水线层次化 All-Reduce 时序图（ar: All-Reduce, bc: Broadcast）	44
图 4.4 流水线调度器在训练流程中的集成架构	46
图 4.5 端到端训练性能对比与耗时分解	50
图 4.6 块大小对通信性能的影响	50
图 4.7 固定张量（256 MB）下，不同 chunk 数在带宽不对称场景中的加速比	51
图 4.8 实验组 A：跨域带宽退化设置	52
图 4.9 实验组 B：跨域 RTT 分级设置	53
图 5.1 跨域流水线并行：激活数据在域间传输，产生域间通信瓶颈	56
图 5.2 跨域数据并行 + 域内流水线并行：各域处理本地数据，仅跨 WAN 同步梯度	57
图 5.3 DP+PP 配置下每步计算/通信开销分解：跨域同步成为主要等待来源	58
图 5.4 DP+PP 下的执行时间线对比	60
图 5.5 训练 loss 随 step 变化：POLAR-SGD 与基线高度一致，优于 DiLoCo	66
图 5.6 训练 loss 随墙钟时间变化：POLAR-SGD 更快进入低 loss 区间	67
图 5.7 消融实验：去除梯度缩放或误差反馈会带来轻微收敛退化或稳定性风险	68
图 5.8 POLAR-SGD 在不同网络条件下的性能表现	69
图 5.9 POLAR-SGD 在不同截断点取值下的性能表现	70
图 6.1 三个通信优化模块协同机制与运行时决策闭环	74
图 6.2 固定模型与 batch 条件下，本系统与基线系统的吞吐、迭代时间与通信时间对比	77

图 6.3 不同跨域带宽下 LLaMA-7B 训练吞吐的横向方法对比	79
图 6.4 5 Gb/s 跨域带宽下 batch size 对吞吐与最终 PPL 的影响	81
图 6.5 高带宽/低 RTT 退化测试下的自动回退与一致性验证	82

附表清单

表 3.1	第三章实验硬件环境	30
表 3.2	云边端协同场景的分层链路参数	31
表 3.3	对比实验方法集合	31
表 3.4	多网络条件实验分组	31
表 3.5	多模型端到端实验矩阵	31
表 3.6	实验环境二上不同模型的低比特通信结果	37
表 3.7	实验环境二上 LLaMA-7B 在受限带宽下的吞吐结果	37
表 4.1	第四章实验硬件环境	48
表 4.2	用于云边端协同场景的分层链路仿真参数	48
表 4.3	不同 All-Reduce 方法的通信性能对比	49
表 4.4	端到端训练性能对比	49
表 4.5	实验环境二上不同 All-Reduce 路径的通信耗时	54
表 4.6	实验环境二上不同 All-Reduce 路径的有效吞吐	54
表 5.1	POLAR-SGD 记号表	59
表 5.2	实验设置	65
表 5.3	跨域训练约束下端到端吞吐	65
表 6.1	第六章系统集成的统一符号与状态记号	73
表 6.2	协同系统集成中的接口与状态变量约定	74
表 6.3	第六章系统级实验设置	76
表 6.4	不同网络环境下本系统与现有训练系统的最终对比结果	78

第一章 绪论

1.1 研究背景

1.1.1 大语言模型的规模演进与分布式训练需求

以 Transformer 架构^[1]为核心的预训练范式推动了大规模语言模型与基础模型（Foundation Model）的快速演进^[2]。从 BERT（3.4 亿参数）的双向预训练^[3]，到 GPT-3（1750 亿参数）的少样本学习^[4]，再到 PaLM（5400 亿参数）的 Pathways 架构^[5]、LLaMA 系列（7B 至 70B 参数，开源高效训练）^[6]，以及近期 DeepSeek-R1（6710 亿参数混合专家架构）的推理增强模型^[7]，语言模型的参数规模在短短数年间完成了数量级的跨越。Kaplan 等人通过实证揭示了模型性能与训练计算量、参数量之间的幂律扩展关系（Scaling Law）^[8]，进一步奠定了“大规模训练范式”的理论基础；Hoffmann 等人则在此基础上给出了计算最优的参数量与训练数据量匹配准则（Chinchilla 定律）^[9]，两项工作共同引导了产业界在算力投入不断增大背景下的“大模型训练策略”。基础模型的概念^[2]进一步将这一范式推广至视觉、多模态、科学计算等领域，极大地拓宽了大规模分布式训练的应用场景。

随着参数规模持续攀升，单 GPU 乃至单节点训练已无力承载百亿至千亿参数模型。以 GPT-3（175B）为例，BF16 精度下仅存储模型参数即需约 350 GB，结合 Adam 优化器的梯度与状态，显存需求可超过 2 TB，远超单张 GPU 的 80 GB 容量上限^[4]。

大规模分布式并行训练因此成为必要的基础设施。学术界与工业界共同构建了多维并行体系：数据并行（Data Parallelism, DP）通过批次切分实现规模扩展并以全归约（All-Reduce）同步梯度；张量并行（Tensor Parallelism, TP）在算子粒度切分权重矩阵以降低单设备显存压力^[10]；流水线并行（Pipeline Parallelism, PP）将模型层划分为多个模型分片（stage）并以微批次（micro-batch）为单位串行注入实现层间并行^[11-12]；专家并行（Expert Parallelism, EP）通过条件计算动态路由实现超大规模模型扩展^[13]；ZeRO 技术通过对优化器状态、梯度与参数进行分层分片，显著降低每节点显存占用^[14]。

Narayanan 等人在千亿参数模型上展示了 DP+TP+PP 组合并行（3D Parallelism）的规模化工程实践^[15]，PyTorch 生态进一步提供了对分片并行的工程化支持^[16-17]，形成了覆盖“计算、通信、内存”三个维度的完整分布式训练工程体系。

1.1.2 云边端协同训练的兴起

训练数据与算力资源正从“集中式单数据中心”走向“云-边-端协同”的跨域形态。

驱动力来自多个层面。其一，数据隐私与合规约束（如 GDPR^[18]、HIPAA^[19]、金融数据本地化规定）要求敏感数据就地留存于边缘站点或终端设备，无法集中回传至中心云；其二，业务地理上的分布要求在多个数据中心并行维护训练与推理能力；其三，5G 边缘节点、企业私有云与端侧加速器的快速部署在全球范围形成了分散而异构的算力资源池；其四，出于弹性成本与算力利用率的考量，企业往往希望在训练高峰期借助边缘节点动态扩展算力，在低峰期释放以节省成本。

“云-边-端协同”跨域训练与推理体系已有诸多企业与科研机构在实际部署过程中遭遇并解决了上述挑战。例如，Google、OpenAI 和 Anthropic 均已启动计划，将模型训练从单一站点扩展到多数据中心园区。Google 率先大规模应用了诸多关键技术，Gemini 1 Ultra 的训练便横跨多个数据中心^[20]。OpenAI 在大规模分布式训练中，采用多地域、多数据中心的架构，同时借助跨地域调度与加密分布式存储、通信协议来兼顾合规与训练效率。类似地，字节跳动、Meta 等公司采用“数据就地训练”（on-site training）及联邦学习（federated learning）技术。在我国，东数西算工程也已进入落地实施，形成跨地域、跨部门协同发展合力，统筹通用算力、智能算力、超级算力协同计算，东中西地区及大中小城市协同布局，算力、数据、算法协同应用^[21]。

1.1.3 跨域通信的特征

层次性特征：在云边端协同的物理环境中，由于各类算力中心和终端设备在地理位置上的分散，系统底层网络在物理形态和性能指标上呈现出显著的“层次异构”特征。通常，这种通信架构可划分为三个层级：第一层是节点内互联，例如同一台多卡服务器内部通过 PCIe 或 NVLink 连接，其单向通信带宽可高达数百 GB/s，通信延迟通常处于微秒级^[22]；第二层是域内局域网（LAN）互联，即同一物理机房或边缘计算节点群内部通过高性能以太网或 InfiniBand 交换机连接，通常可提供数十至数百 Gb/s 的吞吐量，并具有较低抖动^[23]；第三层是跨域广域网（WAN）互联，即不同地理位置的边缘节点之间，或边缘与中心云之间的公共网络链路，其可用带宽往往下降至 1 Gb/s 甚至几百 Mb/s，且网络往返时延（RTT）常高达数十至数百毫秒，并伴随着由复杂路由和流量拥塞引起的显著抖动^[24]。

瓶颈链路特征：这种“域内快、域间慢”的带宽非对称性，以及跨域链路上的长尾延迟特征，与当前主流大规模分布式训练框架的设计假设存在明显背离。现有分布式训练框架（如基于 PyTorch DDP 或 Megatron-LM 的数据与模型并行方案）及其底层通信库（如 NCCL），通常默认所有参与计算的节点位于同构、低延迟、高带宽的超算集群中，并

倾向于采用 Ring All-Reduce 或 Tree All-Reduce 等扁平化集合通信原语，将所有参与者视为处于同一通信平面上的等价节点。然而，一旦将这类框架部署到云边端跨域场景中，系统整体通信效率就会被跨域低速链路所主导：高速的域内互联必须等待跨域链路完成同步传输，导致本可充分利用的计算资源和高带宽资源被迫闲置，从而使跨域通信成为制约云边端分布式训练扩展性的核心瓶颈。

低带宽与高延迟特征：在传统集群内训练的物理环境中，节点间互联最高可达 800Gb/s，而跨域链路带宽最高仅可达到数 Gb/s，最低可到数十 Mb/s，因此，跨域链路本身带宽低的特性，不仅仅要求系统从数据源头削减通信体积；又进一步要求在保证全局同步正确性的前提下，尽可能挖掘计算与通信并行的时间窗口，用以掩盖网络阻塞。

因此，层次异构性所带来的上述挑战，构成了本文从通信量压缩、集合通信调度优化以及计算-通信重叠三个维度重构和优化分布式通信系统的直接动因。

1.2 云边端协同分布式训练的通信挑战

在同步数据并行训练框架中，每次迭代末尾需通过 All-Reduce（或其等价变体）聚合全局梯度均值。当训练跨域展开时，同步通信面临以下三个系统性挑战：

挑战一：跨域带宽受限与梯度规模的耦合放大。大模型梯度同步的通信量与参数规模线性正相关：以 BF16 精度计，175B 参数模型每步梯度约需 350 GB 的单向传输量。在采用层次化 All-Reduce 时，跨域部分至少需传输一次完整的归约梯度（Reduce-Scatter 阶段后）与广播结果（All-Gather 阶段），总量仍达数百 GB 量级。若可用跨域带宽仅有 10 - 30 Gb/s，即使通过梯度量化将通信量压缩至原始的 1/4 至 1/32，跨域传输时间仍可能占据总迭代时间的相当比例，出现算力翻倍但吞吐停滞乃至扩展节点反之更慢的现象。

挑战二：分层拓扑与扁平化集合通信不匹配。传统扁平化 Ring/Tree All-Reduce 将各节点视为对等参与者，无法感知“域内高速—域间低速”的拓扑差异，常导致域内高速链路的聚合潜力被域间低速瓶颈拖累。层次化 All-Reduce（域内 Reduce-Scatter → 跨域 All-Reduce → 域内 All-Gather）能够将跨域通信量从正比于全局节点数降为仅传输一份归约结果，显著降低跨域流量；但如何进一步对层次化各阶段进行流水化与并发调度，以充分利用域内高速带宽并最大化跨域传输的重叠度，仍是调度层面的核心问题。

挑战三：混合并行下计算-通信重叠窗口受限。WAN 链路的高 RTT（跨大洲可达 50 - 200 ms）与抖动（10 - 50 ms 量级）会将集合通信的启动开销与同步等待放大至不可忽略的程度^[24]。在同步 DP 框架中，每步训练须等待最慢节点完成通信，而链路抖动带来的随机慢节点（straggler，指由于各种系统或网络干扰而执行明显慢于其他节点的成员）效应

会使吞吐分布出现长尾，系统整体有效吞吐受最慢单次通信主导。在 DP+PP 混合并行场景中，流水线末尾阶段的 All-Reduce 本已处于计算关键路径之外，高 RTT 进一步扩大尾部阻塞窗口，加重设备空闲^[25-26]。在数据并行框架中，常见的优化手段是将梯度按层分组为若干 bucket（数据桶），在反向传播过程中逐批启动 All-Reduce，使通信与后续层的反向计算形成时间重叠。然而在跨域高 RTT 环境下，最后若干梯度数据桶对应的反向层计算已完成，而其 All-Reduce 尚未结束，形成不可压缩的同步阻塞间隙（synchronization gap）。在 DP+PP 混合并行的流水线排空（flush）阶段，此问题尤为突出：PP 进入排空阶段后已无新 micro-batch 可投入计算，跨域 All-Reduce 仍在进行，所有设备不得不空等，直接拉低整体硬件利用率。

上述三个挑战共同表明：面向云边端协同的跨域分布式训练，单纯依赖单一层面的改进已无法消除系统瓶颈。必须从通信量、集合通信算法以及并行策略优化三个维度构建协同优化的理论与工程闭环：首先，压缩通信产生的绝对数据体积，减少物理传输的载荷（应对挑战一）；其次，重排分层集合通信的调度顺序，以更大限度发掘异构带宽潜力并降低启动延迟（应对挑战二）；最后，在系统关键路径维度上改变同步的触发时机，将不可避免的残余通信掩盖在计算窗口之下（应对挑战三）。沿着通信量、集合通信调度以及计算-通信重叠这三个层面展开协同优化，在不牺牲训练语义的前提下有效提升端到端训练效率。

1.3 研究目标与内容

在此研究背景和问题挑战驱动下，本文围绕云边端协同的约束特征构筑解决方案。为便于刻画跨域网络特征并开展可控实验，在实验中通过流量整形工具（tc-netem）精确复现^[27]跨域网络特征。在此基础上，本文的研究目标与具体内容如下。

1.3.1 研究目标

针对云边端协同的跨域分布式训练场景中，跨域链路异构、跨域带宽受限、高时延与强抖动等网络约束导致梯度同步低效、训练吞吐严重不足这一核心挑战，本文旨在通过系统性优化通信路径与调度策略，提升异构分布式环境下的训练效率。研究围绕“压缩通信量、适配拓扑差异、增强计算通信重叠”三个方向，构建一套完整的跨域训练通信优化框架。具体而言，首先通过低比特量化机制削减跨域链路负载；其次设计层次化流水集合通信调度，以匹配云边端非对称带宽特征；最终引入前缀触发异步与计算-通信深度重叠策略，对冲高时延带来的空闲等待。以下分三个关键技术点展开论述：

面向跨域带宽瓶颈的低比特量化通信机制：跨域带宽资源紧张，传统梯度同步产生的海量数据极易造成网络拥堵与传输延迟。现有压缩方法中，稀疏化（如 Top-K）需额外计算开销且难以无损恢复，而低精度量化（如 INT8）虽减少数据位宽，却常引入较大数值误差，影响模型收敛精度。为此，本文提出一种基于随机舍入的低比特量化策略，配合张量位打包与误差补偿技术，在保证收敛的前提下将梯度数据压缩至较低位宽，显著降低跨域链路上传输的有效字节数，缓解带宽压力。

基于张量分块与双流并行的层次化流水线集合通信调度机制：跨域与域内带宽差异显著，传统扁平化集合通信原语在非对称拓扑中效率低下，导致高速域内链路被低速跨域传输拖累。现有层次化方法（如分层 Ring All-reduce）虽然减少跨域数据量，但各阶段间仍存在串行阻塞。本文设计一种张量分块与双流并行结合的层次化流水线调度方案：首先依据拓扑将 All-Reduce 分解为域内与跨域层次，进而通过张量分块将跨域流程细分为多块，并借助双通信流实现块间流水与阶段接力，从而最大限度重叠通信过程，提升整体链路利用率。

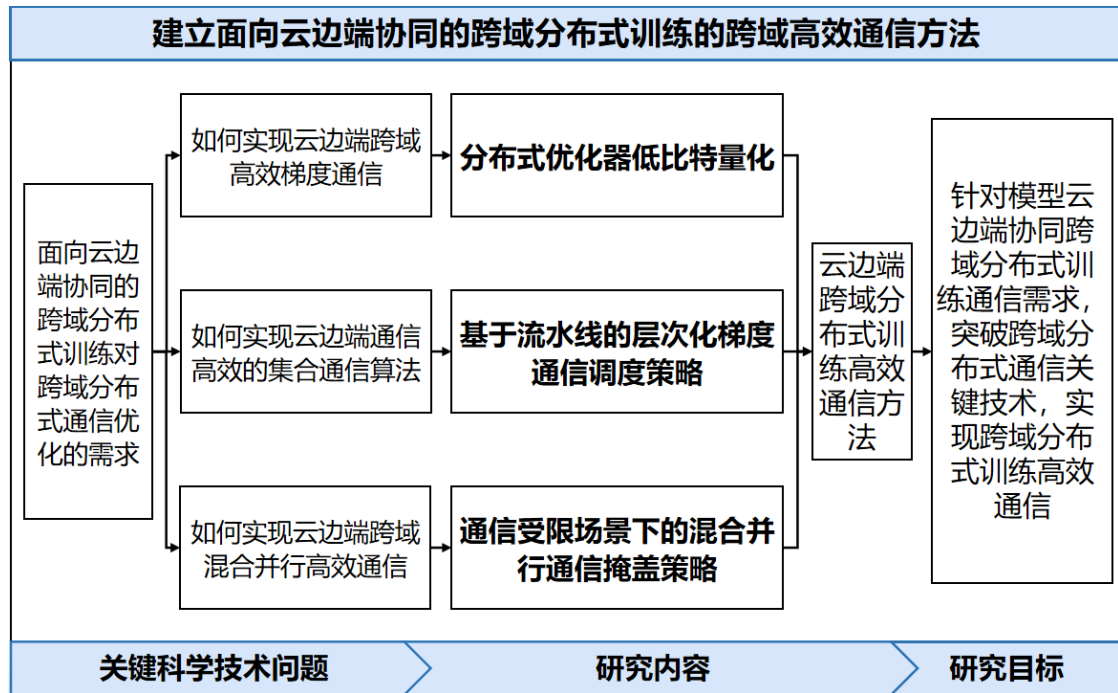


图 1.1 研究目标与三项研究内容之间的关系

面向高 RTT 混合并行场景的前缀触发异步与计算-通信重叠策略：广域网高时延与抖动导致同步等待空窗期延长，尤其在混合并行任务中，流水线排空阶段进一步放大计算设备闲置。现有重叠方法（如梯度分桶调度）仅能在反向计算内部隐藏部分通信，难以完全覆盖极端 RTT；而完全异步更新又会导致梯度陈旧与收敛不稳定。本文结合数据

与流水线混合并行的特点，提出前缀触发异步通信机制：将集合通信的触发时机从反向传播末期提前至早期阶段，使跨域同步与剩余反向计算在时域上完全并发；同时引入陈旧梯度预测与误差修正，以抑制异步更新带来的收敛波动，从而在维持模型收敛精度的前提下显著提升训练吞吐。

通过上述三项关键技术的协同优化，本文致力于在云边端异构资源与受限网络环境下，实现高效、稳定的大规模分布式模型训练，为跨域协同智能任务提供系统级支撑。

图 1.1 进一步概括了本文研究目标与三项研究内容之间的对应关系。整体来看，本文的核心目标是降低跨域受限链路对分布式训练吞吐的限制，使跨域同步尽可能从训练关键路径中移除。围绕这一目标，三个研究内容分别从通信数据规模、通信执行路径和通信触发时序三个方向作用于同一瓶颈：低比特量化直接减少跨域传输的数据载荷，层次化流水线调度提高域内与域间链路的并行利用率，计算-通信掩盖则将仍不可避免的跨域同步前移到可重叠计算窗口中。

三项研究内容之间是并行且相互支撑的协同关系。量化方法降低 T_{comm} 的物理下界，使跨域同步更容易被后续计算覆盖；层次化流水线调度重排通信阶段，减少跨域链路等待并为通信掩盖提供更稳定的执行基础；计算-通信掩盖则直接面向迭代关键路径，将残余同步时延压入反向传播窗口。理想情况下，当量化压缩将跨域通信时长降低到可重叠窗口以内，并由前缀触发机制充分提前执行时，暴露在迭代尾部的跨域同步开销可被完全掩盖，从而在关键路径视角下趋近于 0。上述关系也说明，本文的创新是围绕降低跨域受限链路对分布式训练吞吐的限制，使跨域同步尽可能从训练关键路径中移除这一系统目标构建的组合式通信优化框架。

1.3.2 研究内容

论文拟研究面向云边端协同的跨域分布式训练通信优化技术。在此场景下，一次典型的训练迭代通常遵循“计算-通信”的循环模式：各分布式节点在完成本地的前向传播与反向传播计算后，需触发集合通信操作，跨网络链路同步聚合梯度，以实现模型参数的全局更新。在跨域异构拓扑（高速域内、低速跨域）与高延迟抖动网络条件下，海量的梯度张量需要在不同域的计算节点之间进行高效、可靠地传输与归约。然而，该过程正面临传输开销大、通信链路利用率低、以及混合并行下同步尾部阻塞长等严峻问题。针对这一完整的通信通路，本文旨在从数据体积、调度策略、时序重叠三个层面进行系统性优化，以提升端到端训练效率。具体的研究内容包括：

- (1) 面向跨域带宽瓶颈的低比特量化通信机制

跨域网络带宽受限，大规模模型梯度传输的绝对数据量庞大导致通信延时极长。在跨域带宽极具受限时，庞大体积的数据传输不仅严重拖慢单步训练速度，还极易引发网络拥堵与重传。因此，在跨域分布式训练系统中实现高效的梯度通信量压缩机制以降低网络传输负荷具有重要意义。

为了降低分布式训练系统的通信开销，研究人员主要针对稀疏化和量化两方面开展研究。在稀疏化方面，以 Top-K 为代表的选择策略，通过仅提取并传输绝对值最大的参数或梯度，极大地下降了通信数据量，但往往引入了额外的筛选与排序负担且难以保证精确恢复；在量化方面，以 INT8 或混合精度为代表的量化低精度通信机制，通过降低数据表达精度，减少每个数据元素占用的位宽从而极大缓解了传输时的带宽压力，但低比特量化容易引入较大数值误差而使得模型最终的收敛轨迹受损。

针对上述问题与现状，本文在保证模型收敛的前提下，引入基于随机舍入机制的低比特策略，设计与之配套的张量位打包与误差补偿策略，最大化压缩跨域链路上的梯度有效传输字节数。最终，构建一套面向跨域带宽瓶颈的低比特量化通信机制。

（2）基于张量分块与双流并行的层次化流水线 All-Reduce 调度机制

跨域与域内网络带宽性能差异巨大，传统扁平同步原语在非对称拓扑中执行低效导致链路资源大量闲置。域内互联高速而跨域互联慢速，现有的扁平集合通信策略让所有节点对等参与全连接通信，导致域内高速链路使用不充分，被域间低速瓶颈拖累。因此，在跨域分布式系统中实现与物理拓扑相适配的分层集合通信机制以提升整体带宽利用率是十分必要的。

目前，众多研究人员已经针对复杂异构网络拓扑的集合通信过程展开了广泛的研究，探讨了多种调度机制。其中，以层次化 Ring 算法为代表的分层调度方法，通过解耦域内与域间的归约流程将跨域通信数据量降至理论最低，但未能解决各通信阶段之间相互串行等待的同步阻塞。

针对上述问题与现状，本文依据跨域通信系统内的极度非对称带宽特性，引入张量分块机制将层次化 All-Reduce 的跨域流程进一步微细化拆分，同时结合双通信流协调跨阶段流程的并发与接力执行，实现跨域同步过程中的调度高重叠率。最终，构建一套基于张量分块与双流并行的层次化流水线 All-Reduce 调度机制。

（3）面向高 RTT 混合并行场景的前缀触发异步与计算-通信重叠策略

广域网络高时延特征与网络抖动严重，混合并行任务内的流水线排空期导致长尾通信延迟极易使计算设备长时间闲置。同步迭代更新是保证模型无损收敛的核心准则，但

在跨域高 RTT 场景下，原生训练后端要求等待所有后向传播计算结束后才触发梯度同步的机制，直接暴露出了不可掩盖的超长空窗期。因此，在跨域分布式系统里引入更为积极主动的计算-通信并行重叠机制以对冲广域高时延负面影响并提升吞吐率至关重要。

为隐藏并重叠分布式系统中的通信延迟开销，研究人员提出了诸多时域重叠方法。其中，以张量桶 (Bucket) 分块调度为典型的重叠方案，利用后向梯度逐层计算产生的顺序差，实现计算过程对通信过程的掩盖。但这类优化大多局限于反向计算范围之内，面对跨域极其极端的广域 RTT 网络仍会剩下大量未能遮蔽的间隙时间；以完全异步参数服务器为代表的非同步更新策略，通过完全打破全局屏障消除了集群等待现象，却常常导致梯度更新的陈旧效应与收敛轨迹明显发散。

针对上述问题，本文结合数据与流水线混合并行的模型训练特征，将集合通信任务的触发时机从反向传播最后期抽离并提前移至早期阶段，使得跨域低速通信可以完全与模型的反向计算时段在时域上并发。本文并配合对陈旧参数更新的滞后进行预测和误差修正，平衡异步行为的影响。最终，构建一套面向高 RTT 混合并行场景的前缀触发异步与计算-通信重叠优化机制。

1.4 本文主要工作与创新点

本文面向以云边端协同的广域分布式训练系统，以通过通信优化方式提升面向云边端场景下的分布式训练任务效率为目标，通过充分调研、分析和总结国内外相关工作，针对跨域链路传输模型梯度数据的开销高、在跨域链路上执行集合通信的带宽利用率低、训练各轮所需同步尾部长等问题，从“通信量—调度—重叠”三个层次对跨域分布式训练通信优化进行系统性研究，分别提出，面向跨域带宽瓶颈的低比特量化通信机制，以降低传输模型梯度数据开销；张量分块与双流并行驱动的层次化流水线 All-Reduce 机制，以提升跨域同步时链路间通信的并行度与整体通信效率；前缀触发异步 All-Reduce 与预测误差修正机制，以减少迭代末端的同步阻塞。具体工作与创新点如下：

本文创新性的核心在于最大程度降低跨域同步开销为统一目标，将通信体积削减、拓扑感知调度和同步时机重排三类机制组成可叠加的系统方案：量化降低 T_{comm} ，流水调度提升链路并发与执行效率，前缀触发重叠则将残余同步压入 T_{comp} 窗口。三者共同作用，使跨域同步不再只是被局部加速，而是在理想条件下可被计算过程完全掩盖。

工作一（第 3 章）：面向跨域带宽瓶颈的低比特量化通信机制。 针对跨域链路传输大模型梯度的高开销问题，本文提出基于 k-bit 随机舍入量化的分布式优化器，支持可配置位宽 $b \in \{1, 2, 4, 8, 16\}$ 。在此基础上，设计配套的位打包 (bit-packing)、量化梯度聚合

与误差补偿（error compensation）机制，实现对量化张量的高效通信支持。在受限链路上，16-bit 至 1-bit 的量化最高可将通信数据量降至原始的 1/32，同时通过误差反馈维持收敛稳定性，在压缩率与优化质量之间提供可调折中。

与现有工作相比，本方法通过系统的量化机制设计与工程实现，将低比特量化通信机制成功应用于跨域分布式训练场景，并通过大规模实验验证了其在不同带宽约束下的性能提升与收敛表现，为跨域通信优化提供了实用且高效的解决方案。

工作二（第4章）：张量分块与双流并行驱动的层次化流水通信。 针对“域内快、域间慢”的分层互联特征，本文提出将层次化 All-Reduce 的“域内 Reduce-Scatter—域间 All-Reduce—域内 All-Gather”三个阶段分解为可流水化执行的分块调度策略，并通过双 CUDA 流并行驱动域内与域间通信的并发执行，充分利用域内高速带宽对跨域同步延迟的掩盖窗口，显著提升跨域同步的重叠度与整体通信效率。

与现有工作相比，本方法首先提出创新的分块调度与双流并行机制，成功将流水线化 All-Reduce 从理论框架转化为高效的工程实现，并在跨域分布式训练场景中展示了其显著的性能提升，提供了针对分层拓扑的集合通信优化新思路。

工作三（第5章）：前缀触发异步通信与预测误差修正机制的通信掩盖策略。 针对高 RTT 环境下混合并行训练末尾的同步尾部问题，本文提出将 All-Reduce 的触发时机从反向传播完成后前移至前向传播中某一前缀层完成后立即启动，利用前向与后续反向传播之间的计算窗口覆盖跨域通信延迟；对尚未完成反向传播的参数引入预测误差修正，保证全局同步的统计等价性。该机制在每迭代一次全局同步的宏观语义下，将跨域 All-Reduce 从迭代关键路径末端剥离，有效改善端到端吞吐与收敛速度。

与现有工作相比，本方法在跨域混合并行训练中提出了以数据并行为主要并行策略架构，创新性地结合了流水线并行的特征，重新设计了同步通信的触发时机，并通过预测误差修正机制确保了训练语义的正确性，在高 RTT 条件下显著降低了同步尾部对整体效率的影响，提供了面向跨域分布式训练的重叠优化新范式。

1.5 论文结构安排

全文组织如下：第1章，介绍研究背景、问题挑战与本文贡献。第2章，综述云边端协同训练、分布式并行与系统框架、梯度压缩、集合通信调度、通信重叠与尾部优化等方向的代表性工作，并基于此展开分析，确定研究问题、研究路线。第3章，给出低比特量化分布式优化器与低比特位的通信机制的设计与实现，并讨论其在跨域链路上的适用性。第4章，提出并分析层次化流水线 All-Reduce 调度策略，给出工程集成与实验

评估。第 5 章，提出通信受限场景下的混合并行通信掩盖策略，并在跨域约束下给出实验验证与分析。第 6 章，将低比特量化、层次化流水线通信与计算-通信掩盖整合为统一系统，并给出系统级实验与测试汇总。

第二章 国内外相关研究

本章围绕“云边端协同的跨域分布式训练通信优化”主题，综述相关研究脉络。为便于对齐本文的三项工作，本章按“并行训练与系统框架—跨域/跨数据中心训练—通信量压缩—集合通信调度—通信重叠与尾部优化—云边端协同训练范式”的逻辑组织。

2.1 分布式训练并行策略与系统框架

2.1.1 数据并行与参数/状态分片系统

数据并行 (Data Parallelism, DP) 通过在不同计算节点上复制模型副本并对训练数据批次进行切分，是实现分布式扩展的最基础并行范式且极大地提升了训练吞吐量，但其关键成本来自每步迭代的梯度同步。随着大语言模型时代到来，庞大的模型参数量轻易突破单卡甚至单机节点的显存上限，传统的全复制 DP 难以为继。为此，微软 DeepSpeed^[28] 团队提出了 ZeRO (Zero Redundancy Optimizer) 系列工作^[14]。ZeRO 通过将优化器状态、梯度和模型参数这三大显存开销来源在数据并行组内进行独立的分层切分保存（分别对应 ZeRO-1、ZeRO-2 与 ZeRO-3 阶段），从根本上消除了数据并行节点间的显存状态冗余，极大降低了显存占用。在此理念影响下，PyTorch 官方相继推出了原生的 Fully Sharded Data Parallel (FSDP)^[17] 分布式训练框架技术。DeepSpeed 和 FSDP 逐渐演变成成为当今学术界与工业界进行大规模语言模型训练时最为核心的底层并行系统基础设施。

从云边端协同角度看，由于边缘站点的计算与存储资源往往比云端数据中心更加局限，ZeRO 等分片优化机制使得显存有限的边缘节点也能够承载部分超大参数模型或较大的 batch 重载。这为“边缘侧与端侧参与统一的大模型训练与聚合”提供了基础系统可行性；但在跨域链路受限的大背景下，数据并行模式每轮更新所必备的庞大跨节点 All-Reduce 通信仍大概率成为阻碍端到端吞吐率的核心瓶颈，需进一步引入特定针对性的网络与通信调度优化。

2.1.2 模型并行与细粒度张量切分机制

当单张加速卡的显存不但无法容下整个模型，甚至连单一模型层的张量以及相关的中间激活状态也难以完全保留时，便衍生出了对计算图进行细粒度切分的模型计算并行。NVIDIA 推出的 Megatron-LM^[10] 训练开源骨架标志着大规模张量并行 (Tensor Parallelism, TP) 技术的大规模工业化落地。Megatron 创新地利用了 Transformer 结构的

数学特征，将 Attention 和多层感知机（MLP）中的线性层权重矩阵沿着行和列同时执行跨设备的切分计算，实现了在 GPU 节点集内高效的分布式张量矩阵乘。

在后续的规模化发展中，为解决极长上下文序列训练带来的巨大显存暴涨，研究者们将切分维度进一步拓展：序列并行（Sequence Parallelism, SP）将 Transformer 结构中未被 TP 涉及的那些如 LayerNorm 和 Dropout 模块的部分按序切分以避免激活显存冗余^[29]；而面对数以百万计词元规模的输入，上下文并行（Context Parallelism, CP）则通过将注意力头的计算沿上下文环形拓扑分布式打散^[30]。此外，面对近年风靡的混合专家系统（Mixture of Experts, MoE）大模型架构，专家并行（Expert Parallelism, EP）^[13]则展示了较高的参数拓展效率，通过将不同类别的专家模型静态或动态散列分布在网络中的不同设备节点上并依靠高速交换机进行词元级别的全-To-All 分发路由，成功在只增加部分额外通信的前提下带来了整体模型容量增长。

2.1.3 流水线并行与流水调度策略优化

流水线并行（Pipeline Parallelism, PP）通过将层级极深的大规模神经网络按序列划分出多个阶段（Stage），并将不同的 Stage 均等映射放置于不同的计算节点或设备组上，突破了内存墙与节点独立算力限制。为了削减不同级联 Stage 等待数据的闲置并隐藏空闲的气泡（Bubble）时间，GPipe 系统^[11]等开创性地提出了微批次（Micro-batch）的切分输送机制。

为更高效拉高硬件利用效率、突破深层反向链条带来的峰值显存增长，PipeDream 系统^[12]及其后续工作提出并确立了经典的 1F1B（One Forward One Backward）调度策略。它是目前主流流水线并行训练的基本范式之一，通过让流水设备交替进行当前微批次的前向与另一个微批次的反向阶段的计算投递，从而迅速释放已经过时的中间激活来减少峰值显存。在 1F1B 的基础上，业界在近年不断推出深度的拓扑排列改进以逼近性能理论极限：交错 1F1B（Interleaved 1F1B）通过给每一台物理设备人为指派多个彼此不邻接的阶段负载，进一步通过多阶段重叠大幅削减了原本存在的流水首尾启动气泡空腔的固有比例^[15]。而针对最终残留不可消除的气泡，诸如 ZeroBubble PP^[31]与 DualPipe^[32]等更为激进而先进的双向/三向层级掩盖交叉调度等前沿算法被提出。它们通过在反向阶段进一步细分子执行块，或利用前后向双向流的算子互叠时间等错位手段，几乎把流水线上的设备空载损耗降至理论极限的“零气泡”。

在云边端协同的大规模调度拓扑场景中，由于需要进行极为频繁的激活及其对应特征梯度的逐层前向与后向传递投喂，流水线并行及其演进的系列并发切分方法，通常更

适合在局部边缘域内采用,但当需要在带宽严重劣化的跨域链路间反复进行大量激活的传输仍存有一定的劣势。作为对比而言,在广域跨域层面的宏观网络下,采用在每次模型完整迭代的末期进行单次最终梯度传输交互的数据并行范式与联邦学习场景下的参数服务器范式相对更为常见。

2.2 跨域与跨数据中心训练

跨数据中心/地理分布式训练可视为云边端协同中的域间互联的典型场景,其核心特征除物理距离远外,还包括网络带宽低、时延高、抖动大,以及可调度计算资源呈现出的不稳定性与异构性。与单机房或单集群训练不同,跨数据中心训练往往需要在 WAN 约束下协调多个自治域,因此,围绕跨域训练的研究通常同时关注“如何组织计算”和“如何组织通信”两个层面。

从系统框架看,近年来已经出现了面向跨数据中心流水线调度与训练框架的代表性研究。CrossPipe 针对跨数据中心训练提出更优的流水线调度策略,强调在强约束 WAN 下通过调度重排和阶段组织减轻通信尾部^[25]; GeoPipe 探索了地理分布式 LLM 训练框架中的流水线并行增强,进一步说明了光网络或专线互联条件下跨机房流水线训练的可行性与收益^[26]; JEEVES 则从系统协调和任务编排角度研究跨 DC 训练的调度问题,体现出跨域训练不仅是通信优化问题,也是跨域资源协同问题^[33]。上述工作共同说明,跨域训练的关键不在于简单复制单域并行方案,而在于根据域间链路特征重新设计训练阶段划分、同步位置和执行顺序。

从训练范式看,跨数据中心训练的优化还与低通信频率和弱同步优化紧密相关。经典的 FedAvg 将局部多步更新与周期性模型平均结合,在不频繁同步的情况下实现分布式训练^[34]; Local SGD 进一步从理论和实践角度说明了“少通信、多本地更新”在收敛与泛化上的可行性^[35-36]; SCAFFOLD 则通过控制变量修正局部漂移,缓解了数据异构条件下的优化偏差^[37]。沿着这一思路, OpenDiLoCo 将低通信频率训练推广到全球分布式场景^[38],而 Streaming DiLoCo 进一步强调通信与计算的重叠,试图在保留低通信频率优势的同时缩短同步尾部^[39]。这类方法与本文的关系在于:它们说明在广域网络中,降低同步频率和缩短同步关键路径是提升系统效率的重要方向,但若要保持更严格的训练语义和更可控的收敛过程,仍需要对同步数据量、通信调度和尾部重叠进行优化。

从更广义的宽域并行作业角度看,数据中心之间的优化问题涵盖深度学习等多个领域。PIXIDA 针对宽域数据分析任务提出了面向数据并行作业的优化机制,为理解 WAN 约束下的任务放置、作业切分与跨域传输提供了参考^[40]。该类工作表明,跨域系统设计

必须同时考虑网络拓扑、数据移动和任务执行代价，这一点与本文对云边端协同训练的建模方式是一致的：跨域链路上的通信是决定训练是否能够高效运行的核心约束。

综合上述研究可以看到，跨数据中心训练已经形成了三条相互关联但侧重点不同的技术路线：其一是以 CrossPipe、GeoPipe、JEEVES 为代表的跨域训练系统与调度框架，重点解决如何排布训练策略和放置合理的模型分片来提升并行训练效率；其二是以 FedAvg、Local SGD、SCAFFOLD、OpenDiLoCo 为代表的低通信频率或弱同步范式，重点解决如何降低传输频率来降低同步阻塞占比；其三是 Streaming DiLoCo 等工作为代表的通信-计算重叠方法，重点解决如何提升通信被计算掩盖的程度来提升计算效率。本文第 2 章后续章节中的三项工作分别对应上述技术路线中的后三个关键问题，即在保持跨域 DP 语义的前提下，进一步解决“传多少、怎么排、如何掩盖尾部”的系统化问题。

2.3 梯度压缩与通信量优化

梯度压缩通过减少每步需要传输的数据量来降低通信时间，是应对跨域带宽受限的第一道系统防线。现有研究大体可分为三类：量化压缩、稀疏化压缩与低秩压缩。三类方法的共同目标都是在尽可能保持收敛语义的前提下压缩梯度通信体积，但其信息保留方式、误差来源以及工程实现复杂度各不相同。

2.3.1 量化与低比特表示

量化方法通过降低梯度数值精度来直接压缩传输字节数，是最直观也最易工程化的一类通信压缩策略。早期研究表明，深度学习训练可以采用低精度数值表示：Deep Learning with Limited Numerical Precision 从训练数值稳定性的角度说明了低精度表示的可行性^[41]。随后，1-bit stochastic gradient descent 将梯度直接压缩到符号级别，在分布式训练中证明了极低位宽通信的潜力^[42]。QSGD 进一步从随机量化的角度给出了具有理论收敛保证的通信最优量化框架^[43]；TernGrad 则使用三值梯度表示，在保持较低误差的同时缓解了通信瓶颈^[44]。

在更贴近模型训练的场景中，量化方法通常不再仅仅追求极端低比特，而是强调精度、收敛和工程效率之间的平衡。对于跨域训练而言，这一平衡尤为重要：链路带宽本身就是瓶颈，如果压缩率不足以覆盖 WAN 传输成本，量化就难以体现系统收益；但若位宽过低，又容易引入较大的量化噪声并放大收敛不稳定性。因此，围绕随机舍入、动态缩放与误差补偿的低比特量化框架，逐步成为当前跨域同步压缩的主流方向。

2.3.2 稀疏化与误差反馈

稀疏化方法通过只传输 Top- k 或随机采样的一部分梯度元素来减少通信量，但会引入偏差或方差增大^[45]。为了缓解这种信息损失，研究者提出了带记忆的稀疏化方案：Sparsified SGD with Memory 通过维护残差累积未发送信息，从而改善稀疏传输下的优化轨迹^[46]。Deep Gradient Compression 进一步将动量修正、局部裁剪、稀疏化与延迟补偿组合为完整系统，展示了在大规模分布式训练中实现数量级带宽压缩的可行性^[47]。后续研究还进一步从理论与实践上分析了误差反馈在压缩 SGD 中的关键作用，例如 Error Feedback Fixes SignSGD and other Gradient Compression Schemes 说明误差反馈能够修正符号压缩等方案的偏差，使其恢复稳定性^[48]；Step-Ahead Error Feedback 和 Detached Error Feedback 则继续将误差补偿扩展到更灵活的时序与随机稀疏设置中^[49-50]。

在云边端协同场景中，误差反馈对跨域受限链路尤为重要：当端↔边、边↔云链路带宽波动时，误差反馈可作为一种“残差信道”，将被压缩或延迟的梯度信息在后续迭代中逐步补偿，从而在系统效率与优化稳定性之间提供可控折中。

为了在不同网络与梯度分布下稳定获益，研究还探索了更自适应的稀疏压缩与系统协同设计。例如 PacTrain 将剪枝与自适应稀疏梯度压缩结合，用于更高效的集合通信^[51]。误差反馈也被扩展到更一般的预条件器与优化器状态压缩中^[52]。

2.3.3 低秩与混合压缩

除量化和稀疏化外，低秩压缩也是大规模分布式训练中的一条重要路线。PowerSGD 将梯度矩阵近似分解为低秩因子，只交换近似秩分解所需的子空间信息，从而在保持高精度的同时显著降低通信量^[53]。与逐元素量化或稀疏化相比，低秩压缩更适合具有明显矩阵结构的梯度张量，尤其适用于 Transformer 中的线性层参数更新。但这类方法对矩阵结构与秩近似质量更敏感，通常需要与误差反馈或分层同步结合，才能在复杂训练任务中稳定发挥作用。

综合来看，现有压缩方法已经从单纯减少发送数据量发展为“压缩策略、误差补偿与训练系统协同设计”的复合问题。本文第 3 章聚焦低比特量化路线：在尽量保持训练语义的前提下，将跨域同步通信的字节规模压缩到更低水平，并在后续章节与“分层调度”和“重叠”机制组合，以适配云边端跨域约束。

2.4 集合通信优化与层次化调度

All-Reduce 是 DP 中最关键的集合通信原语之一，其实现方式直接决定了通信带宽利用率、同步尾部和系统可扩展性。经典的集合规约研究已经较早指出，ring、tree 与递

归倍增等算法在不同拓扑和消息规模下具有不同的时间代价^[54]。到了大规模深度学习阶段，这一问题被进一步工程化：Horovod 将 ring-based reduction 作为主流实现路径，证明了简单易用的框架接口与高效的集合通信可以兼得^[55]；NCCL 则成为 GPU 集群中的事实标准，为多 GPU 训练提供了高度优化的底层通信库^[56]。

随着硬件异构性、拓扑层次性和训练负载复杂度不断上升，研究者开始从“如何执行 All-Reduce”转向“如何自动生成和优化集合通信原语”。Blink 通过 packing spanning trees 来生成更适合异构网络的通信原语，表明树形结构在某些场景下能够优于固定实现的 NCCL^[57]；TACCL 则进一步将集合通信优化抽象为算法合成问题，利用 communication sketches 指导 synthesizer 自动生成适配特定拓扑的 collective algorithm，并在 DGX-2 和 NDv2 上证明其可显著超越 NCCL^[58]。此类工作说明，集合通信不再只是固定库调用，而是一个可被拓扑感知、可被自动合成的系统设计空间。

面向训练系统本身，研究者也开始关注集合通信与训练语义、尾部鲁棒性和压缩机制的联合优化。OptiReduce 针对云环境中的落后节点、拥塞和梯度丢包问题，提出了尾部最优的 AllReduce 设计，并通过不可靠传输、Transpose AllReduce 和 Hadamard 变换等机制在性能与精度之间取得折中^[59]。BytePS-Compress 则从“压缩 + 参数服务器”的角度，将双向梯度压缩、CPU 端压缩/解压管线化与高并发传输结合起来，展示了压缩通信与系统实现的协同收益^[60]。本文的研究重点在于充分利用已有通信原语：在跨域、域内带宽高度不对称的前提下，针对分层 All-Reduce 的阶段解耦与调度重排，提升已有原语在云边端协同场景中的有效利用率。

跨域训练研究进一步表明，仅有层次化还不够：当域间 RTT 高且通信启动开销显著时，需要通过流水线化、分块、双流并发等方式提高并行度并隐藏启动开销^[25]。本文第 4 章从工程可实现的角度给出分块与双流s的层次化流水线调度策略。

2.5 通信重叠与同步尾部优化

在常规数据中心环境中，框架通常通过 bucket 化在反向传播中逐步启动 All-Reduce，以便让后续层的反向计算掩盖前面 bucket 的通信时间。这类做法本质上属于经典的计算-通信重叠（computation-communication overlap）范式，目标是在不改变同步语义的前提下，将通信从关键路径中尽可能挤出。对应到工程实现，训练框架往往依赖梯度分桶、通信流与计算流并发、以及后端通信库的异步接口来实现重叠。然而，当训练跨越 WAN 链路时，高 RTT、抖动和带宽不稳定会显著拉长通信尾部，导致最后若干 bucket 无法被剩余计算完全掩盖，最终形成同步尾部和迭代末端空转。

围绕这一问题,研究者首先从“减少同步频率”这一方向寻找折中。**FedAvg** 将本地多步更新与周期性模型平均结合,在一定程度上减少了全局同步的次数^[34];**Local SGD** 则从理论上说明了少通信、多本地更新的收敛可行性^[35-36]。进一步地,**SCAFFOLD** 通过控制变量修正局部漂移,缓解了数据异构条件下因少同步带来的优化偏差^[37]。这类方法与跨域训练高度相关,因为它们指出:只要同步频率足够低,即使跨域链路较慢,系统仍有机会把绝大多数计算放在本地完成,从而将通信成本降到可接受水平。

在更贴近跨域模型训练的系统中,低通信频率与计算重叠开始被显式结合。**OpenDiLoCo** 以全球分布式低通信训练为目标,探索了在跨地域环境下维持训练可行性的系统框架^[38];**Streaming DiLoCo** 则进一步强调通信与计算的重叠,通过流式执行减少同步等待,使得跨域通信不必完全阻塞本地训练过程^[39]。与之相关的 **Compensated Overlap-FedAvg** 则从联邦学习视角将“局部训练 + 重叠补偿”结合起来,说明即使在弱同步条件下,也可以借助补偿机制控制误差积累^[61]。上述工作共同表明,在广域或跨域环境中,单纯依赖传统 **bucket overlap** 往往不足以消除尾部,必须进一步引入低频同步、补偿更新或流式重叠等策略,才能稳定释放系统吞吐。

除降低频率外,另一条关键思路是改变同步触发时机与执行位置,将不可避免的通信从迭代尾部迁移到更早的可重叠窗口中。这个方向的核心在于:只要同步语义允许,梯度同步可以在所有反向计算完成前的更早前缀阶段启动,从而把余下通信尽可能隐藏在后续计算之下。与本文最接近的相关工作是 **Streaming DiLoCo** 这类以重叠通信为核心思想的系统,以及围绕尾部优化的集合通信研究^[39,59]。本文第 5 章提出的“前缀触发的异步 **All-Reduce** + 预测误差修正”机制,正是沿着这一方向进一步推进:它不改变“每迭代一次同步”的宏观训练语义,而是通过提前触发和误差修正,将跨域同步从关键路径尾部剥离,并尽可能转化为可被计算掩盖的后台通信。

综合来看,通信重叠与同步尾部优化在现有研究中,从 **bucket** 计算通信重叠到降低通信频率的弱同步或异步训练,以及在弱同步训练机制上探寻计算通信重叠的进一步优化逐步发展。本文第 5 章正是在这个研究现状上,面向云边端协同的跨域训练场景,聚焦高 **RTT** 与强抖动条件下同步尾部难以隐藏的问题,设计出更适合跨域链路约束的重叠优化机制。

2.6 云边端协同训练范式与研究空白

2.6.1 云边端协同训练问题

结合本文的背景，可以看到，云边端协同训练是一种跨域资源联合训练：

这种跨域性首先体现在网络层：域内链路通常高带宽、低时延，而域间链路普遍低带宽、高 RTT 且抖动更大；其次体现在数据层：训练样本天然分布在云侧数据湖、边缘站点与端侧来源，不宜也不总能集中回传；再次体现在算力层：参与训练的设备来自不同自治域，调度能力和负载上限差异显著。

因此，云边端协同训练的主要矛盾在于如何在异构域间链路约束下组织同步与并行。这一定义也给出本章的阶段性的结论：需要先回答“跨域并行策略如何选择”，再讨论“在既定策略下如何做通信优化”。

2.6.2 云边端场景下的跨域训练策略的选择

综合并行训练与跨域训练研究，可将策略选择归结为两个判断维度：跨域通信对象是否随 batch 增长，以及该通信是否持续位于关键路径。

当采用跨域流水线并行（Cross-domain PP）时，模型 stage 会跨域切分，域间通信对象主要是激活及其反向梯度；这一方案在参数规模受限场景中有助于继续扩展模型，但跨域通信与样本流强耦合，WAN 抖动容易沿流水线级联放大。相比之下，跨域数据并行（Cross-domain DP）将前反向计算留在各域本地，域间交互集中为梯度或参数同步，对网络抖动更鲁棒，但前提是同步路径具有足够高效的通信实现。

本文通过量化推导给出更具体的比较关系。设全局 batch 为 B ，参数规模为 P ，每参数通信字节为 α ，跨域切分时每样本跨域激活字节为 A ：

跨域 DP 的通信量近似为 $V_{DP} \approx \alpha P$ ，在模型固定时对 B 近似不敏感；而跨域 PP 的通信量近似为 $V_{PP} \approx \Theta(B * A)$ ，会随 B 线性增长。

据此可得到如下选择原则：当目标是继续突破单域模型容量上限，且域间链路相对稳定时，可考虑跨域 PP；当目标是在受限 WAN 条件下保证稳定吞吐或为了获取更高吞吐效率，并且为了保证训练语义可控时，应优先跨域 DP。

在更大的 batch 训练下，跨域 PP 的 WAN 压力会持续放大；而跨域 DP 的固定同步开销更容易被计算摊薄。这种情况下，跨域并行训练的选择将逐步收敛到“跨域 DP + 域内 PP”这一分层组织方式：跨域层面优先保证稳定吞吐，域内层面再用流水线并行扩展模型容量与设备利用率^[11-12]。

在此基础上，本文进一步说明为何采用跨域数据并行而非跨域流水线并行。本文的问题设定是“云边端协同下的大规模数据训练”，核心目标是在真实跨域链路约束下提升端到端吞吐并保持优化语义稳定。本文选择跨域 DP 的原因主要有两点：训练使用数据

具有局部性，各域可利用本地数据独立完成计算，避免样本与高频激活长期占用 WAN；训练中通信模式固定，域间同步通信模式固定，便于对通信进行可测量、可治理的优化；

本文将流水线并行和张量并行定位为域内模型扩展手段：在单域内利用高速互联的特性来做 PP/TP 以训练更大的模型，在跨域层面采用 DP，以同时兼顾模型规模扩展、网络鲁棒性与工程可落地性。

2.7 本章小结

在本章节中，介绍了目前国内外在训练通信优化方面的研究。其中包括分布式训练并行策略与系统框架、跨域与跨数据中心训练、梯度压缩与通信量优化、集合通信优化与层次化调度、通信重叠与同步尾部优化，并且，本文分别对上述方向研究现状进行了分析和总结，并且分析了目前云边端协同训练范式的选择，并分析得出了当前在面向云边端协同的分布式训练通信优化上的研究空白。目前主流的梯度压缩量化方法、集合通信优化方法、训练计算-通信重叠方法等无法满足云边端跨域训练的特殊约束：在保持跨域 DP 语义一致的前提下，如何系统性优化通信体积、调度效率与通信重叠来缓解“带宽受限、时延偏高、抖动显著”带来的训练吞吐下降与尾部阻塞。

为解决上述的问题，针对云边端场景下，需要提出梯度压缩与通信量优化、集合通信优化、通信重叠优化三类优化组件并协同运作，构建一个面向云边端协同训练的通信优化系统。

第三章 分布式优化器低比特量化方法

3.1 引言

在分布式深度学习训练中，梯度通信开销已成为制约训练效率的关键瓶颈。传统的 AllReduce 操作需要在各个计算节点间传输完整的 32 位浮点梯度，在大规模模型和多节点环境下，通信时间往往占据训练过程的主要部分^[43]。为降低通信开销，本文引入了 k-bit 随机舍入量化方法，将梯度压缩至可配置的低比特表示 ($b \in \{1, 2, 4, 8, 16\}$)，可灵活调整压缩率，使其与收敛稳定性之间进行折中；其次，引入误差补偿机制与聚合算法来维持模型的优化稳定性，缓解由于量化带来的累计噪声并且实现量化后数据在各节点间的高效聚合。

3.1.1 与云边端协同的关联

云边端协同训练的一个典型形态是：端侧设备持续产生数据，边缘侧承担就近训练/聚合或作为训练入口，云侧提供大规模算力与全局同步能力。在该场景中，训练通信呈现明显的“分层链路”特征：边缘/云内链路可使用较高带宽的局域互联，而端到边、边到云往往受限于接入网与广域网，表现为更低带宽、更高 RTT 以及更大的抖动。

在这种分层拓扑下，梯度同步的瓶颈常由跨域链路主导，尤其当端侧或边缘侧参与数据并行 (DP) 时，同步梯度需要跨“端→边”或“边→云”链路传输。低比特量化的价值因此不仅体现在跨数据中心场景，也天然适用于云边端协同：它以最小的语义改动降低跨域链路上的通信体积，从而直接缓解接入/广域链路的带宽瓶颈，并降低通信排队与尾部等待对端到端时延的放大效应。

3.1.2 分布式训练中的通信瓶颈

随着深度学习模型规模的不断增长，单机训练已无法满足大规模模型的需求。以 GPT-3 为例，其参数量达到 1750 亿，模型大小超过 350GB，单次前向传播就需要数百 GB 的显存^[4]。这使得分布式训练成为必然选择。然而，在分布式训练过程中，梯度同步通信成为了新的性能瓶颈。在标准的数据并行训练模式下，每个节点通过 AllReduce 操作同步梯度。对于包含 N 个参数的模型，每次迭代需要传输的数据量为：

$$\text{Communication Volume} = N \times \text{sizeof(float32)} = N \times 4 \text{ bytes} \quad (3.1)$$

以 ResNet-50 (约 2560 万参数) 为例，每次迭代需要传输约 97.7 MB 数据。而对于 GPT-3 规模的模型，单次通信量约为 651.9 GB (约 667,572 MB)，在带宽受限的跨数据中

心场景下，通信时间可能占据总训练时间的 80% 以上，使得在带宽受限的跨域场景下，通信开销成为主要瓶颈。

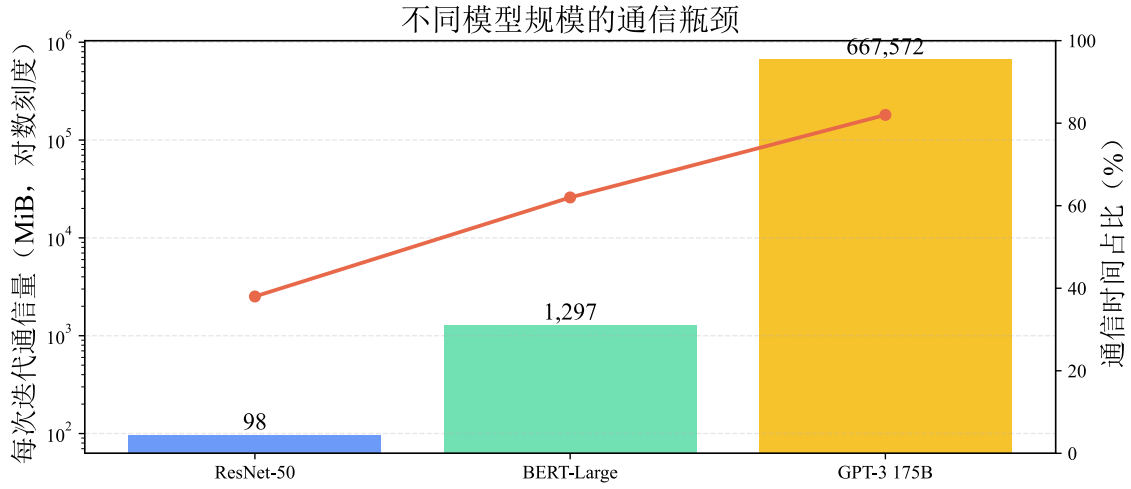


图 3.1 大规模分布式训练中的通信瓶颈

3.1.3 现有梯度压缩方法的局限性

为降低通信开销，学术界和工业界已经提出了多类梯度压缩方法，但在跨域受限链路场景中仍存在明显短板。以梯度稀疏化为例，尽管 Top-k 或阈值筛选能够在数值上显著压缩传输体积，但额外的稀疏索引编码与传输会引入新的开销，并在低带宽、高 RTT 环境中放大同步尾部，导致收敛效率下降^[45]。低精度量化路径则具有更好的工程可用性，FP16 与 BF16 涉及的混合精度训练方法在工业训练中已较成熟，但压缩收益通常只有 2 倍，且其目的通常是为了提升模型本身的计算速度而非提升通信效率；INT8 虽可进一步提升到约 4 倍，却对量化尺度、裁剪区间和反量化策略更敏感，若参数配置不当就可能损伤收敛稳定性^[43]。固定阈值量化方法在实现上最为直接，但由于缺乏对层间梯度尺度差异和训练阶段动态变化的自适应机制，往往难以同时满足高压缩率与稳定优化这两个目标^[44]。

相比之下，本文采用的 k-bit 量化方法结合了 Adam 风格的动量归一化与随机舍入策略，能够在保证收敛性的同时支持多档压缩精度：1-bit 强压缩、2/4-bit 均衡压缩、8/16-bit 低扰动压缩^[41]。

3.2 问题分析与研究思路

在云边端协同分布式训练中，跨域梯度通信的效率成为制约整体训练性能的核心瓶颈。与传统单数据中心内的分布式训练不同，云边端场景下的训练节点往往分布在云端

服务器、边缘节点和端侧设备之间，不同节点之间的网络条件差异显著。尤其是在端到边、边到云等跨域链路上，带宽资源有限、往返时延较高且链路状态容易随网络负载波动而变化，使得梯度同步过程难以像机房内部高速互联环境中那样稳定高效地执行。

3.2.1 跨域训练中梯度通信的困境

在分布式训练中，梯度同步是数据并行的核心操作，但跨域环境下的通信特征与单机房训练有本质区别。云边端协同训练中，端侧和边缘侧节点通过“端→边”或“边→云”的跨域链路进行梯度交换，这些链路普遍表现为低带宽、高 RTT、高波动性。

具体而言，跨域梯度通信面临三重困境：

第一，通信数据量庞大且固定。以 ResNet-50（约 2560 万参数）为例，每次迭代需传输约 97.7 MB 数据；GPT-3 规模模型的单步梯度同步量达 600+ GB。在低带宽链路下（如端→边接入网 10-200 Mb/s），即使单次通信也可能消耗数十秒甚至更长；而在高带宽局域网中（100+ Gb/s）仅需毫秒级。这导致跨域链路成为明显的计算-通信不对称瓶颈：计算可在本地完成，但通信必须经过带宽受限的跨域路径，造成迭代末端的同步阻塞。

第二，现有量化方案的局限性各异。固定精度量化（FP16/INT8）虽然工程成熟，但压缩率上限固定（通常 2-4 倍），无法应对极端低带宽场景；稀疏化方案（Top-k）虽然压缩率更高，但需维护索引、编码、解包等额外开销，并且大多数的 Top-k 方案不支持高效的规约方法。在高 RTT 条件下这些开销会被显著放大，反而可能增加同步尾部；低秩压缩对梯度矩阵结构敏感，难以在异构网络层间梯度差异大的场景中保持稳定性。更关键的是，现有方法通常采用静态量化参数，不能根据链路条件动态调整压缩强度。

第三，量化噪声与收敛稳定性的权衡困难。低比特量化必然引入量化误差，若压缩位宽过低（1-2 bit），量化噪声会显著破坏梯度信息，导致训练末端振荡甚至发散；若位宽过高（16 bit 接近 FP16），压缩收益有限。而不同任务、不同训练阶段对量化容忍度不同：视觉分类相对鲁棒，语言模型对梯度精度更敏感。缺乏误差补偿机制的方案难以平衡这一矛盾。

3.2.2 现有量化方法的不足与启示

综合现有方案，可以发现它们各有所长但均存在明显短板。固定精度量化提供了稳定性，但在极限压缩场景下效果递减；稀疏化提供了高理论压缩率，但工程复杂度高且在 WAN 中开销反而放大；低秩方法在某些模型结构上高效，但通用性受限；现有方法工程上缺乏自适应调整压缩策略的机制来在网络状态变化时动态调整策略；此外，大多数方

法仅关注压缩算法优化，缺乏对压缩算法和通信算法的协同设计。这为跨域分布式训练通信压缩策略提供了一个明确的研究方向。设计一种可调、低开销的量化方案，能够在不同位宽之间灵活切换来适应云边端环境下不同的网络链路条件，并通过结合完善的误差补偿机制来保证收敛的稳定性，同时实现专用于低位宽的集合通信算法来满足量化后的数据的高效传输，从而实现在云边端环境下对通信量化策略进行高效的端到端部署。

3.2.3 本文核心研究问题

基于上述分析，本章聚焦于以下核心问题：

(1) 如何设计一种可调、低开销的梯度量化方案，能在不同位宽（1-16 bit）之间灵活切换，使系统能根据链路条件自主选择最优压缩强度，而不是被动采用固定参数。

(2) 如何通过误差补偿保证收敛稳定性，使得低比特量化不会过度破坏梯度信息，即使在 1-4 bit 的强压缩条件下也能维持可接受的优化轨迹。

(3) 如何最小化量化与打包的工程开销，在不侵入现有分布式训练框架的前提下，实现高效的端到端部署。

3.2.4 算法核心思想与解决方案

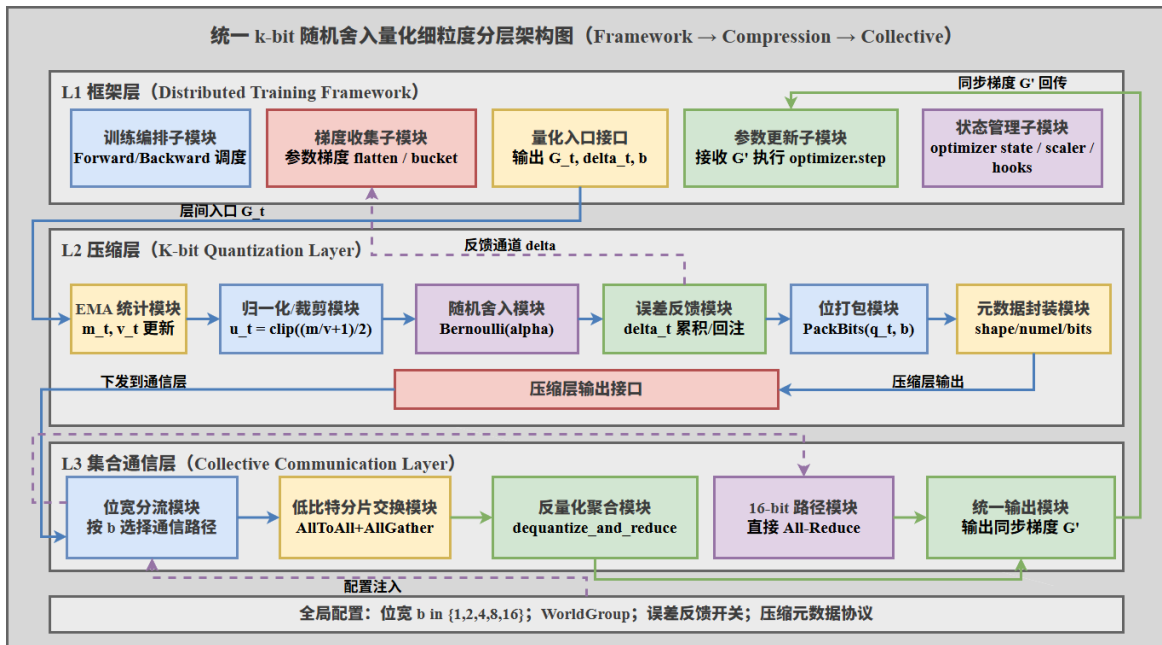


图 3.2 k-bit 随机舍入量化核心工作流程

针对上述问题，本文提出的分布式低比特量化方法（后简称为 k-bit 量化方法）借鉴了 Adam 优化器的动量机制，通过维护梯度的一阶矩估计 m 与二阶矩估计 v ，将连续的浮点梯度映射至 2^b 个离散量化级别。该方法首先利用指数移动平均 (EMA) 平滑历史梯度

的统计特性，随后通过一阶矩与二阶矩的比值实现自适应归一化，这一设计使得训练过程中不同网络层以及不同迭代周期的梯度均能被动态映射至统一的数值区间，适应梯度尺度的变化。在此基础上，本方法采用相邻量化级别间的随机舍入（Stochastic Rounding）以确保量化分布的无偏性，并将当前迭代产生的局部量化残差保存，作为补偿项注入后续迭代过程。这一误差补偿通道有效抑制了低比特截断带来的累积量化噪声，使得算法在降低传输开销与维持模型优化稳定性之间构建了可控的平衡。此外，本方法通过重新设计面向低比特的集合通信，使得量化数据得以高效的进行节点间规约。

k-bit 量化的整体工作流程如图 3.2 所示，包括动量维护、自适应归一化、随机舍入与残差补偿四个关键步骤。这一流程在每次迭代中循环执行，确保量化误差被逐步补偿而不会累积。

3.2.5 数学表达

设第 t 次迭代的梯度为 g_t ，量化比特宽度为 b ，量化过程可表示为：

$$\begin{aligned}
 m_t &= \beta \cdot m_{t-1} + (1 - \beta) \cdot g_t \\
 v_t &= \beta \cdot v_{t-1} + (1 - \beta) \cdot |g_t| \\
 u_t &= \text{clip}\left(\frac{1}{2}\left(\frac{m_t}{v_t + \varepsilon} + 1\right), 0, 1\right) \\
 s &= 2^b - 1 \\
 y_t &= s \times u_t \\
 l_t &= \lfloor y_t \rfloor, \quad \alpha_t = y_t - l_t \\
 z_t &\sim \text{Bernoulli}(\alpha_t), \quad q_t = l_t + z_t
 \end{aligned} \tag{3.2}$$

其中， β 表示动量系数（默认取 0.999）， ε 表示数值稳定项（默认取 10^{-8} ）， $b \in \{1, 2, 4, 8, 16\}$ 表示量化位宽，而 $q_t \in \{0, 1, \dots, 2^b - 1\}$ 则对应离散化后的量化索引。

随机舍入后的归一化估计值为 $\hat{u}_t = \frac{q_t}{s}$ ，其对应的对称区间表示为 $\hat{g}_t = 2\hat{u}_t - 1$ 。由于 $E[z_t] = \alpha_t$ ，可得 $E[\hat{u}_t] = u_t$ ，从而保证量化无偏性。

3.3 详细方案设计与实现

3.3.1 量化过程

梯度量化算法的核心流程如算法 3.1 所示。该算法接收待量化的梯度张量列表作为输入，输出压缩后的 k-bit 张量列表。

算法 3.1: k-bit 随机舍入梯度量化算法

输入: 梯度张量列表 $G = \{g_1, g_2, \dots, g_n\}$, 位宽 b , 动量系数 β , 误差补偿标志 comp_flag
输出: 压缩张量列表 $C = \{c_1, c_2, \dots, c_n\}$

```

1: for 每个梯度张量  $g_i \in G$  do
2:   if 动量状态未初始化 then
3:      $m_i \leftarrow g_i \times (1 - \beta)$ 
4:      $v_i \leftarrow |g_i| \times (1 - \beta)$ 
5:   else
6:      $m_i \leftarrow \beta \times m_i + (1 - \beta) \times g_i$ 
7:      $v_i \leftarrow \beta \times v_i + (1 - \beta) \times |g_i|$ 
8:   end if
9:    $u_i \leftarrow \text{clip}((m_i/(v_i + \varepsilon) + 1)/2, 0, 1)$ 
10:  if  $\text{comp\_flag} = \text{True}$  then
11:     $u_i \leftarrow \text{clip}(u_i + \delta_i, 0, 1)$ 
12:  end if
13:   $s \leftarrow 2^b - 1; y_i \leftarrow s \times u_i; l_i \leftarrow \lfloor y_i \rfloor$ 
14:   $\alpha_i \leftarrow y_i - l_i; z_i \sim \text{Bernoulli}(\alpha_i)$ 
15:   $q_i \leftarrow l_i + z_i$ 
16:   $\delta_i \leftarrow \delta_i + (u_i - q_i/s)$ 
17:   $c_i \leftarrow \text{PackBits}(q_i, b)$ 
18: end for
19: return  $C$ 

```

从实现逻辑看, 算法首先在第 2-8 行完成一阶矩与二阶矩的动量维护, 以指数移动平均方式持续吸收历史梯度统计; 随后在第 9 行通过二者比值进行自适应归一化, 将不同尺度的梯度映射到统一的 $[0, 1]$ 区间。若启用补偿机制, 第 10-12 行会把历史量化残差 δ 回注到当前归一化值并进行截断约束, 从而抑制误差累积。第 13-15 行执行基于伯努利采样的随机舍入, 得到离散索引 $q_i \in \{0, 1, \dots, 2^b - 1\}$, 该过程保障了量化的统计无偏性; 最后在第 17 行按位宽进行紧凑打包, 减少通信与存储开销。

3.3.2 位打包优化

为进一步减少内存占用, 本实现采用通用位打包技术, 如图 3.3 所示。对于位宽 b , 每个量化值占用 b bit, 原始 float32 梯度可获得理论压缩比 $\frac{32}{b}$ [42]。

具体实现时, 系统首先执行填充对齐操作。由于量化后的索引以 b bit 为单位存储, 而底层内存与通信接口通常以字节为最小访问单元, 因此当所有量化索引展开后的总 bit 数不是 8 的整数倍时, 需要在比特流末尾补充若干个 0, 使其对齐到完整字节边界。该

填充过程仅用于满足字节级存储和传输要求，不参与后续梯度恢复计算，因此不会改变原始量化索引的有效信息。

完成对齐后，系统将每个量化索引拆分为长度为 b 的二进制表示，并按照连续比特流的形式依次拼接。随后构造字节内的位权向量 $\mathbf{w}$ ，其中每个元素表示对应 bit 在一个字节中的权重。对于一个 8 bit 字节而言，位权向量可表示为：

$$\mathbf{w} = [2^7, 2^6, 2^5, 2^4, 2^3, 2^2, 2^1, 2^0] = [128, 64, 32, 16, 8, 4, 2, 1] \quad (3.3)$$

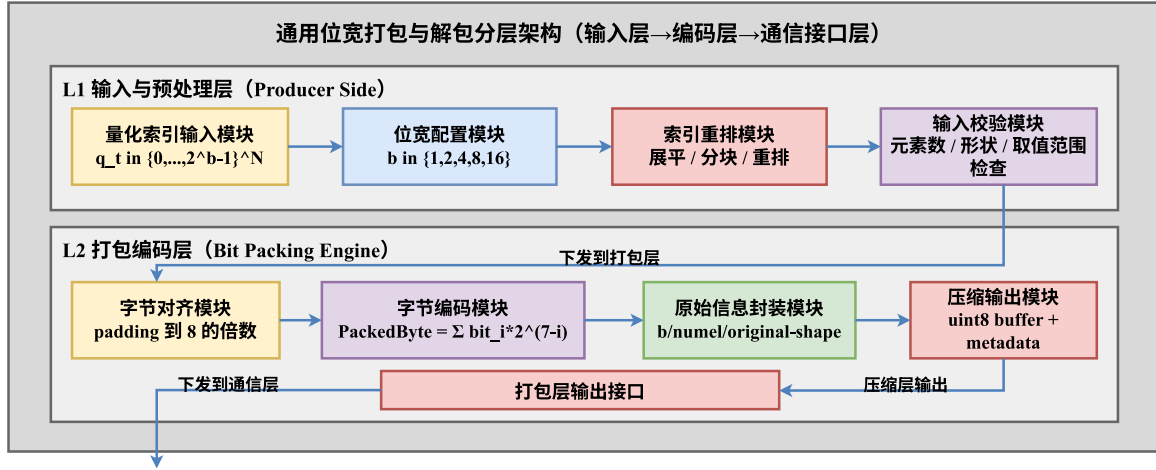


图 3.3 位打包与解包过程示意图

在此基础上，量化索引会被展开为 bit 流并按字节分组，与位掩码逐元素运算后得到 uint8 表示：

$$\text{PackedByte} = \sum_{i=0}^7 \text{bit}_i \times 2^{7-i}, \quad \text{bit}_i \in \{0, 1\} \quad (3.4)$$

该方法适用于不同位宽。若忽略对齐与元数据开销，通信量相对 float32 的理论压缩比分别为：1-bit 为 32 ×，2-bit 为 16 ×，4-bit 为 8 ×，8-bit 为 4 ×，16-bit 为 2 ×。

因此，该方法可在强压缩（1/2-bit）与高保真（8/16-bit）之间灵活切换，满足不同网络条件下的训练需求。

3.3.3 反量化与聚合

在接收端，各节点需要将来自所有 worker 的压缩梯度解包、聚合并恢复到可用于优化器更新的梯度空间。解包过程则与打包过程相反。接收端首先读取压缩后的字节数组，并根据位权向量将每个字节还原为 8 bit 的二进制序列；随后按照量化位宽 b 对比特流重新分组，恢复出每个元素对应的量化索引。由于发送端在打包前记录了原始张量形状以

及有效量化元素数量，接收端可以在解包后去除末尾填充产生的无效 bit，并将量化索引重新 reshape 为原始梯度张量的形状。最终，系统再根据量化尺度与反量化规则将索引恢复为近似梯度值，用于后续规约或参数更新。总的来说，首先压缩字节会被恢复为离散索引张量 $q \in \{0, 1, \dots, 2^b - 1\}$ ；实现上通过位运算完成高效解包，即将每个 uint8 字节展开为 8 位矩阵，再与位掩码 $\mathbf{w} = [128, 64, 32, 16, 8, 4, 2, 1]$ 做按位与，并按位宽 b 重组为量化索引序列。该过程可写为：

$$q = \text{UnpackBits}(\text{PackedBytes}, b), \quad q \in \{0, 1, \dots, 2^b - 1\}^N \quad (3.5)$$

完成解包后，系统收集来自所有 W 个 worker 的量化索引并执行逐元素累加：

$$Q_{\text{sum}} = \sum_{w=1}^W q^{(w)} \quad (3.6)$$

其中 $q^{(w)} \in \{0, 1, \dots, 2^b - 1\}^N$ 表示第 w 个节点的量化索引， Q_{sum} 则对应参数维度上的离散级别和。随后进入反归一化阶段，将聚合后的离散值映射回原始梯度空间。设 $s = 2^b - 1$ ，其反向映射可写为：

$$g_{\text{recovered}} = 2 \times ((Q_{\text{sum}}/W)/s) - 1 \quad (3.7)$$

该公式将离散索引均值线性映射回 $[-1, 1]$ 区间，从而恢复梯度的符号与相对尺度信息。最终得到的 $g_{\text{recovered}}$ 可作为梯度的无偏估计参与参数更新。

3.3.4 集成到分布式训练框架

k-bit 量化方法已集成到分布式训练流程中，用于替代传统数据并行训练中的梯度 AllReduce 通信阶段。与修改模型结构或优化器更新规则的方案不同，本文方法主要作用于反向传播之后、参数更新之前的梯度同步环节，因此能够在保持原有训练语义基本不变的前提下，对通信过程进行压缩优化。其分布式训练流程如算法 3.2 所示。在每个训练批次中，各计算节点首先独立完成前向传播与反向传播，得到本地梯度；随后系统根据配置判断是否启用量化压缩通信。若未启用压缩，则流程退化为标准分布式数据并行训练，即直接对梯度执行 AllReduce；若启用压缩，则调用 QuantizedAdamReduce 对梯度进行量化、通信、反量化与规约，并将规约后的梯度返回给优化器用于参数更新。

这种集成方式的核心优势在于将量化压缩逻辑封装为独立的通信原语，使其在训练循环中呈现出与 AllReduce 类似的调用接口。对于上层训练代码而言，模型定义、损失计算、反向传播和优化器更新过程均无需大幅修改，只需要在梯度同步阶段将原始通信接口替换为量化通信接口即可。因此，该方法能够较好地兼容现有基于 PyTorch Distributed

的训练框架，并支持通过配置项灵活切换标准通信路径与低比特通信路径。这种低侵入式集成方式降低了方法的部署成本，也便于在不同模型、不同任务和不同网络条件下开展对比实验。

算法 3.2: 集成量化的分布式训练循环

```

1: for 每个训练批次 do
2:   前向传播:  $y = f(x; \theta)$ 
3:   计算损失:  $\mathcal{L} = \text{Loss}(y, y_{\text{true}})$ 
4:   反向传播:  $\nabla \leftarrow \nabla_{\theta} \mathcal{L}$ 
5:   if 启用量化压缩 then
6:      $\nabla \leftarrow \text{QuantizedAdamReduce}(\nabla, b)$ 
7:   else
8:      $\text{AllReduce}(\nabla)$ 
9:   end if
10:  参数更新:  $\theta \leftarrow \theta - \eta \times \nabla$ 
11: end for
  
```

在通信流程内部，本文采用分支式通信策略，以适配不同量化位宽下的通信需求。当量化位宽 $b < 16$ 时，系统进入低比特压缩通信路径。该路径首先初始化 k -bit Adam 量化器，并根据当前位宽对梯度张量进行量化，将原始 float32 梯度转换为低比特量化索引；随后通过位打包操作将低比特索引压缩为连续字节流，以减少实际传输的数据量；最后利用 All-to-All 通信完成压缩数据在各进程之间的交换，并在接收端执行分片反量化与规约，恢复出聚合后的梯度结果。该过程将“压缩、传输、恢复、规约”封装在统一的通信接口中，使低比特梯度能够以端到端方式参与分布式训练。

当 $b = 16$ 时，系统不再进入低比特压缩聚合路径，而是直接执行标准 AllReduce。这样设计主要基于两方面考虑：一方面，16 bit 情况下压缩收益相对有限，继续引入低比特打包、A2A 交换和反量化规约可能无法带来明显通信优势；另一方面，标准 AllReduce 已在主流分布式训练框架和通信库中经过充分优化，在中高位宽场景下通常具有更好的工程稳定性和执行效率。因此，本文将 $b = 16$ 作为标准通信路径的边界条件，使系统能够在低位宽强压缩场景下发挥量化通信优势，同时在高位宽场景下保留成熟通信后端的性能和可靠性。

量化通信流程如算法 3.3 所示。输入为梯度张量列表 G 、量化位宽 b 以及全局进程组 WorldGroup，输出为聚合后的梯度张量列表 G' 。在低比特路径中，系统对每个压缩张量分别分配 A2A 接收缓冲区，并将本地压缩后的梯度分片发送给其他进程。接收端获得

来自不同进程的压缩分片后，根据量化器记录的尺度信息、张量形状和残差补偿状态执行反量化，并在本地完成对应梯度片段的规约。由于通信过程中传输的是打包后的低比特索引，而不是完整浮点梯度，因此跨节点传输量能够随位宽降低而近似线性下降。对于 $b = 1, 2, 4, 8$ 等设置，理论通信量分别约为原始 float32 梯度的 $\frac{1}{32}$ 、 $\frac{1}{16}$ 、 $\frac{1}{8}$ 和 $\frac{1}{4}$ ，从而能够显著缓解低带宽跨域链路中的同步阻塞问题。

算法 3.3: 通信流程

输入： 梯度张量列表 G ，位宽 b ，全局进程组 WorldGroup

输出： 聚合后的梯度张量列表 G'

```

1: if  $b < 16$  then
2:   初始化量化器:  $Q \leftarrow \text{KBitAdamQuantizer}(b)$ 
3:   量化梯度:  $C \leftarrow Q.\text{quantize}(G)$ 
4:   for 每个压缩张量  $c_i \in C$  do
5:     分配 A2A 接收缓冲区:  $\text{RecvBuf} \leftarrow \text{allocate\_a2a}(|c_i|)$ 
6:     A2A 通信:  $\text{AllToAll}(\text{RecvBuf}, c_i, \text{WorldGroup})$ 
7:     分片反量化并规约:  $g'_i \leftarrow Q.\text{dequantize\_and\_reduce}(\text{RecvBuf})$ 
8:     更新梯度:  $G'[i] \leftarrow g'_i$ 
9:   end for
10:  return  $G'$ 
11: else
12:   直接同步:  $G' \leftarrow \text{AllReduce}(G, \text{WorldGroup})$ 
13:  return  $G'$ 
14: end if

```

从工程实现角度看，该设计兼顾了透明性、可扩展性与可维护性。透明性体现在量化通信接口对上层训练流程屏蔽了内部的量化、打包、通信和反量化细节，使训练脚本只需要感知“是否启用量化”和“采用何种位宽”两个核心配置；可扩展性体现在量化器与通信流程之间保持相对解耦，后续可以替换为其他量化、稀疏化或误差补偿策略，而无需重写完整训练循环；可维护性则体现在标准 AllReduce 路径被完整保留，当网络环境较好、模型对精度敏感或需要进行基线对照时，系统可以直接退回成熟通信实现。由此，本文方法不仅能够支撑低带宽跨域场景下的高压缩通信，也能够作为一个可配置通信模块嵌入现有分布式训练系统，为后续算法迭代和实验评估提供统一接口^[16]。

3.4 实验与验证

本节旨在验证本文提出的 k-bit 低比特量化方法在多模型、多网络条件的实际性能。

实验覆盖了视觉、语言及自回归模型，以评估端到端训练收益与收敛稳定性。同时，实验设计考虑了从高带宽数据中心链路到受限云边端链路的不同网络条件。通过系统侧和优化侧指标的联合分析，可全面体现量化方法在真实训练场景中的优势与局限。

3.4.1 实验设置

本节将实验设计扩展为三条主线：

主线 A（方法对比）：在统一训练脚本与并行配置下，横向比较 FP32 All-Reduce、FP16 All-Reduce、INT8 量化、稀疏化（Top-k）以及本文 k-bit 随机舍入量化。

主线 B（网络分组）：覆盖从数据中心内高带宽到云边端跨域低带宽的多组链路条件，显式考察带宽、RTT 与抖动变化对通信尾部与吞吐的影响。

主线 C（多模型端到端）：在视觉、语言模型三个层面评估端到端训练收益，避免结论只对单一模型成立。

其中，网络环境分为环境 A（数据中心内高带宽、低 RTT）与环境 B（云边端跨域、受限带宽）两大类。环境 B 采用网络整形（限速、RTT 注入与抖动注入）复现实网约束，用于验证低比特通信在跨域链路路上的稳定增益。

为保证不同任务之间的结果可比较，本章实验在统一训练脚本、通信后端与日志统计口径下完成。除特别说明外，系统侧统一记录通信字节数、通信时间、尾部等待与吞吐，优化侧统一记录按训练迭代次数对齐和 墙钟时间对齐的训练曲线；跨域链路条件均通过网络整形方式注入，并在同一组基线方法下比较。这样可以避免把模型差异、网络差异和通信策略差异混在一起解释，使后续多模型结果能够对应到相同的实验口径。

表 3.1 第三章实验硬件环境

环境	计算节点与 GPU	节点内互联	节点间互联
实验环境一	2 节点 NVIDIA A100 云集群	PCIe 全互联，约 126 Gb/s	IB 网口，约 50 Gb/s
实验环境二	2 节点 Iluvatar BI-V150 x 8	同 NUMA PCIe，板内芯粒成对互联，约 126 Gb/s	Ethernet，约 12.5 Gb/s

本章实验涉及两类硬件环境，如表 3.1 所示。实验环境一为 NVIDIA A100 云上双节点集群，节点内 GPU 通过 PCIe 全互联，网卡使用 IB 网口；节点内 PCIe 有效带宽约为 126 Gb/s，节点间互联带宽约为 50 Gb/s。实验环境二为天数智芯（Iluvatar CoreX）双节点国产 GPU 物理集群；每节点 CPU 为 Intel Xeon Gold 6330，GPU 为 Iluvatar BI-V150 x 8，板卡内部各包含 2 个 GPU 芯粒。该环境中 GPU 两两成对走板内集成互联路径，每

对 GPU 之间经 PCIe Switch 互联，GPU 与 NIC0 挂载在不同 NUMA 节点上；节点间通过 Ethernet 互联，节点内全 PCIe 互联带宽为 126 Gb/s，节点间互联带宽为 12.5 Gb/s。

表 3.2 云边端协同场景的分层链路参数

链路	带宽（示例）	RTT（示例）
端→边（接入网）	10 - 200 Mb/s	5 - 30 ms
边→云（广域网）	10 - 30 Gb/s	30 - 100 ms
边/云域内（局域互联）	100 - 200 Gb/s	< 1 ms

表 3.3 对比实验方法集合

对比类别	方法	实现说明
无压缩基线	FP32 All-Reduce	标准分布式同步路径，作为准确性与稳定性参考
低精度基线	FP16 All-Reduce	工业常用低精度同步，压缩比约 2x
量化基线	INT8 量化同步	固定 8-bit 量化方案，压缩比约 4x
稀疏化基线	Top-k + 误差反馈	稀疏通信代表方案，关注索引开销与尾部效应
本文方法	k-bit（1/2/4/8/16）	动量归一化 + 随机舍入 + 位打包

表 3.4 多网络条件实验分组

分组	目标场景	带宽区间	RTT 区间
N1	机房/数据中心内	100 - 200 Gb/s	< 1 ms
N2	跨可用区或同城跨域	10 - 30 Gb/s	5 - 30 ms
N3	边→云广域链路	1 - 10 Gb/s	30 - 100 ms
N4	端→边接入受限链路	10 - 200 Mb/s	5 - 30 ms

表 3.5 多模型端到端实验矩阵

模型	任务类型	端到端指标	对比重点
ResNet-50	视觉分类	吞吐指标（items/s）、收敛轮数、目标精度时间	低比特量化对吞吐与最终精度的影响
BERT-Large	语言理解	吞吐指标（items/s）、loss-vs-time、最终验证集指标	中等规模语言模型的稳定性与速度平衡
LLaMA-7B	自回归语言建模	吞吐指标（tokens/s）、目标损失到达时间、端到端训练时长	大模型场景下通信压缩收益与收敛可行性

如表 3.2，表 3.3，表 3.4 中所示：这些表格明确展示了实验所用的网络条件、对比方法以及多网络分组策略。通过覆盖不同带宽与 RTT 区间，可以评估低比特量化在受限

链路下的端到端性能表现。对比方法集合保证了横向对比的公平性，从无压缩到不同压缩策略均纳入实验。多模型端到端矩阵确保了结果在视觉、语言及自回归模型上均具代表性，便于验证方法的通用性。。

评价指标：系统侧统计每步通信字节数、通信耗时、P95/P99 尾部等待和 GPU 利用率；优化侧统计 loss-vs-step、loss-vs-time、目标精度/目标 loss 到达时间（time-to-target），用于联合评估“是否更快且是否稳定”。本文统一以吞吐指标（items/s）表示吞吐：视觉任务对应 samples/s，语言任务对应 tokens/s。

3.4.2 实验结果与分析

不同网络条件下的量化性能与收敛性分析：

基于端到端实验的评估数据，如图 3.7 所示，k-bit 量化方法在网络带宽逐步受限的场景中表现出明显的吞吐提升趋势。以 $b = 4$ 的配置为例，相较于无压缩的 FP32 全精度基线，其在 10,000 Mb/s、1,000 Mb/s 和 100 Mb/s 三档典型通信带宽下，所取得的端到端吞吐加速比分别达到约 $1.28 \times$ 、 $1.58 \times$ 与 $2.43 \times$ 。这一性能差异表明：在云边端跨域的受限链路环境中，物理传输载荷构成了迭代循环的关键瓶颈，而低比特通信压缩技术能够有效缩减全局同步网络等待时间，从而提升系统的数据处理效率。

从模型优化收敛维度审视，8-bit 与 4-bit 量化配置在最终模型损失（Loss）预测上与全精度 FP32 轨迹相近，未表现出明显的精度退化。然而，1-bit 压缩策略尽管提供了更高的通信效率，却伴随了更为明显的优化精度损耗，并引发了训练末端损失值的抖动偏移现象。该结果刻画了分布式量化通信的权衡关系：更低的量化位宽对应更高的传输效率，但也同时引入更强的随机扰动。

从机制上看，上述结果与通信时延分解模型一致。在每步通信时延可写为 $T_{\text{comm}} \approx T_{\text{startup}} + \frac{V}{\{BW\}}$ 的近似条件下，低比特量化主要作用于 $\frac{V}{\{BW\}}$ 项：当带宽较高时， T_{startup} 占比上升，压缩对总时延的边际收益有限；当带宽下降时， $\frac{V}{\{BW\}}$ 项快速放大，压缩收益随之显著上升。因此，同一压缩策略在 N1/N2 与 N3/N4 网络区间下表现出的增益差异是链路受限程度变化导致的结构性结果。

进一步地，4-bit 在多数网络条件下呈现收益-稳定性较优平衡，说明其在本实验设置下能够同时满足两类约束：一方面压缩率足以降低跨域同步等待，另一方面量化噪声未显著破坏优化轨迹。相对地，1-bit 在极低带宽下虽然系统侧收益更高，但会引入更强估计扰动，导致最终 loss 与收敛速度出现偏移。这一现象支持本文在方法选择上的核心结论：跨域受限场景并不总是压缩越强越好，而应在通信收益与优化稳定之间做联合选择。

该组实验还揭示了一个工程意义上的阈值效应：当网络从高带宽退化到低带宽，使得通信主导迭代时间占比上升后，压缩策略对端到端性能的影响会从次要优化项转变为主导优化项。这意味着在云边端真实部署中，压缩策略不应固定为单一位宽，而应根据链路状态和任务阶段进行策略切换，例如预热阶段采用保守位宽、瓶颈阶段采用激进位宽，以获得更稳定的全程收益。

综合上述分析，本组结果不仅验证了 k-bit 在受限网络下可显著改善系统吞吐，也支撑了按链路条件选择位宽的方法。结论层面可归纳为：低比特量化在跨域链路受限时具备明确有效性，但其最优配置依赖网络条件与训练稳定性目标，需通过系统侧指标与优化侧指标联合决策。

与其他压缩方法的对比结果：除位宽内部对比外，本章补充采用表 3.3 中的方法集合进行横向评估。对比实验统一报告以下三类结果：

系统效率：迭代时间、通信时间占比、P95/P99 尾部等待与 GPU 利用率；

收敛质量：相同 step 下的验证指标、相同 wall-clock 时间下的 loss 下降速度；

综合收益：达到同等目标精度的总训练时长（time-to-target）。

该对比用于评估两个方面：k-bit 相比固定精度量化（FP16/INT8）是否具备更好的可调压缩-收敛平衡，以及相较稀疏化方案是否在跨域链路下具有更低的索引开销与更稳定的尾部时延。

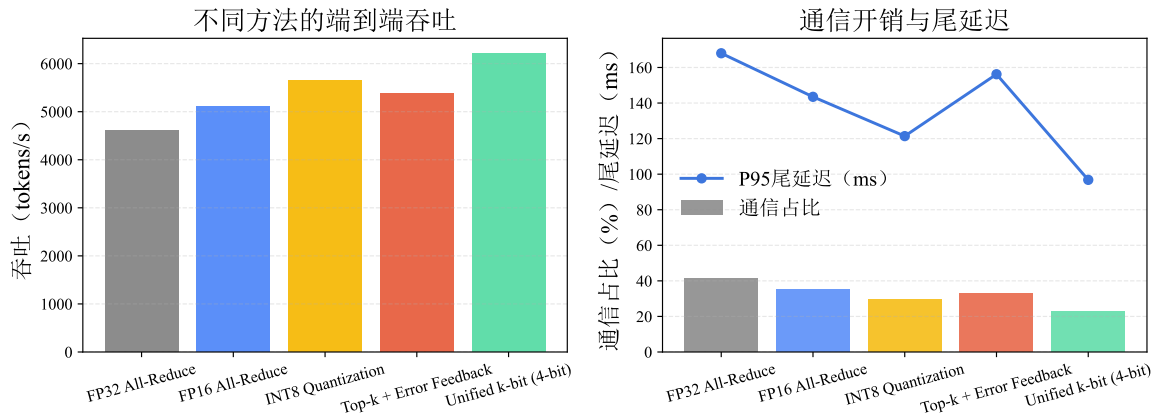


图 3.4 k-bit 与主流压缩方法的端到端性能对比

如图 3.4 所示，在统一训练配置下，k-bit（4-bit）相较 FP32 的吞吐提升约为 $1.35 \times$ （6216 vs 4602，单位为 items/s），通信时间占比由 41.2% 降至 22.5%，P95 尾部时延由 168.0 ms 降至 96.8 ms。与 FP16、INT8 及 Top-k 基线相比，k-bit 在吞吐与尾部稳定性上均取得更优平衡，说明其在跨域受限链路下更能稳定释放系统吞吐。

从对比结构上看，四类基线方法分别对应不同技术路径：FP16/INT8 属于固定精度量化，Top-k 属于稀疏化压缩，k-bit 则采用“动量归一化 + 随机舍入 + 位打包”的一体化方案。实验结果显示，k-bit 在通信占比与尾部时延两个关键指标上同时占优，说明该方案在系统与优化目标之间也实现了更可控的折中。

其原因可从两方面解释。第一，固定精度量化（如 INT8 等）虽然实现成熟，但压缩率上限受限，在跨域低带宽场景下难以继续压缩尾部；第二，稀疏化方案虽然压缩率较高，但索引维护、打包与重组、规约聚合开销会在高 RTT 条件下被放大，且稀疏模式波动会增加尾部抖动。k-bit 通过位打包与无偏随机舍入，降低了索引型开销与偏差积累风险，因此更稳定。

从训练稳定性角度看，最终 loss 的差异幅度处于可控范围，表明系统侧收益并非由显著牺牲优化行为换取。尤其在与 Top-k 路径比较时，k-bit 在尾部时延和最终损失上均表现更稳，支持本文关于“量化路径在跨域场景中更具工程可部署性”的结论。

本组对比还给出一个可落地的实践建议：在跨域链路受限且对收敛质量有约束的训练任务中，优先选择具备误差补偿与可调位宽能力的量化方案；对于极端带宽受限但可容忍收敛扰动的场景，再考虑更激进压缩配置。该建议与本文方法设计目标一致，即在保障训练语义可控的前提下释放通信优化收益。

多模型端到端训练结果：

针对表 3.5 的三类模型，本章以端到端训练耗时与到达目标精度时间作为主指标。实验重点在于完整训练流程中的真实收益，包括前后向计算、梯度同步、优化器更新与数据流水线等环节共同作用后的整体加速比。

该设置说明了本方法可在多任务不同模型场景下发挥作用。

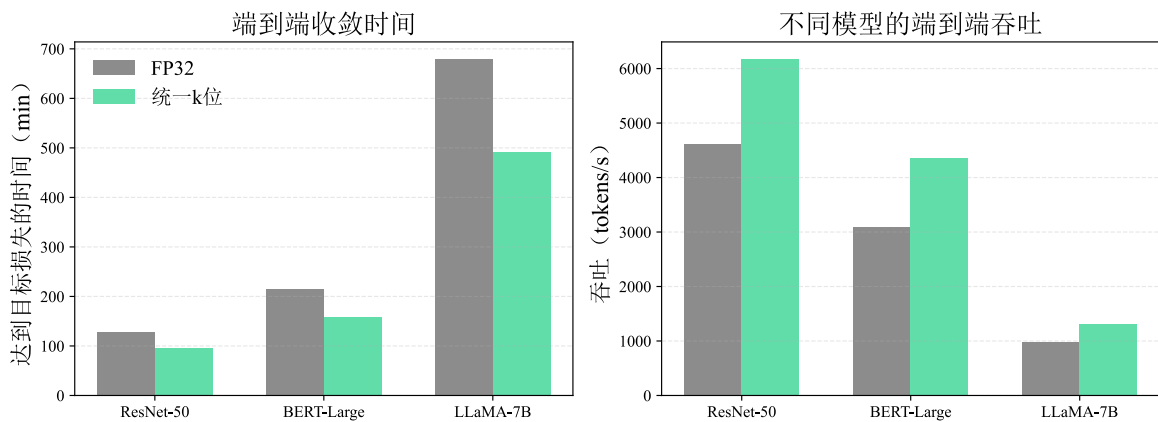


图 3.5 k-bit 在多模型端到端训练中的收益

图 3.5 实验取通信链路带宽为 3 Gb/s 进行实验。该带宽设置处于云边端跨域训练常见的中低带宽区间，能够较好反映压缩通信在真实受限链路中的收益，因此适合作为端到端对比的统一参考条件。

从图 3.5 可见，k-bit 在 ResNet-50、BERT-Large 与 LLaMA-7B 三类负载上均带来稳定端到端收益：time-to-target 分别由 128/214/680 min 降至 95/158/492 min；吞吐由 4620/3090/980 提升至 6180/4350/1320（其中 ResNet 和 BERT 单位为 samples/s，LLaMA-7B 单位为 tokens/s）。这说明低比特量化带来的收益并不局限于单一模型结构，而是能够随着训练规模和序列长度的变化持续体现为 wall-clock 级别的节省。

该组实验的核心价值在于跨任务一致性验证。视觉分类、语言理解与自回归建模在数据访问模式、梯度统计特征与训练稳定性敏感点上存在显著差异；若某通信优化仅对单一任务有效，通常难以在三类任务中同时保持正增益。当前结果显示三类任务的 time-to-target 均明显缩短，说明 k-bit 的收益主要来自通信路径效率提升这一跨任务共性。

进一步比较三类任务的收益幅度可以看到，大模型与长序列任务的绝对节省时间更为显著。这与其更高通信负载和更长训练时间窗口相一致：当通信在总时长中的占比较高时，压缩策略对 wall-clock 的改善更容易累积为可观的总时长缩减。换言之，k-bit 在通信主导型工作负载中的边际收益更大，这一趋势与前述网络受限实验中的结论相互印证，也说明第 3 章的方法更适合被部署在云边端协同训练中“通信占比高、同步频繁”的训练阶段。

同时，该组结果并未显示任务切换导致的稳定性失效，说明所采用的误差补偿机制在不同任务分布下具备一定鲁棒性。对工程部署而言，这一性质非常关键：它意味着策略可在同一系统中服务多类训练任务，而无需为每类任务完全重写通信路径，降低了运维和调参成本。

需要指出的是，不同任务之间的最优位宽可能并不完全一致。本章给出的统一配置验证了可行且有效，但若追求最优收益，仍可在保持统一框架的前提下按任务进行位宽微调。该观察进一步说明本文方法的优势在于提供了可调、可扩展的统一通信优化底座，而图 3.6 所展示的 loss 曲线则从优化侧补充了这一结论。

如图 3.6 所示，在等效跨域带宽约 3 Gb/s 的条件下，4-bit 曲线与 FP32 在主要训练阶段保持接近，且末段验证损失差值维持在约 0.02 量级。这表明压缩通信带来的吞吐收益没有明显破坏主训练路径中的优化过程，验证损失的走势仍与基线保持同阶。更重要的是，图 3.6 与前述吞吐结果形成了互补关系：前者解释了压缩后是否能够稳定收敛的

问题，后者解释了在多类任务上能够节省多少时间。两者合起来说明，k-bit 在可接受的误差扰动范围内提升了端到端训练效率。

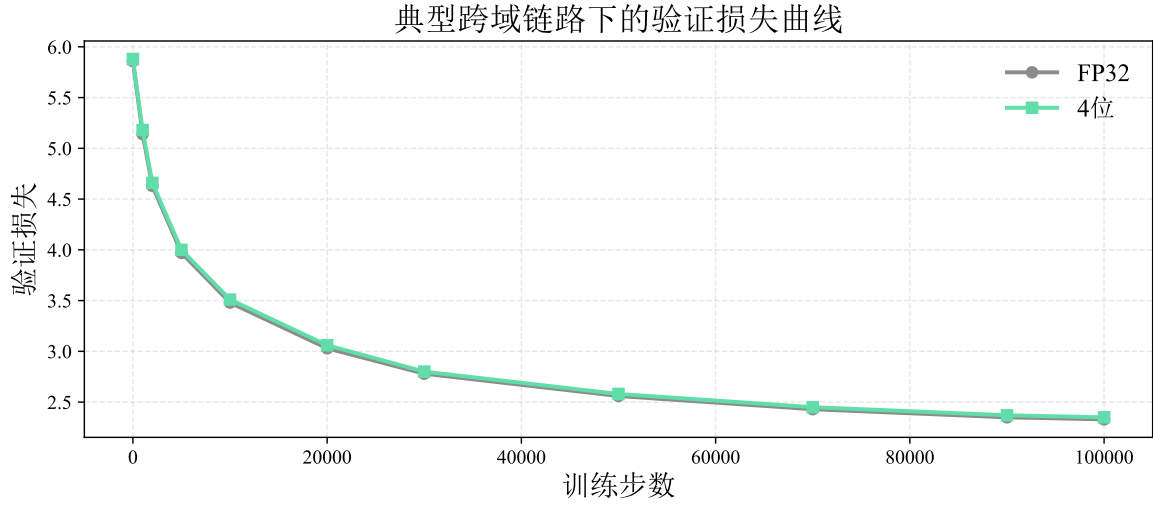


图 3.6 FP32 与 4-bit 在代表性网络条件下的 validation loss 收敛曲线

图 3.7 采用 LLaMA-7B 作为代表模型，展示不同带宽条件下各量化位宽的吞吐与最终验证损失变化。该图进一步说明，位宽选择与链路条件之间存在明显耦合：带宽越受限，低位宽带来的系统收益越突出；而在较宽带宽下，较高位宽则更有利于保留优化稳定性。

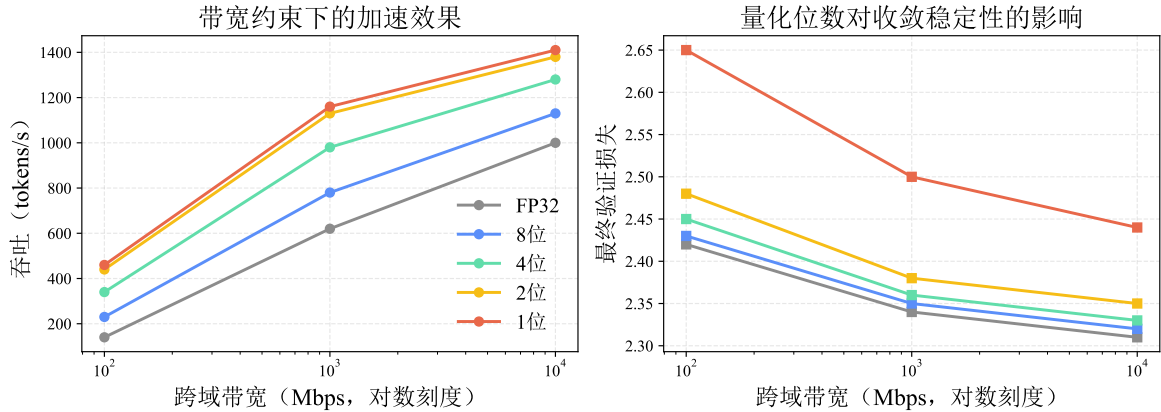


图 3.7 k-bit 量化方法的收敛性与训练加速比验证（以 LLaMA-7B 为例）

上述结果从端到端层面支撑了本文方法的通用性结论：在多任务、多负载条件下均可稳定降低端到端训练时延，并保持可接受的优化行为偏差。

为验证低比特通信方法在国产 GPU 物理集群上的可迁移性，本章进一步在实验环境二中进行了实验。实验使用两节点共 8 个训练进程，即每节点选取 4 张 GPU 参与训

练,以便与实验环境一中的主体实验保持相同的并行规模。与实验环境一不同,CoreX 集群的节点间链路为 12.5 Gb/s Ethernet,通信后端、设备驱动与集合通信实现均与 A100 云集群不同,因此该实验重点关注两个问题:其一,低比特压缩路径在异构 GPU 软件栈中是否能够保持数值收敛稳定;其二,在真实物理 Ethernet 跨节点链路下,压缩收益是否仍能在大模型通信负载中体现出来。

表 3.6 实验环境二上不同模型的低比特通信结果

模型	基线耗时	优化后耗时	加速比	收敛表现
ResNet-50	246.7 ms/step	228.4 ms/step	1.08×	loss 基本一致
YOLOv8	210.0 s/100 step	202.6 s/100 step	1.04×	loss 未见恶化
BERT-Large	903.1 ms/step	384.3 ms/step	2.35×	loss 基本一致
LLaMA-7B	4402.4 ms/step	1540.8 ms/step	2.86×	loss 基本一致

从表 3.6 可以看到,在 CoreX 物理集群中,低比特通信路径在不同模型上均未造成明显的最终损失退化,主要训练曲线与 FP32 基线处于同一量级。这说明动量归一化、随机舍入与误差补偿机制在异构 GPU 后端上仍具备数值稳定性,方法并不依赖单一厂商硬件才能维持优化行为。

系统性能方面,CoreX 实验呈现出明显的负载相关性。ResNet-50 与 YOLOv8 的端到端提升较低,分别约为 1.08 × 与 1.04 ×,主要原因在于视觉模型的单步计算、数据处理与小粒度梯度桶开销占比较高,跨节点通信并非唯一主导瓶颈。相对地,BERT-Large 与 LLaMA-7B 的参数规模和梯度同步载荷更大,低比特压缩能够显著减少跨节点 Ethernet 上的传输时间,因此分别取得约 2.35 × 与 2.86 × 的端到端加速。该结果表明,在 CoreX 集群上,本文方法的主要收益仍来自通信数据体积削减,并且收益会随模型通信负载增大而快速放大。

表 3.7 实验环境二上 LLaMA-7B 在受限带宽下的吞吐结果

带宽	1F1B+DDP 基线	1F1B+DDP k-bit	吞吐加速比
1 Gb/s	34.0 tokens/s	123.8 tokens/s	3.64×
2 Gb/s	66.6 tokens/s	220.4 tokens/s	3.31×
5 Gb/s	157.1 tokens/s	474.4 tokens/s	3.02×
10 Gb/s	287.5 tokens/s	819.4 tokens/s	2.85×

进一步地,表 3.7 给出了 LLaMA-7B 在不同受限带宽下,仅启用 k-bit 低比特同步时的吞吐趋势。结果显示,当跨节点链路从 10 Gb/s 降至 1 Gb/s 时,低比特通信的相对收

益由 $2.85 \times$ 提升至 $3.64 \times$ ，与前述 LLaMA-7B 默认端到端实验中约 $2.86 \times$ 的收益基本对齐。

该现象说明， k -bit 的主要收益仍来自跨节点同步载荷下降；链路越受限，通信体积压缩越容易转化为端到端吞吐提升。综合来看，实验环境二支撑了以下结论：低比特压缩在 CoreX 集群上能够保持收敛稳定，但端到端加速幅度取决于模型通信占比。对于 ResNet-50、YOLOv8 等通信占比较低或梯度桶较小的任务，收益更多体现为小幅缩短同步时间；对于 BERT-Large、LLaMA-7B 等通信主导型负载，压缩后的跨节点传输量下降能够直接转化为显著吞吐提升。这进一步说明本文方法适合部署在大模型和低带宽跨域训练阶段，并需要在不同硬件后端上结合模型规模与链路状态选择启用策略。

3.5 本章小结

本章围绕分布式训练中跨域通信链路的数据传输瓶颈问题，提出并详细介绍了一种基于 k -bit 随机舍入的低比特梯度量化通信策略。首先，针对现有固定精度压缩在跨域场景下的局限性，本章给出了基于动量归一化与无偏随机舍入的量化方法，通过附加的残差误差补偿机制在压缩率与优化偏差之间取得了稳健的折中。接着，设计了适配该量化算法的张量位打包与合并解包机制，并以极低的工程侵入性将其集成于现有分布式训练框架中。多网络条件与多模型端到端实验表明：该核心机制在强带宽受限场景下可将通信数据量压缩至原有的 $\frac{1}{32}$ ，在保持最终模型收敛质量基本一致（验证损失误差不超过 0.02）的前提下，实现总体端到端最高约 $2.43 \times$ 的吞吐提升，并且由于物理负载幅度的直线下降有效削减了受限环境下的长尾同步时延。国产 GPU 集群实验进一步表明，该方法在 CoreX 物理集群上能够保持收敛稳定，并在 BERT-Large、LLaMA-7B 等通信主导型负载中取得显著加速，其中 LLaMA-7B 端到端加速达到约 $2.86 \times$ ；同时，ResNet-50 与 YOLOv8 的低收益结果也说明压缩策略需要结合模型规模、硬件后端和链路状态动态启用。

第四章 基于流水线的层次化梯度通信调度策略

4.1 引言

随着分布式训练规模持续扩大，梯度同步不再只是简单的数据交换过程，而逐渐成为决定系统端到端效率的关键环节。尤其在多节点、异构互联以及云边端协同场景中，不同通信链路之间的带宽、时延和稳定性差异会显著放大同步阶段的等待开销。仅依赖传统扁平化 All-Reduce 难以充分利用节点内高速互联，也难以避免跨节点或跨域链路造成的通信尾部阻塞。因此，本章从异构集群通信特征出发，分析传统同步算法的局限，并提出面向分层拓扑的流水线化层次 All-Reduce 优化方法。

4.1.1 异构集群通信特征

在分布式深度学习训练中，梯度同步的通信开销已成为制约训练效率的关键瓶颈^[16]。特别是在多节点集群环境中，通信架构表现出明显的异构性，如图 4.1。节点内（Intra-node）通信通过高速 NVLink 或 PCIe 完成，带宽可达数百 GB/s；而节点间（Inter-node）通信受限于以太网或 InfiniBand 带宽，通常仅为 100-200 Gb/s^[22-23]。这种带宽不对称性（Bandwidth Asymmetry）可能达到 10:1。这使得跨节点通信成为通信的主要性能短板。

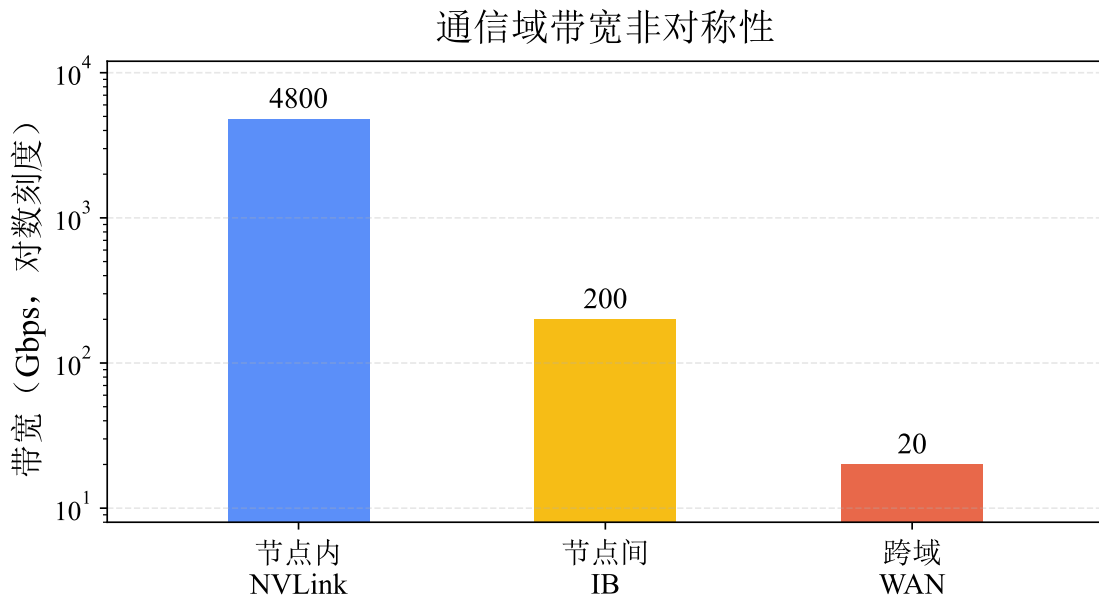


图 4.1 异构带宽不对称性导致的通信瓶颈

在云边端协同训练中，这种异构性会进一步跨域化：云内与边缘域内往往具备较高带宽互联（如 RDMA/以太网集群），但边缘到云、边缘到边缘的跨域链路带宽更低且 RTT

更高；端到边链路还可能受到无线接入与动态拥塞影响，抖动显著。因而，梯度同步往往需要跨越至少一条“低带宽/高 RTT”的跨域链路，导致同步尾部更突出，也使得如何在分层拓扑上合理调度集合通信成为云边端协同训练的关键系统问题。

4.1.2 传统同步算法的局限

传统的 All-Reduce 操作通常采用同步阻塞方式。所有进程必须在此阶段等待通信完成才能继续进行下一轮迭代的计算^[54]。在异构网络环境下，这种扁平化的通信模式会导致高速的节点内互联被低速的节点间链路拖累，造成计算资源闲置。

为解决上述问题，本研究提出了一种基于流水线的层次化 All-Reduce 方法 (Pipelining Hierarchy All-Reduce)，通过张量分块、双流并行和层次化通信策略，实现计算与通信的重叠，从而降低梯度同步延迟。

4.2 问题分析与研究思路

在分布式深度学习训练任务中，梯度同步作为迭代收敛的必要环节，其通信效率直接决定了整体训练吞吐。当训练任务部署于异构集群时，节点内高速互联与节点间低速链路形成的带宽差异，使得在梯度同步的过程中，集合通信无法充分利用带宽，从而导致训练各进程因等待跨节点通信完成而长时间等待，而高速域内带宽却因受限于低速域间带宽无法被充分利用。造成该现象的原因可归结为以下两个方面：

一方面，传统 All-Reduce 算法（如 Ring 或 Tree 拓扑）采用扁平化的通信组织方式，将所有参与进程置于同一逻辑平面进行数据交换^[54]。这种设计在同构网络环境下能够充分发挥集体通信的理论带宽，但在带宽不对称的异构集群中，慢链路会成为整个通信组的瓶颈。具体而言，当节点间链路带宽仅为节点内的 1/10 甚至 1/50 时，跨节点数据传输耗时将主导同步总时间，而节点内高速互联在完成本地聚合后需被动等待，造成计算资源与通信资源的双重闲置。

另一方面，现有通信优化方案在应对异构拓扑时存在适应性局限。梯度压缩类方法^[47,60]通过降低传输数据量缓解带宽压力，关键的是，这些上述方法大多将集合通信视为原子操作进行整体优化，未能挖掘在集合通信内部，存在域内 - 域间通信并行的潜力。

然而不同于通用的分布式任务调度，梯度同步通信的调度具有强数据依赖与严格一致性约束：本地归约必须在跨节点交换前完成，全局聚合结果必须在广播前就绪。这种模式下，域内通信和域间通信在通信语义上是要求要串行的。同时，面向深度学习模型的集合通信具有一定的启动开销，若调度粒度过小，大量的通信事件将会占用 CPU 处理

能力，通信过程将会受限于 CPU，若调度粒度过大则调度前后的效率提升不明显。这种权衡关系为异构环境下的通信调度带来了额外的参数敏感性与调优复杂度。

因此，在带宽不对称的异构集群中进行梯度同步时，需同时满足三重目标：在通信算法层需要匹配域内快、域间慢的物理互联特征来设计通信算法，在调度时序层面需要实现域内通信与域间通信操作的有效重叠，在控制层保障数据依赖正确性与工程可实现性。现有方案主要侧重通信拓扑的适配或是算法本身的优化，缺乏专用于在跨域场景下的优化工作，尚未形成在跨域场景下进行训练时针对集合通信操作的系统设计。

本章所研究问题的主要难点与挑战在于：

(1) 如何在保障数据一致性的前提下实现层次化通信的流水线重叠。梯度同步的三阶段（本地归约→跨节点归约→本地广播）存在严格的前后依赖，需设计轻量且精确的同步原语，在解耦域内/域间操作的同时避免引入额外等待。

(2) 如何动态选择分块通信的粒度以平衡并行收益与调度开销。块大小直接影响流水线深度与集合通信启动频率，需建立与张量规模、网络带宽、硬件特性相适应的自适应策略，避免静态配置在异构场景下的性能波动。

(3) 如何将算法机制无缝集成至主流训练框架。优化方案需在最小侵入的前提下兼容 PyTorch DDP 等现有接口，支持梯度累积、混合精度等常见训练模式，并提供降级路径以保障系统鲁棒性。

针对以上问题，本文研究了一种基于流水线的层次化 All-Reduce 调度策略。该方法首先将通信拓扑抽象为“本地组 - 跨节点组”两级结构，将慢链路通信压缩至代表进程路径；进而通过张量分块构建预热 - 主流水 - 冷却三阶段执行框架，利用双流并行与 CUDA Event 依赖同步实现本地归约、跨节点归约与本地广播的时序重叠；最后完成与训练循环的工程级集成。通过拓扑匹配、时序重排与系统协同的多层优化，本章方案旨在不改变优化语义的前提下，有效缓解异构集群中的通信尾部延迟，为云边端协同训练提供可叠加的通信加速基础。

4.2.1 层次化通信拓扑

如图 4.2 所示层次化通信拓扑，本方法将分布式训练环境划分为两层通信组：其中，本地组由同一节点内的 GPU 进程构成，依赖 NVLink/PCIe 等高带宽互联完成域内聚合；跨节点组则由各节点代表进程（通常为 Rank 0）组成，负责跨节点数据交换与同步。

层次化通信将 All-Reduce 操作分解为三个阶段：在执行顺序上，系统先在节点内完成本地归约（Local Reduce），再由代表进程执行跨节点归约（Inter Reduce），最后将聚合

结果在节点内广播（Local Broadcast）到其余 GPU。该分解方式能够把慢链路上的同步数据量压缩到代表进程粒度，并将更多通信负载留在高带宽域内链路上。

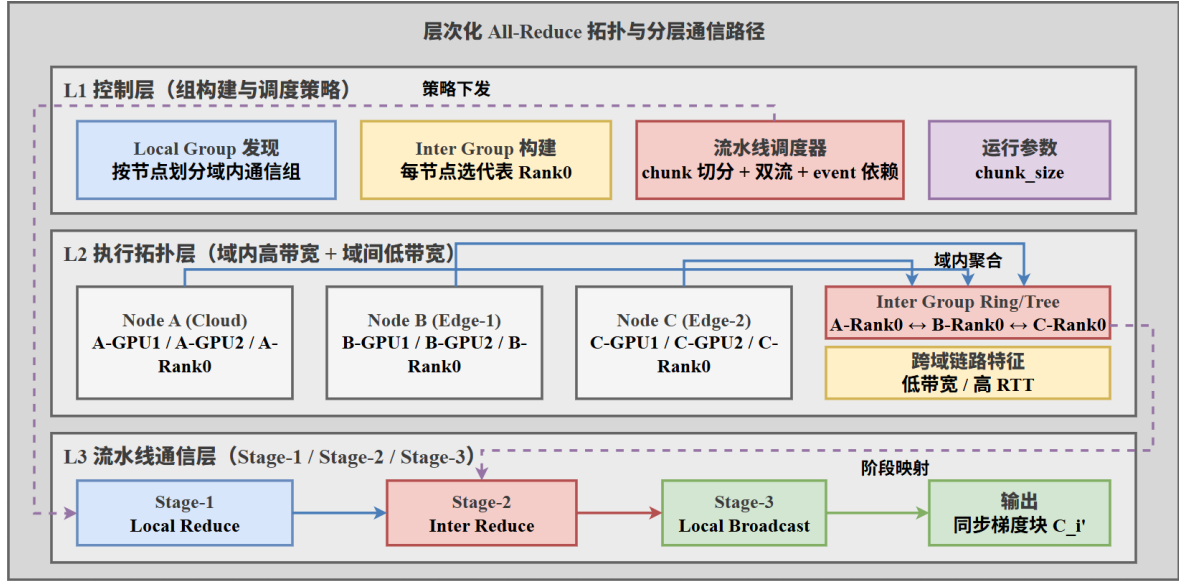


图 4.2 层次化 All-Reduce 通信拓扑与数据流向

从云边端协同的角度看，上述两层抽象可自然映射为“域内/域间”两级：Local Group 对应云内或边缘域内的高带宽通信组，Inter Group 对应跨域（边缘↔云或边缘↔边缘）的代表进程通信组。这样，层次化 All-Reduce 能将跨域通信压缩为少量代表进程的数据交换，同时尽可能把域内高带宽链路用于聚合与分发，从拓扑上契合云边端协同的分层互联特征。

4.2.2 流水线分块策略

算法首先将待传输的梯度张量分割为 N 个固定大小的块（chunks），记为 $\{C_0, C_1, \dots, C_{N-1}\}$ 。分块的目的在于将原本一次性完成的大规模梯度同步拆解为多个较小粒度的通信任务，使得不同通信阶段能够以流水线方式交错执行，从而提高链路利用率并减少同步尾部等待。对于大规模梯度张量而言，若直接对完整张量执行 All-Reduce，则必须等待整个张量完成跨节点传输和规约后才能进入下一阶段；而分块后，前一个梯度块完成节点内规约后即可立即进入跨节点通信阶段，后续块则可以继续执行本地规约或等待调度，从而形成连续的数据流。

分块粒度由参数 chunk_size 控制，其大小直接影响流水线并发度与通信调度开销之间的平衡。首先，从流水线并发度角度看，若 chunk_size 过大，则张量被划分出的块数较少，流水线深度不足，不同通信阶段之间难以充分重叠，跨节点通信延迟仍会暴露在

关键路径上。此时算法退化为接近传统层次化 All-Reduce 的执行方式，无法有效隐藏低速链路带来的等待开销。其次，从系统调度与启动开销角度看，若 `chunk_size` 过小，则块数 N 过多，每个块都需要触发独立的通信调度、缓冲区管理和 CUDA stream 同步，集合通信原语的启动开销会显著累积。当启动开销超过分块带来的重叠收益时，过细粒度的切分反而会降低整体通信效率。

因此，`chunk_size` 的选择需要在“足够多的块以形成流水线”和“避免过多小块导致调度开销放大”之间取得折中。本文实验中采用自适应分块策略，根据梯度张量规模动态确定块大小，使每个张量通常被划分为 4 到 8 个块：

$$\text{chunk_size} = \frac{\text{tensor_size}}{4 \sim 8} \quad (4.1)$$

其中，`tensor_size` 表示当前梯度张量的元素数量或字节规模， $4 \sim 8$ 表示经验上较优的分块数量区间。该设置能够在保证流水线具备一定深度的同时，避免过多通信任务引入额外调度负担。对于较大的梯度张量，分块可以提供更充分的通信重叠空间；对于较小的梯度张量，算法则避免过度切分，以减少启动和同步开销。通过这种自适应策略，系统能够在不同模型层、不同张量规模以及不同网络条件下保持较稳定的通信效率。

4.2.3 双流并行机制

在底层硬件调度中，GPU 默认将提交的计算或通信任务排列在单一执行队列中串行执行，这导致不同层级的通信任务无法在物理时间上真正重叠。为充分利用硬件多并发能力，本算法显式创建了两个独立的 CUDA 流（CUDA Stream，即 GPU 上的独立异步指令队列）：

节点内流（Intra-stream）专用于调度处于高带宽互联下的本地归约与广播操作；节点间流（Inter-stream）则专用于调度处于低带宽链路的跨节点归约操作。由于这两种操作在使用网络资源侧重点不同（例如局部总线与跨设备网卡），它们在底层硬件上具备同时运作的客观条件。

为确保跨流操作的数据一致性（例如，必须在本地块归约完成之后，代表节点才能将该块数据发往外部进行跨节点归约），两条流之间引入了 CUDA Event 作为轻量级同步原语。借助该原语，算法在保障各数据块前后逻辑依赖绝对正确的前提下，最大化了两条通信链路的调度重叠率。

4.3 详细方案设计与实现

从执行行为上看，算法由预热、主流水和冷却三个连续阶段构成，如图 4.3 所示。

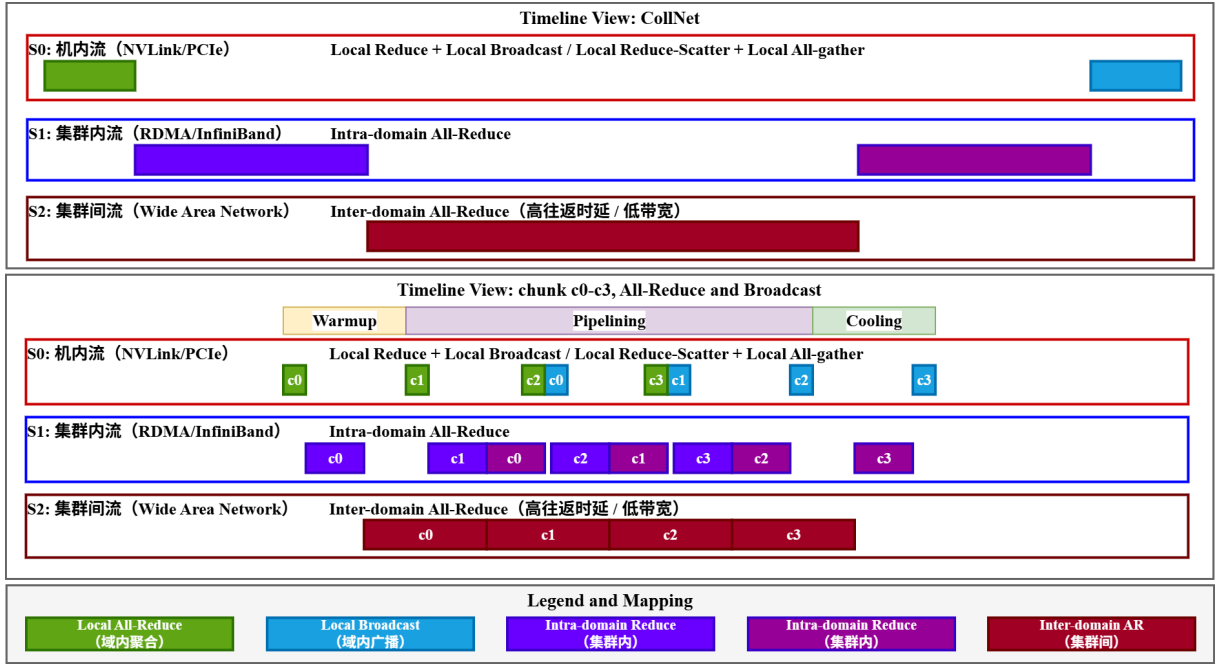


图 4.3 流水线层次化 All-Reduce 时序图 (ar: All-Reduce, bc: Broadcast)

4.3.1 阶段一：预热阶段

预热阶段启动前两个块的处理流程。首先启动第一个块的本地归约操作。如算法 4.1 所示，该阶段的主要目的在于启动集合通信流水：第一个块完成本地归约后立即开始跨节点归约，同时第二个块开始本地归约，实现两个操作的并行执行。

算法 4.1: 预热阶段流水线启动

输入： chunks, local_group, inter_group, 事件列表

输出： 初始化流水线依赖图

- 1: 在节点内流启动 C_0 的本地归约: `all_reduce(chunks[0], group=local_group)`
- 2: 记录 `event_list_intra[0]`
- 3: 在节点内流启动 C_1 的本地归约: `all_reduce(chunks[1], group=local_group)`
- 4: 记录 `event_list_intra[1]`
- 5: 在节点间流等待 `event_list_intra[0]`
- 6: 在节点间流执行 C_0 跨节点归约: `all_reduce(chunks[0], group=inter_group)`
- 7: 执行 `barrier(local_group)` 保证同节点进度一致
- 8: 记录 `event_list_inter[0]`

4.3.2 阶段二：流水线主循环

对于剩余的块 $i \in [2, N - 1]$ ，并行执行三个操作：

算法 4.2: 流水线主循环

输入: 块索引 $i \in [2, N - 1]$, 事件列表, local_group, inter_group

输出: 完成所有中间块的重叠调度

```

1: for  $i = 2$  to  $N - 1$  do
2:   ▷ 节点内流: 并行执行广播和本地归约
3:   等待 event_list_inter[ $i-2$ ]   ▷ 等待  $C_{i-2}$  跨节点归约完成
4:   broadcast(chunks[ $i-2$ ], group=local_group)   ▷ 广播  $C_{i-2}$ 
5:   all_reduce(chunks[ $i$ ], group=local_group)   ▷ 本地归约  $C_i$ 
6:   记录 event_list_intra[ $i$ ]
7:   ▷ 节点间流: 跨节点归约
8:   等待 event_list_intra[ $i-1$ ]   ▷ 等待  $C_{i-1}$  本地归约完成
9:   if 当前进程是节点代表 then
10:    | all_reduce(chunks[ $i-1$ ], group=inter_group)
11:   end
12:   barrier(local_group)
13:   记录 event_list_inter[ $i-1$ ]
14: end

```

如算法 4.2 所示, 该阶段是算法的核心, 其并行性体现在同一时刻由节点内流处理块 $i - 2$ 的广播与块 i 的本地归约, 同时由节点间流处理块 $i - 1$ 的跨节点归约; 通过 CUDA Event 的精确依赖同步, 上述三个操作能够在保持正确性的前提下实现稳定重叠。

4.3.3 阶段三: 冷却阶段

处理最后两个块的广播和跨节点归约。如算法 4.3 所示, 该阶段负责处理流水线末尾的收尾流程, 确保最后两个块的跨节点归约与广播能够正确完成。

算法 4.3: 冷却阶段收尾流程

输入: 事件列表与最后两个块索引

输出: 完成全部块的传播闭环

```

1: 等待 event_list_inter[N-2]
2: broadcast(chunks[N-2], group=local_group)   ▷ 广播倒数第二块
3: 等待 event_list_intra[N-1]
4: all_reduce(chunks[N-1], group=inter_group)   ▷ 跨节点归约最后一块
5: barrier(local_group)
6: 等待 event_list_inter[N-1]
7: broadcast(chunks[N-1], group=local_group)   ▷ 广播最后一块

```

4.4 理论性能与开销分析

4.4.1 时间复杂度

传统 All-Reduce 方法的总时间为各阶段时间之和：

$$T_{\text{total}} = T_{\text{local}} + T_{\text{inter}} + T_{\text{broadcast}} \quad (4.2)$$

而流水线 All-Reduce 方法通过并行执行三个操作，理论加速比为：

$$\text{Speedup} = \frac{T_{\text{local}} + T_{\text{inter}} + T_{\text{broadcast}}}{\max(T_{\text{local}}, T_{\text{inter}}, T_{\text{broadcast}}) + O(N_{\text{chunks}})} \quad (4.3)$$

当块数足够多且三个操作耗时相近时，可接近 1.5× 加速。

4.4.2 空间复杂度

额外内存开销分析：额外开销主要来自 CUDA Event 对象管理，其规模约为 $O(N_{\text{chunks}})$ ；而张量分块本身采用视图操作，不引入新的实质性数据拷贝。PyTorch 等框架中的 Tensor Chunk 仅创建新的 Metadata 视图（View）而不拷贝底层显存，因此对动辄数十 GB 的模型梯度而言，仅有的几十个轻量级 CUDA Event 对象所占用的系统内存几乎可以忽略不计。因此总体空间复杂度仍为 $O(N_{\text{chunks}})$ 。

4.5 系统集成与工程优化

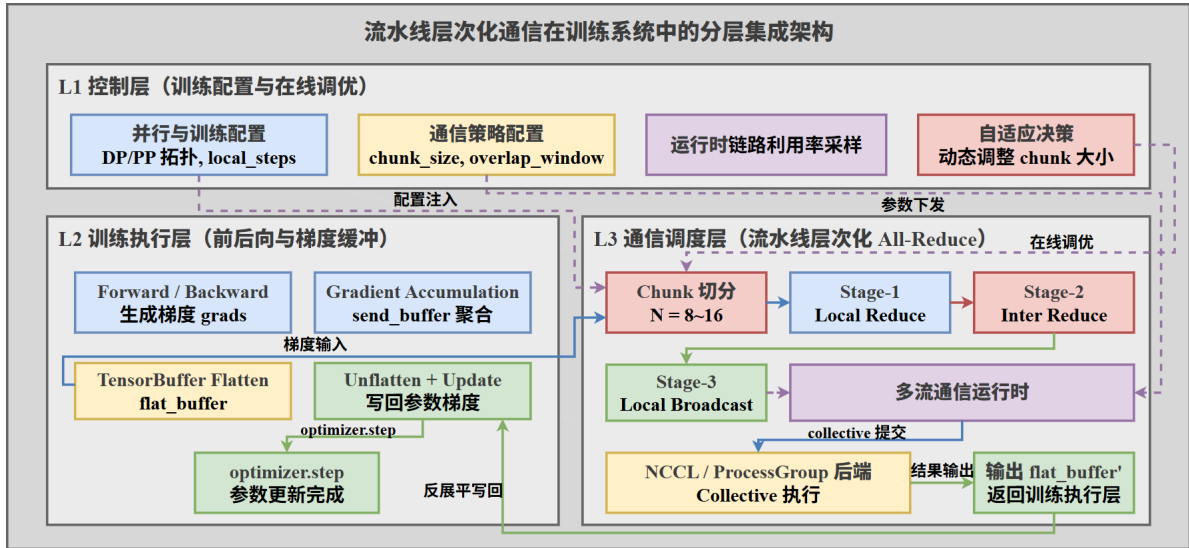


图 4.4 流水线调度器在训练流程中的集成架构

系统集成如图 4.4 所示，主要分为控制层，训练执行层，通信调度层；分别负责管理训练配置，梯度数据获取，通信原语调度。首先，为降低频繁触发底层通信原语所带来

的系统调用与握手开销，本方法引入了梯度展平（Tensor Flattening）机制。在深度神经网络中，模型参数通常由众多尺寸不一的离散张量构成，直接对独立的参数梯段进行流水线分块不仅无法形成有效的并发深缩，反而会使通信栈过载。系统通过预先在发送缓冲区（Send Buffer）分配一块连续的底层显存空间，并在计算结束后将各层梯度视图按固定偏移拷贝至该大块缓冲中（TensorBuffer）。这不仅确保了后续流水线分块（Chunks）的物理连续性，也使通信带宽利用率得以大幅提升。其次，对于采用梯度累积（Gradient Accumulation）的大规模训练场景，系统在设计上将通信触发时机与局步步数（Local Steps）深度绑定。如算法 4.4 所示，仅当 `global_iteration` 满足目标累积步数要求时，才会执行展平与随后的流水线同步逻辑。在其它的局部累加步骤中，各工作节点仅在本地进行正常的矩阵运算累加，通信栈保持等待。最后，系统保留了原生扁平通信的后备降级路径。可以通过配置开关在并行流水线调度与传统的 `all_reduce` 之间进行运行时切换。这种透明化集成策略将通信协议管理的复杂度限制在底层引擎，对上层用户训练脚本而言并无侵入性。

算法 4.4: 流水线层次化 All-Reduce 集成

输入： 梯度列表 `grads`，本地步数 `local_steps`

输出： 聚合后的梯度缓冲

```

1: if global_iteration mod local_steps = 0 then
2:   ▸ 累积梯度到发送缓冲区
3:   for 每个梯度 gradi do
4:     | send_bufferi ← gradi
5:   end
6:   ▸ 展平所有梯度为连续张量
7:   flat_buffer ← TensorBuffer(send_buffers)
8:   if 启用流水线通信 then
9:     | pipelining_all_reduce(flat_buffer, local_group, inter_group, chunk_size)
10:  else
11:    | all_reduce(flat_buffer)
12:  end
13: end

```

4.6 实验与验证

本章围绕流水线层次化 All-Reduce 方法开展实验验证，重点评估其在异构集群和跨域链路下的通信优化效果。实验从通信微基准、端到端训练性能、分块参数敏感性以及带

宽不对称影响等多个角度展开，以验证方法收益的稳定性与适用范围。为保证结论可靠，本章同时在 NVIDIA A100 云集群和 Iluvatar CoreX 国产 GPU 物理集群上进行测试，考察方法在不同硬件后端下的可迁移性。通过与 PyTorch 原生 All-Reduce 和朴素层次化 All-Reduce 对比，进一步分析性能提升来源是否来自拓扑分层与流水线重叠的共同作用。

4.6.1 实验环境

本章实验涉及两类硬件环境，如表 4.1 所示。实验环境一为 NVIDIA A100 云上双节点集群，节点内 GPU 通过 PCIe 全互联，网卡使用 IB 网口；节点内 PCIe 有效带宽约为 126 Gb/s，节点间互联带宽约为 50 Gb/s。软件侧采用 PyTorch/NCCL/CUDA 分布式训练栈，并固定通信后端、batch 配置和日志采集脚本，以保证不同通信策略之间仅在集合通信路径与调度方式上存在差异。任务侧采用 ResNet-50 与 ImageNet 组合，并设置每 GPU batch 为 256^[62-63]。对比对象包括 PyTorch 原生 All-Reduce 与无流水线的朴素层次化 All-Reduce。实验环境二为天数智芯（Iluvatar CoreX）国产 GPU 物理集群，节点 n210 对应 Domain-1，节点 n62 对应 Domain-2；每节点 CPU 为 Intel Xeon Gold 6330，GPU 为 Iluvatar BI-V150 x 8，板卡内部各包含 2 个 GPU 芯粒。该环境中 GPU 两两成对走板内集成互联路径，每对 GPU 之间经 PCIe Switch 互联且不跨 NUMA，所有 GPU 位于同一 NUMA，NIC0 挂载在另一 NUMA 节点上；节点间通过 Ethernet 互联，节点内全 PCIe 互联带宽约为 126 Gb/s，节点间互联带宽约为 12.5 Gb/s。由于该环境的通信后端、网卡位置与节点间链路均不同于 A100 云集群，本章将其作为异构硬件可迁移性验证。

表 4.1 第四章实验硬件环境

环境	计算节点与 GPU	节点内互联	节点间互联
实验环境一	2 节点 NVIDIA A100 云集群	PCIe 全互联，约 126 Gb/s	IB 网口，约 50 Gb/s
实验环境二	2 节点 Iluvatar BI-V150	同 NUMA PCIe，板内芯粒成对互联，约 126 Gb/s	Ethernet，约 12.5 Gb/s

表 4.2 用于云边端协同场景的分层链路仿真参数

链路层级	带宽（示例）	RTT（示例）
域内（云内/边缘域内）	100 – 200 Gb/s	< 1 ms
域间（边缘↔云/边缘↔边缘）	10 – 30 Gb/s	30 – 100 ms

为体现本文方法在跨域互联中的适用性，实验还采用网络整形对应的域间链路注入带宽约束，用于浮现训练时的广域网特征；域内链路仍保持域内高带宽互联，以刻画典

型的域内快、域间慢的云边端跨域训练环境，如表 4.2。该设置重点用于验证层次化流水线在跨域瓶颈下的调度收益与敏感性趋势。

4.6.2 通信性能对比

表 4.3 展示了不同方法的通信性能对比。

表 4.3 不同 All-Reduce 方法的通信性能对比

方法	通信耗时 (ms)	相对基线	GPU 利用率
PyTorch 原生	128.5	1.00×	68%
朴素层次化	91.3	0.71×	76%
流水线层次化	74.1	0.58×	89%

从结果看，流水线层次化方法相对 PyTorch 原生通信路径将通信耗时降低了 42.3%，并将 GPU 利用率从 68% 提升到 89%；即便与朴素层次化方法相比，流水线机制仍额外带来 18.8% 的通信时间缩减，说明收益不仅来自拓扑分层，还来自时序重排与并行重叠。

这一结果可以从拓扑匹配 + 时序重叠两个维度解释。朴素层次化主要解决的是拓扑不匹配问题，即将经由慢链路通信的数据块数降低从而降低由于高 RTT 和低带宽导致的通信效率低的问题；流水线层次化则进一步重排执行时序，把本地归约、跨节点归约和本地广播并行化，从而进一步提升了在整体网络环境下的带宽利用率。GPU 利用率提升与通信耗时下降是因为，同步通信缩短后，计算流等待通信完成的空转时间减少，GPU 能够更连续地执行前后向计算。由此，通信层优化直接转化为计算资源利用效率提升，从而可以提升整体训练效率。

4.6.3 端到端训练加速

表 4.4 展示了端到端训练性能的提升。

表 4.4 端到端训练性能对比

方法	迭代时间 (ms)	吞吐指标 (items/s)	加速比
PyTorch 原生	445.2	4,602	1.00×
朴素层次化	408.0	5,024	1.09×
流水线层次化	329.7	6,216	1.35×

端到端结果进一步验证了通信优化能够转化为训练层面的实质收益。相较通信耗时指标，迭代时间和吞吐更接近业务目标，因此其改善更具说服力。数据表明方法从减少通信等待推进到缩短训练迭代。将端到端收益与前述通信收益对照可发现，两者量级并

非线性一致，这符合真实训练系统特征：迭代时间由计算、通信、数据流水等多环节共同决定，通信下降会被其余环节部分吸收。即便如此仍能获得显著加速，说明通信在当前设置中确实是主要瓶颈之一。此外，朴素层次化与流水线层次化形成了清晰的递进关系，避免了仅与单一基线比较导致的解释歧义。先证明拓扑分层有效，再证明时序流水化在分层基础上继续增益，使结论链条更完整：收益来源可分解、可解释、可复现。

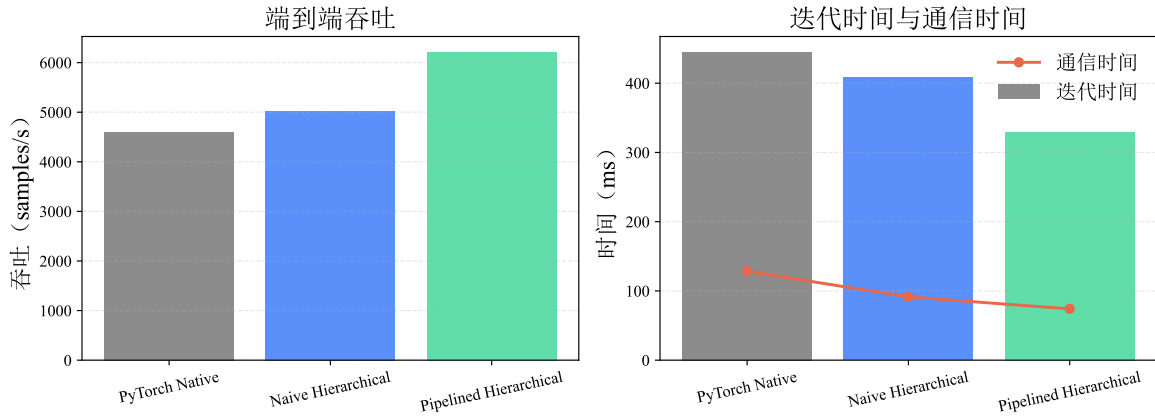


图 4.5 端到端训练性能对比与耗时分解

4.6.4 块大小敏感性分析

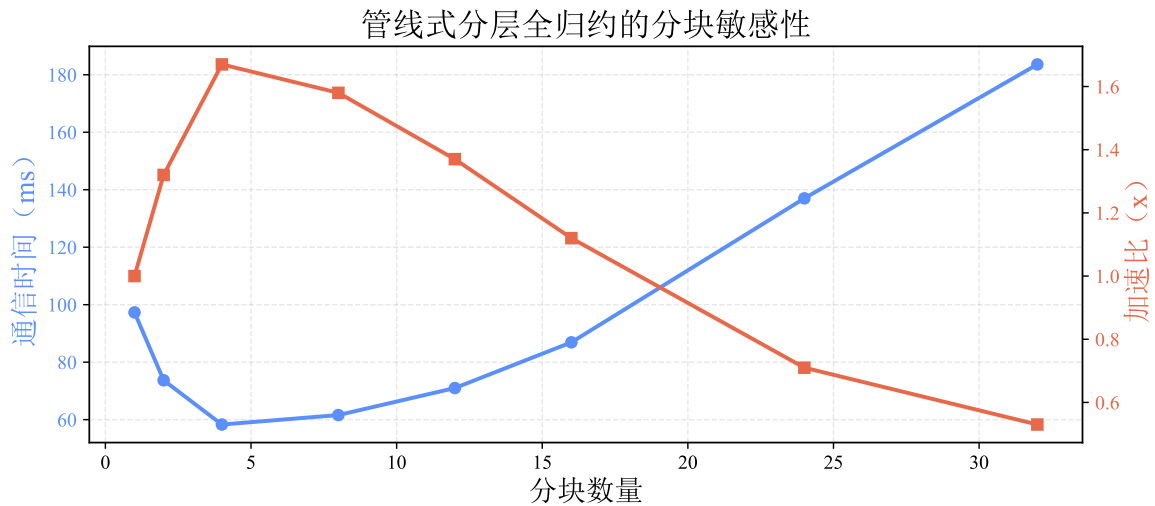


图 4.6 块大小对通信性能的影响

图 4.6 展示了不同块大小对性能的影响。敏感性结果显示，块数在 4-8 区间时性能最优，对应块大小约为张量尺寸的 $\frac{1}{4} \sim \frac{1}{8}$ ；当块数低于 4 时，并行深度不足，通信难以被充分掩盖；而当块数增加到 16 时，启动开销与事件同步开销已经超过流水线重叠收益，开始出现负优化。

块大小敏感性结果揭示了典型的并行深度与调度开销的权衡。块数过少时，流水线阶段不足以覆盖跨节点归约延迟，重叠窗口利用不充分；块数过多时，集合通信启动、事件同步和调度管理开销快速累积，抵消分块带来的并行收益。最优区间的存在说明该方法无法通过无限细化块大小来获得持续收益，需要在系统开销模型下选择稳定工作点。

这一观察直接支撑了本章提出的自适应分块策略。固定块大小难以在不同张量规模和网络条件下同时最优，而以区间方式给出可行工作带，能够在工程部署中降低调参成本，并减少因环境变化导致的性能波动。对于跨域链路波动场景，采用区间内保守配置通常比追求单点最优更稳健。

4.6.5 带宽不对称性影响

图 4.7 分析了节点内外带宽比在固定通信张量下，不同分块数的加速效果。

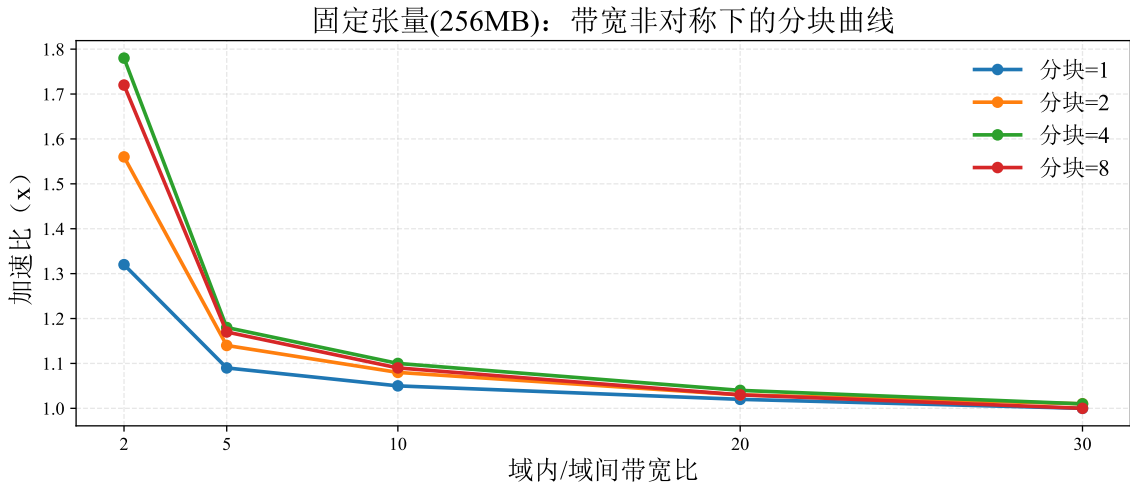


图 4.7 固定张量（256 MB）下，不同 chunk 数在带宽不对称场景中的加速比

该实验采用三个变量控制：变量 1（网络条件）：节点内/节点间带宽比（5:1 30:1）；变量 2（通信张量大小）：固定为 256 MB；变量 3（分块数）：1、2、4、8（对应不同 chunk 粒度）。

结果表明：流水线层次化方案在轻度到中等带宽不对称场景中收益最明显；当节点内/节点间带宽比继续增大时，节点间慢链路逐渐成为不可隐藏的主瓶颈，加速比随之收敛到 1 附近。在固定 256 MB 通信张量下，chunk=4 取得整体最优结果，chunk=8 与其接近但在极端失衡时受到调度开销影响，而 chunk=1 受并行深度不足限制。该现象表明分块粒度与网络条件存在显著耦合关系，合理选择 chunk 数是低质网络优化的关键。带宽不对称实验给出了本章方法适用性的关键证据。若方法仅在近似同构网络下有效，则其

跨域价值有限；当前结果显示，在 2:1 等轻度失衡场景中流水线重叠能够显著缩短等待时间，而在 20:1、30:1 等极端失衡场景中收益逐步收敛，说明层次化流水线能够缓解但不能完全消除跨域慢链路瓶颈。

`chunk=4` 与 `chunk=8` 在低到中等不对称区间表现更优，进一步说明跨域慢链路场景下需要足够流水深度来提升重叠效率，但不应无限细化到引入过高调度开销。该结论与块大小敏感性实验形成互证：参数选择应与网络结构联合考虑，而非脱离网络条件做静态配置。

4.6.6 低质网络全面实验设计

为更全面验证层次化集合通信在低质网络下的有效性，补充 4 组针对性实验。四组实验统一采用三种对比方法（PyTorch 原生、朴素层次化、流水线层次化），并保持模型、batch、优化器与并行拓扑一致，仅改变网络条件或系统扰动因素。

实验统一采集四类指标：通信平均耗时、迭代时长、吞吐量、GPU 空闲占比；同时记录跨域链路的有效带宽利用率，用于解释性能差异来源。

组 A：带宽退化实验（Bandwidth Throttling）

目标是验证在纯带宽受限场景下层次化流水线的稳健性，关注在 5-30 Gb/s 区间的收益保持能力。

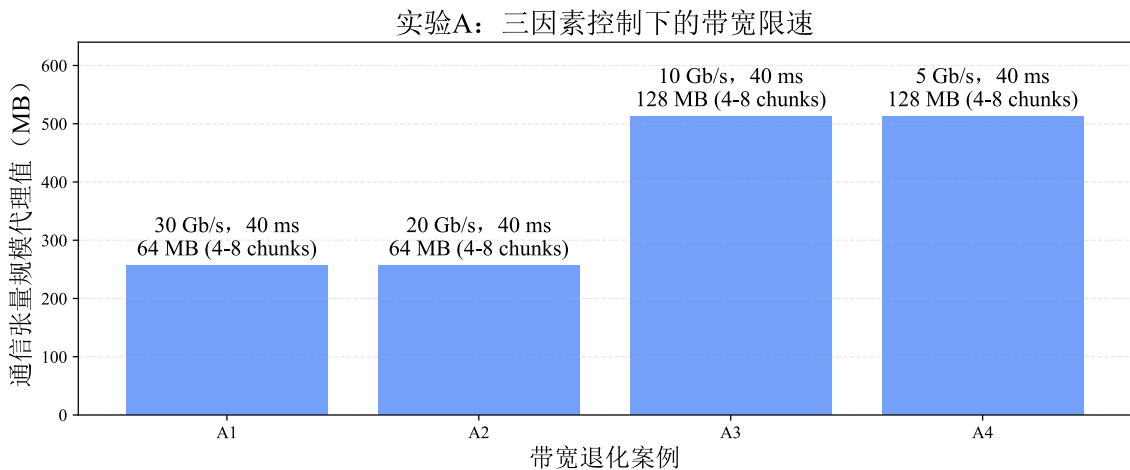


图 4.8 实验组 A：跨域带宽退化设置

组 A 的分析重点在于验证纯带宽受限条件下的收益保持能力。随着可用带宽逐级下降，跨域归约时延接近似 $\frac{1}{\{BW\}}$ 规律上升，流水线层次化通过分块重叠与代表进程路径压缩，能够持续降低有效等待时间。若该方法仅在高带宽区间有效，则在低带宽下应迅速失效；图中趋势显示其在退化区间仍保持可观优势，支持本章关于低质链路稳健性的结论。

该组实验同时说明收益来源主要是在跨域路径上的通信效率提升。实验控制了模型与并行配置，仅改变带宽变量，性能变化可主要归因于通信路径。由此可将结论外推到同类型带宽受限场景：当跨域同步主导迭代尾部时，层次化流水线是有效优先策略。

组 B：高时延实验 (RTT Escalation)：目标是隔离 RTT 对同步尾部的影响，验证流水线调度是否能持续隐藏高 RTT 带来的阻塞。

组 B 用于隔离 RTT 对同步尾部的影响。高 RTT 场景下，collective 启动与轮次同步等待会被显著放大，传统扁平通信更易形成尾部串行堆叠。实验趋势表明，流水线层次化在 RTT 升高时仍能维持较优性能，说明其通过重叠执行与阶段解耦降低了时延放大效应。

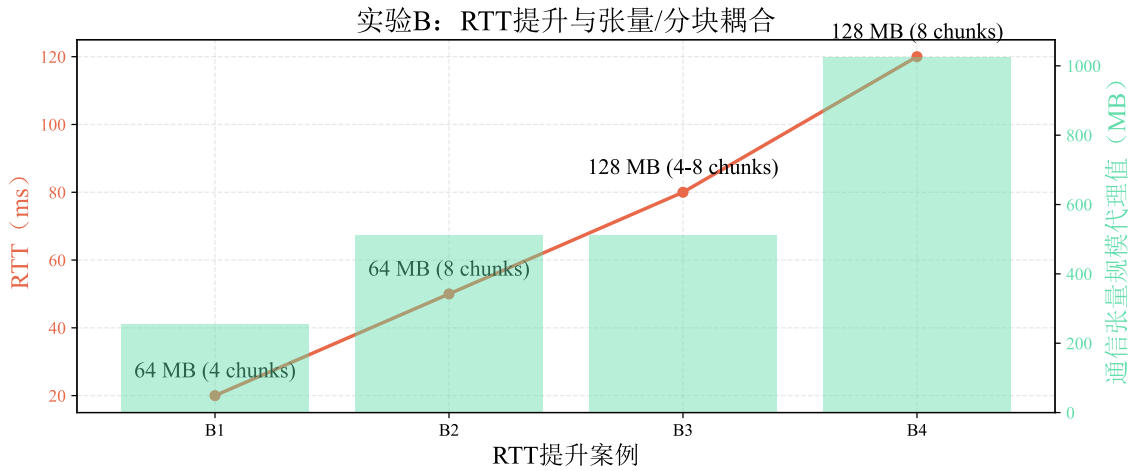


图 4.9 实验组 B：跨域 RTT 分级设置

该结果支撑了“跨域高时延场景优先做时序重排”的实践结论。与仅靠提升带宽相比，时序层优化对 RTT 的敏感性更低，因此在无法显著改善物理链路时，仍可通过调度层手段取得稳定收益。这一结论与第 5 章关于同步尾部治理的思路一致，体现章节间方法协同性。

国产 GPU 集群验证实验：

为进一步验证层次化集合通信调度在异构国产 GPU 物理集群中的行为，本章在实验环境二上进行了 All-Reduce 微基准实验。该实验保持两节点、每节点 4 GPU 的分布式进程布局，即在每个物理节点中选取 4 张 GPU 参与测试，以对齐实验环境一的 8 进程规模。实验比较 PyTorch/NCCL 原生 All-Reduce、朴素层次化 All-Reduce 与流水线层次化 All-Reduce 三种路径，并在 4 MB 至 128 MB 的张量规模下统计平均通信耗时与有效吞吐。从表 4.5 和表 4.6 可以看到，在 CoreX 物理集群中，层次化流水线的收益随张量规模增大而逐渐显现。对于 4 MB 与 16 MB 小张量，PyTorch/NCCL 原生路径分别为 1.56

ms 与 4.73 ms，仍略优于流水线层次化；这是因为小张量下通信启动、进程组调度与事件同步开销占比较高，流水线尚未形成足够长的重叠窗口。对于 64 MB 和 128 MB 张量，流水线层次化分别将耗时降至 12.31 ms 和 21.12 ms，相比原生 All-Reduce 的 17.08 ms 和 33.28 ms 明显更优；其中 128 MB 场景下有效吞吐由 4.03 GB/s 提升至 6.35 GB/s，约为 1.58 ×。

表 4.5 实验环境二上不同 All-Reduce 路径的通信耗时

张量规模	PyTorch 原生	朴素层次化	流水线层次化	最快方法
4 MB	1.56 ms	9.44 ms	3.11 ms	PyTorch 原生
16 MB	4.73 ms	11.29 ms	4.95 ms	PyTorch 原生
64 MB	17.08 ms	18.70 ms	12.31 ms	流水线层次化
128 MB	33.28 ms	27.40 ms	21.12 ms	流水线层次化

表 4.6 实验环境二上不同 All-Reduce 路径的有效吞吐

张量规模	PyTorch 原生	朴素层次化	流水线层次化
4 MB	2.69 GB/s	0.44 GB/s	1.35 GB/s
16 MB	3.55 GB/s	1.49 GB/s	3.39 GB/s
64 MB	3.93 GB/s	3.59 GB/s	5.45 GB/s
128 MB	4.03 GB/s	4.90 GB/s	6.35 GB/s

朴素层次化与流水线层次化的差异进一步说明，收益并不仅来自拓扑分层，还来自阶段之间的时序重叠。以 128 MB 张量为例，朴素层次化耗时为 27.40 ms，而流水线层次化进一步降低到 21.12 ms，说明本地归约、跨节点传输与本地广播按块接力执行后，能够更充分利用节点内 PCIe 与节点间 Ethernet 两类链路。随着张量规模增大，单次通信的数据传输时间占比上升，流水线的预热与冷却开销被摊薄，因此其优势更加明显。

该补充实验为第四章结论提供了本方法更明确的适用范围：层次化流水线更适合中大张量和跨节点通信占比较高的同步阶段，而对于小张量通信，原生 collective 的低启动开销仍具有优势。因此，在国产 GPU 集群等新后端上部署时，不应静态替换所有 All-Reduce 操作，而应根据张量规模、拓扑结构和在线基准测试结果选择通信路径。对于大模型训练中常见的大梯度桶，实验环境二的结果仍支持本文关于“域内快、域间慢”拓扑下采用流水线层次化调度的核心结论。

同时，该实验也指出了后续工程优化方向：一是减少本地组和跨节点组切换的固定开销，二是提高流水线分块的批处理效率，三是在运行时根据张量规模、NUMA 位置和

链路带宽选择原生 All-Reduce 或层次化流水线。该方向与前文块大小敏感性实验一致，说明集合通信调度优化需要由硬件拓扑和在线性能共同驱动。

统计检验与可复现性约束：每个子实验至少运行 3 次独立重复并报告均值与标准差；对关键指标（迭代时长、吞吐量）执行配对显著性检验（如 `paired t-test`），保证结论不依赖单次偶然波动。网络整形参数、随机种子、日志脚本和绘图脚本统一纳入仓库，确保实验可复现。

4.7 本章小结

本章聚焦“域内快、域间慢”导致的跨节点通信瓶颈问题，提出了基于流水线的层次化 All-Reduce 调度方案。方法层面，将 All-Reduce 分解为域内归约、域间归约与域内广播三阶段，并通过分块、双流与事件依赖构建预热—主流水—冷却的稳定流水线执行框架；工程层面给出在 PyTorch/NCCL 下的可落地实现与关键参数选择（如 `chunk_size` 的区间化配置）。实验在多节点 GPU 集群中显示该方案显著缩短通信等待并提升端到端训练效率，取得约 $1.35\times$ 的整体加速；在轻度到中等带宽不对称场景下收益更明显，而在极端失衡时逐步受限于跨域慢链路瓶颈。CoreX 物理集群补充实验显示，流水线层次化在 64 MB 及以上张量规模下优于原生 All-Reduce，128 MB 场景下有效吞吐提升至约 $1.58\times$ ，但小张量场景仍受启动开销影响。因此部署时需要结合拓扑探测与在线基准测试动态选择通信路径。综合来看，本章方案在不改变优化语义的前提下，通过调度重排与并行重叠有效缓解通信尾部，为后续量化与重叠机制提供了可叠加的通信加速基础。

第五章 通信受限场景下的混合并行通信掩盖策略

5.1 引言

单数据中心互联（如 InfiniBand/RDMA）相比，广域网（WAN）通常只有 10 - 30 Gb/s 量级带宽、30 - 100 ms 往返时延（RTT），使得同步训练中的通信开销直接进入关键路径，造成 GPU 长时间空闲与吞吐下降^[24]。

从云边端协同训练的视角，当训练以跨域数据并行（DP）为并行策略骨架时，梯度的 All-Reduce 同步集合通信过程必须跨越域间的 WAN 链路，从而在每一次训练迭代的反向传播后形成显著的同步尾部。域内的设备持续将数据/梯度贡献汇入域间的网络出口，造成了严重的域间通信瓶颈，进一步放大 WAN 抖动对全局吞吐的影响。

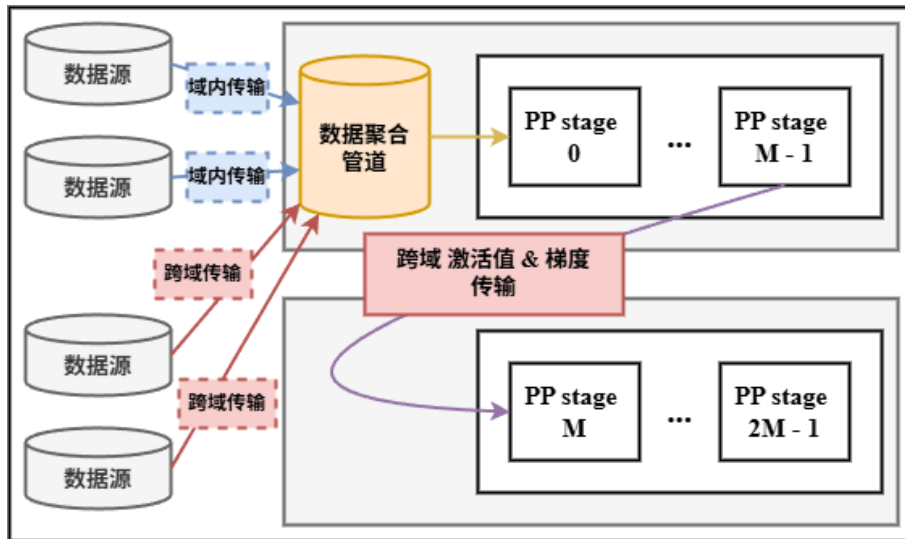


图 5.1 跨域流水线并行：激活数据在域间传输，产生域间通信瓶颈

从系统角度看，跨域训练的难点不只在带宽更低，同时存在着通信时延更高且更难被隐藏的问题。当 RTT 达到毫秒量级时，许多传统在单数据中心可忽略的等待（例如一次集合通信的启动、同步点的排队）都会叠加成显著的迭代尾部。与此同时，跨域的网络抖动还会放大同步屏障对全局吞吐的影响：任何一个域内计算进程的短暂变慢，都可能让所有的计算资源在屏障处空等。

5.1.1 并行策略骨架的选择

在跨域训练的环境中，以流水线并行为骨架会迫使训练样本/激活在域间频繁穿梭，入口与跨段链路容易成为瓶颈，如图 5.1 所示^[25]。相对地，以数据并行为主要策略将前/

后向计算限定在域内，仅在迭代末进行梯度同步，天然契合数据就地处理，并对网络波动更鲁棒，如图 5.2 所示。

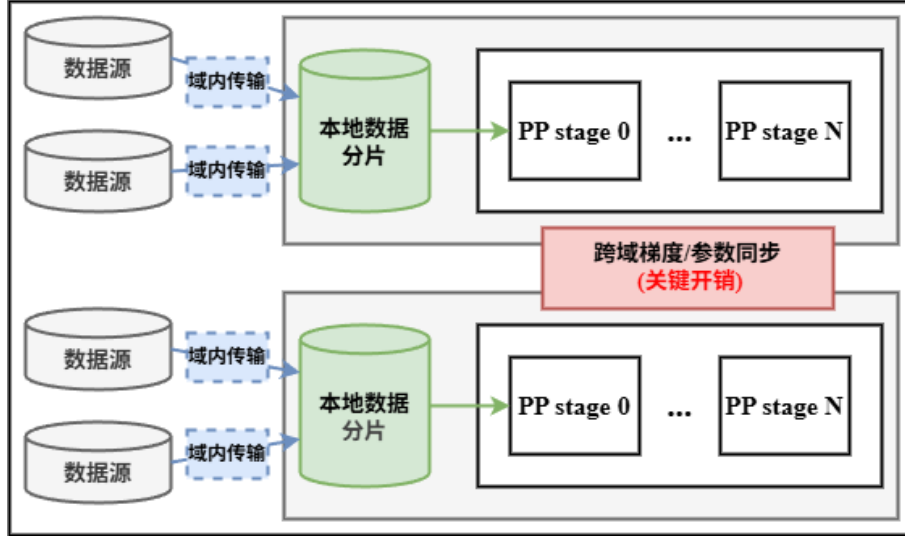


图 5.2 跨域数据并行 + 域内流水线并行：各域处理本地数据，仅跨 WAN 同步梯度

为进一步说明跨域数据并行优于跨域流水线的结构性原因，可以用通信量随 batch 规模的增长趋势来刻画。设共有 D 个域，每个域需要处理本域内的数据 batch b ，则全局 batch 为 $B = D \cdot b$ 。令 α 表示每个参数参与同步的字节数（与精度/压缩有关）， P 表示模型参数规模（以参数个数或字节计）， A 表示当流水线切分跨域训练时每个样本需要跨 WAN 传输的激活（及其反向激活梯度）大小。

当跨域训练采用数据并行时，跨 WAN 的通信主要来自每步一次的梯度同步，其通信量近似与模型规模相关：

$$V_{DP} \approx \alpha \cdot P \quad (5.1)$$

在模型固定时， V_{DP} 随 B 增加近似不变，从而更容易通过增大 batch 或提升域内计算并行来摊薄 WAN 固定开销。

当跨域训练采用流水线并行时，一旦流水线切分跨域，则每个 micro-batch 都需要跨 WAN 传输激活，迭代内 WAN 流量与样本数近似线性增长：

$$V_{PP} \approx \Theta(B \cdot A) \quad (5.2)$$

并且流水线并行的跨域传输位于受限通信链路的路径之上，并且每个流水线 stage 间的计算强依赖于流水线间的激活值传输，一旦 WAN 出现波动，容易级联放大为全流水线停顿。因此，本文选择跨域进行数据并行和域内采用流水线的层次化骨架，以在数据就地训练的前提下，尽可能将 WAN 通信从强依赖链路中剥离。

综上，本章聚焦本文设定的层次化混合并行：跨域采用数据并行，域内采用流水线并行或其他的模型并行方法。该并行策略的设置避免了跨域传输激活/样本的开销，但数据并行策略骨架引入了新的关键瓶颈：跨域梯度同步尾部。

5.1.2 同步尾部成为主要瓶颈

在 WAN 延迟主导时，即使框架在反向传播期间做了梯度分桶来使梯度同步 All-Reduce 与计算重叠，也容易在每次训练迭代末尾出现无法被剩余计算掩盖的通信同步阻塞尾部。原因在于，WAN 的 T_{ar} 可能大于单个 bucket 之后剩余的可用计算窗口，导致最后几个 bucket 的同步受限于串行堆叠，形成长尾。图 5.3 展示了 DP+PP 配置下跨域与域内开销分解：跨域同步在每步中占据一定比例，导致 GPU 等待。

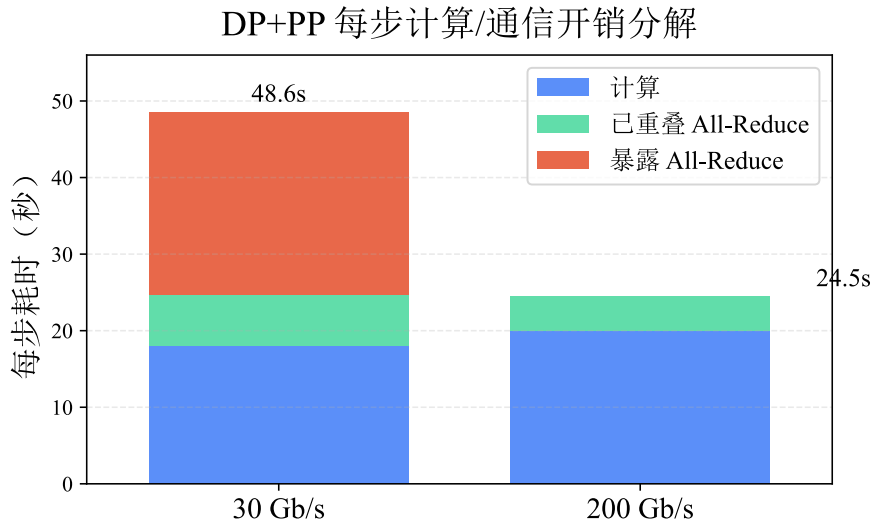


图 5.3 DP+PP 配置下每步计算/通信开销分解：跨域同步成为主要等待来源

面对同步尾部，一类直观做法是弱化同步(如 Local-SGD)，该类做法通过在各个计算域内部维护一个局部模型，并通过约定好的同步频率来维护各个域的模型参数一致，但这类方法的本质是降低同步频率而非消除同步尾部，且会引入更强的优化漂移风险^[35]。本章提出在通信受限场景下的混合并行通信掩盖策略(本文简称其为 POLAR-SGD)：在不改变“每迭代一次全局同步”这一基本语义的前提下，通过前缀触发的异步 All-Reduce + 预测误差修正，创造了一个更早、更长的可重叠窗口(在 micro-batch 前缀处触发一次 all-reduce 集合通信)，让整个跨 WAN 的 All-Reduce 得以被后续反向传播的通信所掩盖，从而将同步从阻塞尾部中去除。

5.2 问题分析与研究思路

考虑一个 DP+PP 混合训练过程：为减少显存负载压力并缓解各阶段 GPU 气泡 (Bubble)，系统通常将一个全局迭代 (step) 的单个批次 (mini-batch) 进一步切分为 M 个更细粒度的微批次 (micro-batch)。在执行引擎层面，常采用 1F1B (One-Forward-One-Backward) 调度执行，即打破单纯的串行执行逻辑，令处于流水线前后的设备交替执行前向与反向传播。但在单次迭代的末尾，这种流水线必须经历一个排空阶段 (Flush)，不再下发新的数据批次，最终使得所有设备退避到同步状态。

在本章的实验拓扑映射中，跨域使用 DP (DP 组大小为 N)，同一域内使用 PP (用于模型切分与计算流水线)。

表 5.1 POLAR-SGD 记号表

符号	含义
M	每个 mini-batch 的 micro-batch 数
m	micro-batch 索引
t	全局迭代索引
r	DP 组内 worker (rank) 索引
$g_{r,t,m}$	worker r 在迭代 t 、micro-batch m 的梯度
$g_{r,t}^{\tau}$	前缀累积梯度：从 $m = 1$ 到 $m = \tau$ 的累积
$g_{r,t}^M$	完整累积梯度：从 $m = 1$ 到 $m = M$ 的累积
$g_{\text{pred},(r,t)}$	截断点处构造、送入 All-Reduce 的预测梯度 (缩放后)
$g_{\text{sync},t}$	All-Reduce 的同步梯度 (DP 组内均值)
$e_{r,t}$	误差缓冲 (error buffer)
$s_{r,t}$	通信发送缓冲 (send buffer)
h_t	异步通信句柄 (handle)
w_t	迭代 t 的模型参数
η	学习率
τ	通信触发的截断 micro-batch 索引

在该设定下，域内 PP 负责把模型切分成多层以提升显存可承载的模型规模，并通过 1F1B 将多个 micro-batch 的前向/反向交错执行来提高单域内的流水线利用率；跨域 DP 则要求每步获得一致的更新方向，需要对 DP 组内的梯度做均值 All-Reduce。该 All-Reduce 因 WAN 特性成为尾部瓶颈。

在一个迭代 t 内、DP 组内的 worker (rank) r 上，micro-batch m 的局部梯度记为 $g_{r,t,m}$ 。我们关心如何选择一个 micro-batch 截断点 τ ，在 $m = \tau$ 时启动一次非阻塞 All-Reduce，并利用误差反馈在后续迭代补齐未同步的梯度信息。

5.3 详细方案设计与实现

图 5.4 对比了标准 DP+PP 与 POLAR-SGD 的单次迭代时间线，可以看到 POLAR-SGD 通过前缀触发的异步 All-Reduce 缩短同步尾部，每一行代表着在各个 DP 组内的各个 DP 进程中的对等 PP rank 的行为。其中，红色块代表一个微批次的前向传播，白色块代表一个微批次的反向传播，紫色块代表在反向传播结束后的梯度通信同步。标准做法往往在处理完全部 M 个 micro-batch 的反向传播后，才在迭代尾部执行阻塞式 All-Reduce，形成明显同步尾部；POLAR-SGD 则在 micro-batch 前缀完成时 ($m = \tau$) 触发一次非阻塞 All-Reduce，并放入独立通信 stream，使其与后续 $(M - \tau)$ 个 micro-batch 的反向传播重叠，从而缩短迭代尾部。

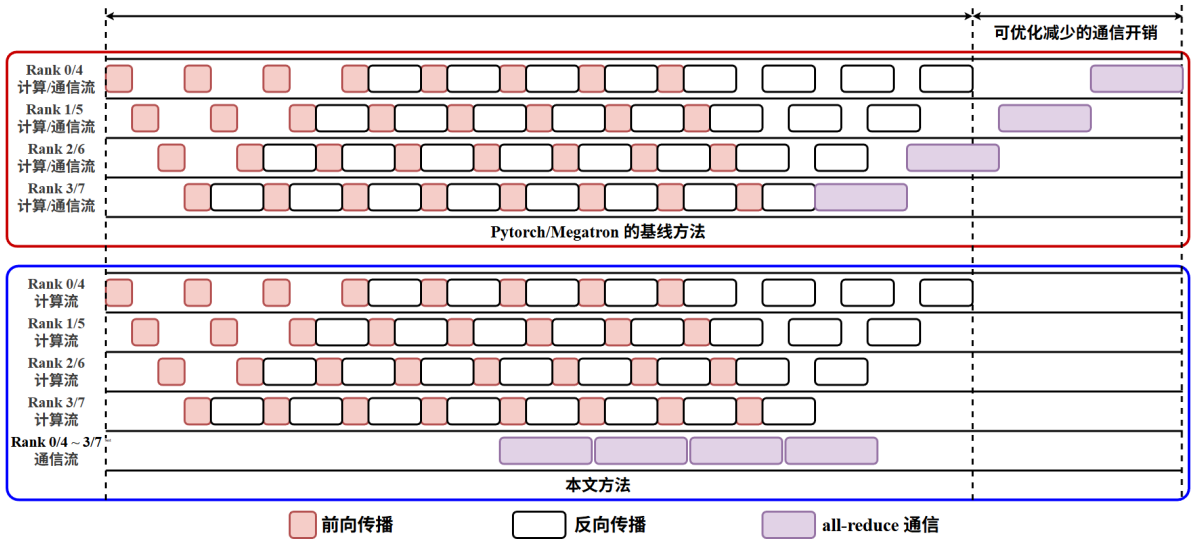


图 5.4 DP+PP 下的执行时间线对比

POLAR-SGD 通过梯度缩放 + 误差反馈将后缀信息以残差形式注入下一次迭代，以同时保持系统重叠收益与优化语义稳定性。POLAR-SGD 的设计可概括为系统效率、语义约束、优化稳定性三者并重：在系统层面尽可能将跨域 All-Reduce 前移到更早位置，以获得更长的计算重叠窗口并缩短迭代尾部；在语义层面保持“每迭代一次 DP 同步”的宏观节奏，避免完全异步导致的严重的梯度滞后 (staleness)；在优化层面通过可控残差通道补齐后缀梯度信息，使收敛轨迹尽量贴近基线同步训练。

5.4 前缀触发的异步梯度同步

本节进一步给出 POLAR-SGD 在单次训练迭代内的具体同步流程，说明如何选择触发点、构造前缀梯度并在独立通信流上发起非阻塞 All-Reduce。整体思路是在不改变每

步一次全局同步语义的前提下，将原本位于迭代末尾的阻塞通信前移到仍有反向计算可执行的窗口中，从而为后续的梯度缩放与误差反馈机制提供执行基础。

5.4.1 梯度 All-Reduce 的触发时机

POLAR-SGD 在每个 micro-batch 反向结束时累积梯度。当到达截断点 $m = \tau$ 时，对当前前缀梯度做快照并构造发送缓冲 $s_{r,t}$ ，随后触发一次非阻塞 All-Reduce；在 $m = M$ （mini-batch 结束）时等待通信完成，得到 $g_{\text{sync},t}$ 并执行优化器更新。

算法 5.1: POLAR-SGD 的前缀触发异步 All-Reduce（每迭代一次）

全局状态： 累积梯度 g_a ；前缀快照 g^τ ；通信句柄 h_t
回调： 在每个 micro-batch 反向结束时调用

```

1:  $g_a \leftarrow g_a + g_{r,t,m}$   $\triangleright$  PP 的梯度累积
2: if  $m = \tau$  then
3:    $g^\tau \leftarrow g_a$   $\triangleright$  记录前缀梯度快照
4:    $s_{r,t} \leftarrow \text{BuildSendBufferAtCutoff}(t, g^\tau)$ 
5:    $h_t \leftarrow \text{AsyncAllReduceMean}(s_{r,t})$ 
6: end
7: if  $m = M$  then
8:   wait ( $h_t$ )  $\triangleright$  得到同步梯度  $g_{\text{sync},t}$ 
9:    $\text{OptimizerStep}(g_{\text{sync},t})$ 
10:   $\text{UpdateError}(t, g_a)$   $\triangleright$  用完整累积梯度更新误差缓冲
11:   $g_a \leftarrow 0; h_t \leftarrow \text{NULL}$ 
12: end

```

5.4.2 截断点 τ 的选择规则

令 T_{ar} 表示本次梯度 All-Reduce 的平均通信时延估计， $T_{\text{comp}(\tau)}$ 表示在 $m = \tau$ 触发通信后，迭代内剩余的计算时间（通常来自后续 micro-batch 的反向传播）。本文采用最早可完全掩盖的截断点：

$$\tau^* := \max\{\tau \in \{1, \dots, M\} : T_{\text{comp}(\tau)} \geq T_{\text{ar}}\} \quad (5.3)$$

实践中，可通过滑动窗口在线 profiling 获得 T_{ar} 与 $T_{\text{comp}(\tau)}$ 的估计：较小 τ 带来更长的可重叠窗口，但也会加大对误差反馈的依赖；较大 τ 则更接近基线的同步语义。

当对所有 τ 都满足 $T_{\text{comp}(\tau)} \geq T_{\text{ar}}$ 时，可以退化为 $\tau^* = M$ ，此时算法行为接近在所有微批次迭代末同步的基线。该退化机制使得 POLAR-SGD 能在网络条件较好或模型计算较短等情形下自动回到保守策略。

在工程实现上, T_{ar} 可由最近若干步 All-Reduce 的完成时延统计得到; $T_{\text{comp}(\tau)}$ 则可由 profiler 记录从 micro-batch τ 反向完成到本迭代结束 ($m = M$) 的剩余反向时间估计。为了避免 τ 在相邻迭代间剧烈抖动, 通常会对估计值做滑动平均, 并在更新 τ 时加入简单的迟滞 (例如只有当新 τ 带来显著收益时才切换)。

5.5 预测误差修正

前缀触发意味着本次迭代的同步更新方向来自前缀梯度。为避免后缀 micro-batch 的梯度信息被忽略, POLAR-SGD 通过前缀放缩的机制构造全 batch 尺度的预测梯度, 并维护误差缓冲记录预测与真实完整累积之间的偏差。

5.5.1 前缀梯度与完整梯度

在 worker r 的迭代 t 中, 定义:

$$g_{r,t}^{\tau} := \sum_{m=1}^{\tau} g_{r,t,m}, \quad g_{r,t}^M := \sum_{m=1}^M g_{r,t,m} \quad (5.4)$$

当 $m = \tau$ 时, 累积缓冲 g_a 等于 $g_{r,t}^{\tau}$; 当 $m = M$ 时, g_a 等于 $g_{r,t}^M$ 。

5.5.2 发送缓冲与误差更新

在截断点处构造发送缓冲 (同时作为预测梯度的未归约形式):

$$s_{r,t} = \left(\frac{M}{\tau}\right) \cdot (g_{r,t}^{\tau} + e_{r,t}), \quad g_{\text{pred},(r,t)} := s_{r,t} \quad (5.5)$$

对 $s_{r,t}$ 做 DP 组内均值 All-Reduce, 得到同步梯度:

$$g_{\text{sync},t} = \left(\frac{1}{N}\right) \sum_{r=1}^N g_{\text{pred},(r,t)} \quad (5.6)$$

当迭代内全部 M 个 micro-batch 完成后, 用完整累积梯度 - 预测梯度更新误差缓冲:

$$e_{r,t+1} = g_{r,t}^M - g_{\text{pred},(r,t)} \quad (5.7)$$

该残差包含后缀 $(M - \tau)$ 个 micro-batch 的梯度信息以及前缀预测误差, 并在下一代通过 $e_{r,t}$ 注入到发送缓冲中, 从而以受控方式延迟使用后缀梯度的信息。

从直观上看, $\left(\frac{M}{\tau}\right)$ 的缩放因子承担了把前缀的梯度尺度外推到全 batch 的作用: 如果梯度的统计分布在 micro-batch 维度上相对平稳, 那么 $g_{r,t}^{\tau}$ 的期望规模与 τ 成正比, 乘以 $\left(\frac{M}{\tau}\right)$ 后就与 $g_{r,t}^M$ 更接近。误差缓冲 $e_{r,t}$ 则用于获取外推梯度与后缀梯度信息的偏差, 把这些信息以误差补偿的形式带到下一步同步中。

算法 5.2: POLAR-SGD 的误差反馈（前缀缩放 + 误差更新）

全局状态：误差缓冲 $e_{r,t}$ ；预测梯度 $g_{\text{pred},(r,t)}$ ； τ 与 M

```

1: procedure BuildSendBufferAtCutoff( $t, g_{r,t}^\tau$ )
2:    $s_{r,t} \leftarrow \left(\frac{M}{\tau}\right) \cdot (g_{r,t}^\tau + e_{r,t})$ 
3:    $g_{\text{pred},(r,t)} \leftarrow s_{r,t}$ 
4:   return  $s_{r,t}$ 
5:
6: procedure UpdateError( $t, g_{r,t}^M$ )
7:    $e_{r,t+1} \leftarrow g_{r,t}^M - g_{\text{pred},(r,t)}$ 

```

此外，对 DP 组内取平均，令 $|g_t^M := \left(\frac{1}{N}\right) \sum_{r=1}^N g_{r,t}^M$ 且 $|e_t := \left(\frac{1}{N}\right) \sum_{r=1}^N e_{r,t}$ ，则在误差更新定义下可得到

$$g_{\text{sync}, t+1} = |g_t^M + |e_{t+1}| \quad (5.8)$$

该等式说明：同步更新方向与下一步的平均残差分解获取了本步完整累积梯度的信息；也就是说，后缀 micro-batch 的信息并未被忽略，只是被延迟到下一代中体现。

5.6 工程实现与系统集成

虽然本章重点在算法机制，但在真实的 DP+PP 系统中获得稳定收益，还需要在控制流与执行资源（CUDA stream）层面保证异步通信确实与计算并行。这里给出更贴近工程实现的集成要点。

5.6.1 与 1F1B 流水线的集成位置

在 1F1B 调度中，micro-batch 的反向传播会按固定节奏在各 stage 上完成。POLAR-SGD 的关键是：在每个 DP rank 上，在某个 micro-batch 的反向结束时刻统一触发一次 All-Reduce。实现时通常会在梯度累积完成处设置钩子函数（hook）（或在训练循环里显式判断 $m = \tau$ ），以便在该时刻将 $s_{r,t}$ 交给通信后端。由于各 stage 的反向完成存在相位差，实际系统中更常见的做法是：在每个 DP rank 内先对所有参数做一次完整梯度张量的累积（或以 bucket 形式累积），然后在 $m = \tau$ 时对该累积缓冲做快照并外推梯度，而后进入 All-Reduce 通信环节。这样能避免在每层/每 bucket 上重复触发通信，符合每个训练迭代执行一次 All-reduce 梯度同步的设计初衷。

5.6.2 通信流与计算流的隔离

要实现重叠，All-Reduce 必须在独立的通信流上启动，且计算流不应等待其完成。

一个常见实现骨架是：计算流在 micro-batch $m = \tau$ 反向结束后先记录 event；通信流等待该 event 以确保 $s_{r,t}$ 写入完成后，再启动异步 All-Reduce（如 NCCL `async_op`）；随后计算流继续处理后续 micro-batch 的反向传播和梯度累积，直到 $m = M$ 的更新点才显式等待通信句柄并获取 $g_{\text{sync}, t}$ 。

5.6.3 误差缓冲的存储与开销

$e_{r,t}$ 与模型梯度同形，最直接的实现是在每个 DP rank 上维护一个与梯度张量同大小的 buffer。其更新发生在 $m = M$ 之后：此时已得到完整累积梯度 $g_{r,t}^M$ ，且预测梯度 $g_{\text{pred},(r,t)}$ 在 $m = \tau$ 时已记录。更新 $e_{r,t+1}$ 是一次向量差，额外开销相对一次完整反向传播通常较小，但会带来一定显存占用；因此工程上常结合张量扁平化/分桶来减少 buffer 管理复杂度。

5.7 理论分析

在标准随机优化假设下（ L -光滑、随机梯度无偏、方差有界），本文给出 POLAR-SGD 的收敛性分析：其渐近收敛行为与同步 SGD 一致，且仅额外引入一个由误差缓冲控制的项。

设 $e_t := \left(\frac{1}{N}\right) \sum_{r=1}^N e_{r,t}$ ，当学习率满足 $\eta \leq \frac{1}{2L}$ 时，经过 T 次全局迭代，有：

$$\left(\frac{1}{T}\right) \sum_{t=0}^{T-1} \mathbb{E}[\|\nabla f(w_t)\|^2] \leq \frac{2(f(w_0) - f^*)}{\eta T} + \frac{L\eta\sigma^2}{N} + \frac{L\eta}{T} \sum_{t=0}^{T-1} \mathbb{E}[\|e_t\|^2] \quad (5.9)$$

该上界表明：当按 τ 选择规则使得通信尽量被掩盖、并使误差缓冲保持有界时，POLAR-SGD 的优化稳定性可以与基线同步训练保持接近。

从解释角度看，该上界包含三个互补部分：一是随迭代次数增长而下降的 $O\left(\frac{1}{T}\right)$ 优化项，二是与标准 DP 均值 All-Reduce 一致的 $\frac{\sigma^2}{N}$ 方差缩减项，反映“DP rank 越多，随机梯度方差越小”的统计规律，三是与 $\|e_t\|^2$ 相关的误差项，用于刻画前缀触发近似偏差在 error feedback 机制下的受控程度。

因此， τ 的选择在理论与实践中都扮演折中角色：更早触发（更小 τ ）带来更强的重叠潜力，但也可能使 $\|e_t\|$ 更大；更晚触发则降低误差项，但可能无法完全掩盖通信。

5.8 实验与验证

5.8.1 实验设置

本文在跨域训练约束下评估 POLAR-SGD，并与标准 DP+PP 基线（DDP+1F1B，迭代末阻塞 All-Reduce）以及放松同步的 DiLoCo ($k = 4$) 对比。实验配置如下。

从云边端协同角度看，该实验设置对应跨域 DP（跨云-边）+ 域内 PP（边缘域内流水线）的层次化训练骨架：WAN 的带宽与 RTT 约束刻画边缘↔云（或边缘↔边缘）链路特性，而域内互联与多卡节点刻画边缘域内部的高带宽计算集群。因而，本章实验可用于验证在云边端协同场景中，如何通过更早触发、可重叠的异步 All-Reduce 来降低跨域同步尾部。

表 5.2 实验设置

组件	配置
Model	LLaMA-2 7B
Dataset	WikiText-103
Hardware	32× NVIDIA A100 (40GB), 4 nodes
Network	Bandwidth: 30 Gb/s; RTT: 50 ms
Parallelism	Hybrid DP+PP (PP=8, DP=4); batch=256; microbatch=32
Software	PyTorch 2.5; NCCL 2.23; CUDA 12.1
Metrics	吞吐 (tokens/s), 训练 loss

5.8.2 端到端吞吐

表 5.3 跨域训练约束下端到端吞吐

方法	吞吐 (tokens/s)	相对加速比
DDP+1F1B	35,350	1.00×
DiLoCo ($k = 4$)	62,216	1.76×
POLAR-SGD	66,104	1.87×

从吞吐对比可以看出：在跨域训练约束下，基线 DDP+1F1B 的尾部同步显著拉长了单步时间；DiLoCo 通过减少同步强度/频率获得了较大吞吐提升，但其优化语义偏离基线更明显；POLAR-SGD 在保持每步一次同步的节奏下仍获得最高吞吐，约为基线的 1.87 倍。

该结果的关键意义在于区分了两条不同优化路径：DiLoCo 主要通过降低同步强度来换取系统速度，而 POLAR-SGD 主要通过重排同步时机来提升重叠效率。前者本质上改变了同步语义，后者在每步一次同步的训练语义下优化系统训练速度。因此，当目标是兼顾吞吐与语义一致性时，POLAR-SGD 的方法路线更符合跨域 DP+PP 的实际需求。

从关键路径角度看，吞吐提升与通信总量变化有关，也由通信被计算掩盖的程度有关。即使总通信字节不变，只要将 All-Reduce 前移到可重叠窗口，迭代末端的纯等待区间就会明显缩短。表格中的相对加速比正体现了这一点：优化对象是迭代尾部而非全程任意位置的通信片段，因此收益具有明确因果指向。

此外，POLAR-SGD 与 DiLoCo 的对比还支持本文关于系统优化与优化语义协同的结论。DiLoCo 在系统侧收益较高，但其训练语义偏离基线更明显；POLAR-SGD 在保持步级同步节奏的前提下达到更高吞吐，说明通过异步触发与误差修正可以在不通过降低训练同步频率的语义约束下实现系统增益。这一结果为后续系统集成章节提供了直接证据。

5.8.3 收敛曲线与消融

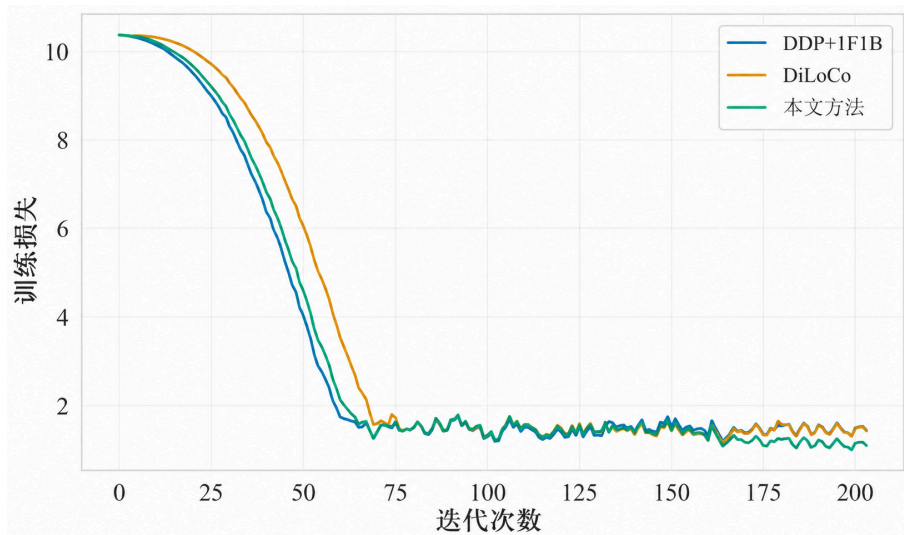


图 5.5 训练 loss 随 step 变化：POLAR-SGD 与基线高度一致，优于 DiLoCo

本文进一步给出了按迭代次数对齐的 loss 曲线（验证训练稳定性）、按墙钟时间对齐的 loss 曲线（验证目标 loss 到达时间），以及对“梯度缩放 / 误差反馈”两项关键机制的消融结果。该组实验的目的并不只是观察最终 loss 是否接近，而是从优化轨迹、系统时间收益和机制必要性三个角度共同验证 POLAR-SGD 的有效性。

若按迭代次数对齐的曲线产生漂移，则说明提前同步改变了优化语义；若墙钟时间曲线没有优势，则说明系统加速未能转化为训练效率；若消融后曲线明显退化，则可进一步定位收益依赖的关键机制。

如图 5.5 所示，按迭代次数对齐的结果回答了算法是否改变每一步优化行为这一问题。POLAR-SGD 的 loss 曲线与 DDP+1F1B 基线基本重合，在训练前期快速下降阶段

和后期趋于平缓阶段均未出现持续偏离，说明前缀触发并没有引入系统性的更新方向偏差。换言之，虽然 POLAR-SGD 在每个 step 内提前使用前缀 micro-batch 构造同步梯度，但经过梯度缩放和误差反馈后，其长期优化轨迹仍能贴近完整梯度同步路径。

该结果也说明 POLAR-SGD 与 DiLoCo 类降低同步频率的方法在优化语义上存在差异。DiLoCo 通过减少全局同步频率获得通信收益，其局部模型会在多个 step 内独立演化，因此 step 对齐曲线更容易表现出与同步基线的间隔；POLAR-SGD 则仍保持每步一次全局同步，只是在 step 内重排同步触发时间，因此更容易维持与基线一致的收敛轨迹。这一点对跨域训练尤其重要，因为系统吞吐提升若伴随明显 loss 漂移，后续还需要额外调参或延长训练来弥补，工程收益会被削弱。

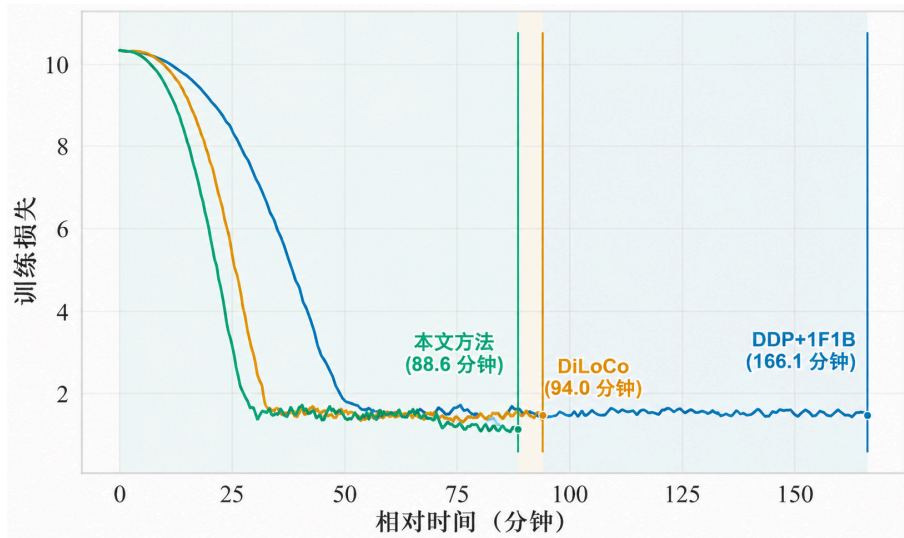


图 5.6 训练 loss 随墙钟时间变化：POLAR-SGD 更快进入低 loss 区间

如图 5.6 所示，按墙钟时间对齐则更直接反映系统收益。由于 step 对齐曲线已经表明不同方法的优化轨迹接近，墙钟时间曲线中的差异主要来自单步迭代时间的缩短，而不是来自更激进或更不稳定的优化过程。POLAR-SGD 曲线更早进入低 loss 区间，说明前文表格中的吞吐提升能够实际转化为 time-to-target 收益，即在相同训练时间预算内完成更多有效 step，或在达到相同目标 loss 时消耗更少实际时间。

从系统角度看，这一结果补充了端到端吞吐表的不足。吞吐指标只说明单位时间内可处理的 token 数增加，但无法单独证明模型质量会随时间同步改善；墙钟时间 loss 曲线则说明 POLAR-SGD 的收益能够落到训练任务最终关心的目标 loss 到达时间上。因此，step 对齐与时间对齐两类曲线构成了互补证据：前者验证语义稳定，后者验证效率收益。

如图 5.7 所示，消融结果强调了 **梯度缩放 (Gradient Scaling)** 与 **误差反馈 (Error Feedback)** 两个机制的必要性。一方面，若不执行 $\left(\frac{M}{\tau}\right)$ 缩放，同步梯度只反映前缀 micro-

batch 的平均贡献，其尺度相对完整 mini-batch 梯度会系统性偏小，导致参数更新幅度不足，表现为 loss 下降速度变慢。该现象说明，前缀触发不是简单地“提前发送部分梯度”即可成立，必须通过尺度校正使同步梯度在期望意义上对齐完整梯度。

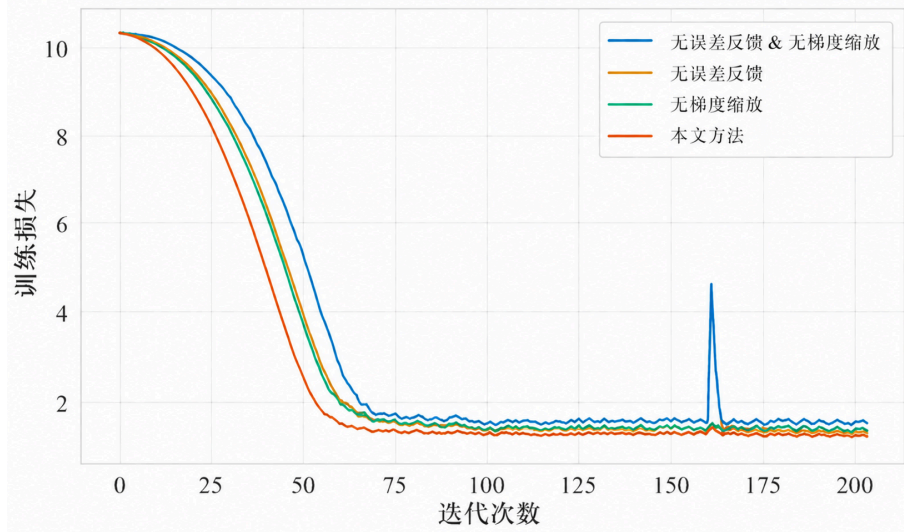


图 5.7 消融实验：去除梯度缩放或误差反馈会带来轻微收敛退化或稳定性风险

另一方面，若不维护误差反馈，后缀 micro-batch 中未被本轮同步完整表达的信息会在当前 step 后直接丢失，造成长期累积偏差。误差反馈的作用在于把前缀近似与完整梯度之间的差异显式保留下来，并在后续迭代中逐步补偿，使每一步的近似误差不会单向累积。消融曲线中去除误差反馈后的退化表明，该机制主要负责控制长期稳定性；而去除梯度缩放后的退化则更多体现为短期更新尺度不足。二者分别对应“单步梯度尺度正确性”和“跨步误差守恒性”，共同保证 POLAR-SGD 在提前通信的同时仍维持接近同步训练的优化行为。

综合三组曲线可以得到更完整的结论：POLAR-SGD 的性能收益来自通信触发时机前移和计算通信重叠，但其可用性依赖于梯度缩放与误差反馈对优化语义的约束。也就是说，本方法并非通过放松同步语义或牺牲收敛质量换取吞吐，而是在保持每步同步节奏的前提下，将原本暴露在迭代尾部的跨域通信转移到可重叠窗口中，并通过误差修正机制限制由前缀近似带来的训练偏差。

5.8.4 网络敏感性分析

在不同网络条件下评估 POLAR-SGD 的性能，可以分析其对 WAN 特性的适应性。理论上，随着 RTT 增加或带宽降低，POLAR-SGD 的相对优势应该更明显，因为其核心设计目标就是掩盖 WAN 同步的尾部；但随着网络进一步劣化，POLAR-SGD 的优势可

能会减弱，因为当通信延迟远大于每次迭代中的反向传播计算时间时，即使将通信前移至 $\tau = 1$ 的位置，仍然会出现无法掩盖的通信开销，此时仅凭 POLAR-SGD 难以完全解决通信瓶颈；相反，如果网络条件较好，POLAR-SGD 可能会退化为与基线类似的行为。

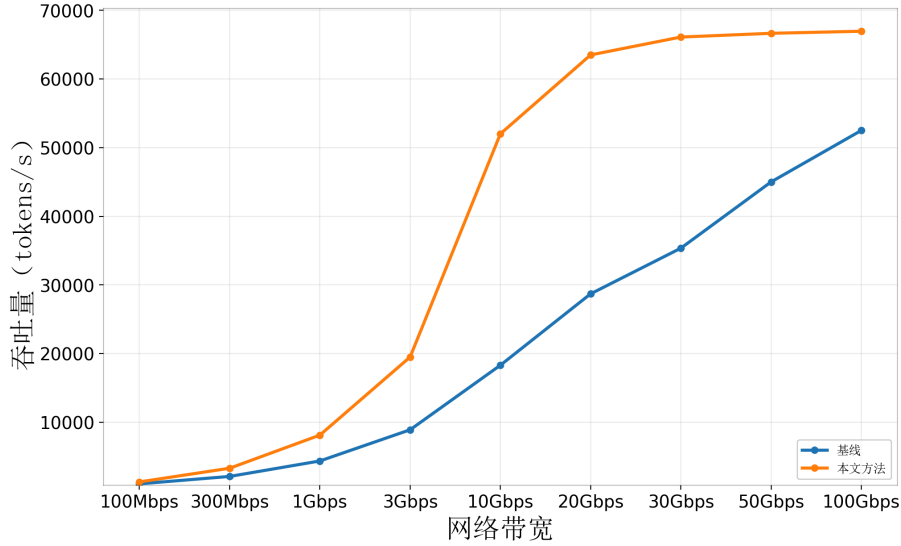


图 5.8 POLAR-SGD 在不同网络条件下的性能表现

如图 5.8 所示，在网络条件较好的情况下，POLAR-SGD 的性能与基线相近；随着网络条件恶化，POLAR-SGD 相对基线的性能提升更明显。但总体来看，在约 20 Gb/s 的环境中，POLAR-SGD 的绝对性能已开始减弱，并在进一步的带宽恶化下，相对基线的性能提升也逐渐降低。这一结果验证了 POLAR-SGD 在 WAN 同步受限场景下的设计初衷，同时也揭示了其适用范围和潜在局限。在极端恶劣的网络条件下，仅靠 POLAR-SGD 可能无法完全解决通信瓶颈问题，此时需要结合前文所述的压缩与调度优化策略进一步提升性能。

5.8.5 截断点敏感性分析

在不同的截断点取值下，POLAR-SGD 的性能表现可能也会产生差异。本文通过实验验证了在不同的截断点取值下，POLAR-SGD 的性能表现是否一致。理论上，较小的截断点（如 $\tau = 1$ ）会带来更长的重叠窗口，但也可能使得误差缓冲中的残差更大，从而增加优化漂移的风险；较大的截断点则更接近基线的同步语义，但可能无法完全掩盖通信。本文据此设计实验，测试 POLAR-SGD 方法是否在不同的截断点取值下具有鲁棒性。

如图 5.9 所示，整体上来说，在流水线并行阶段数为 8，微批次数量为 32 的实验设置下，POLAR-SGD 在不同截断点（ $\tau \in [1, 31]$ ）取值下的性能表现基本一致，并未有显著差别能够证明不同截断点取值对性能产生影响。

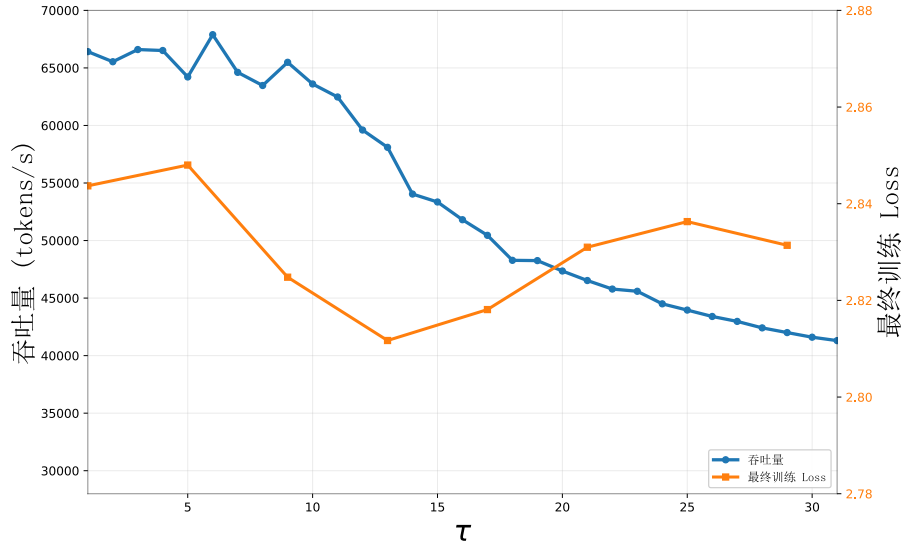


图 5.9 POLAR-SGD 在不同截断点取值下的性能表现

这一结果验证了 POLAR-SGD 的设计初衷，即通过前缀触发的异步 All-Reduce 来掩盖 WAN 同步的尾部，同时通过误差反馈机制来补偿前缀触发带来的偏差，从而在保持每步一次同步的训练语义下实现系统性能提升。该结果还表明，在实际应用中，POLAR-SGD 对于截断点的选择具有一定的鲁棒性，不需要过于精细地调整截断点以获得性能提升。此外，我们还观察到当截断点 τ 取值大于 10 时，吞吐开始下降，并在 τ 取值为 31 时，吞吐下降到与基线相近的水平。这一现象验证了前面关于截断点选择的理论分析：较大的截断点虽然更接近基线的同步语义，但可能无法完全掩盖通信，从而导致性能下降。因此，在实际应用中，选择一个适当的截断点可以在获得最大重叠收益的同时，保持优化稳定性。

5.8.6 实验结论

收敛曲线分析显示，POLAR-SGD 与基线在 step 对齐下保持高度接近，说明前缀触发和误差修正并未破坏每步优化方向。该现象是对方法有效性的核心验证：如果系统加速是以明显训练漂移为代价，则其工程价值会大幅下降；当前曲线结果表明该风险被有效控制。

墙钟对齐曲线进一步解释了吞吐提升如何转化为训练效率提升。由于收敛轨迹接近，单步时间缩短会直接体现为更快达到同等 loss 区间。

消融实验则从反证角度补强结论。去除缩放后出现尺度偏差，去除误差反馈后后缀信息回流不足，两者都导致收敛性能劣化，说明 POLAR-SGD 的有效性来自梯度缩放和误差控制两部分的协同作用。

综合吞吐、收敛与消融三组证据，方法在系统侧缩短尾部等待，在优化侧保持轨迹稳定，在机制层具备可解释的必要性。因此，本章关于 POLAR-SGD 在跨域 DP+PP 场景中的适用性结论具备充分实验支撑。此外，本章还进行了 POLAR-SGD 的网络敏感性分析，验证了其在不同网络条件下的性能表现，证明了方法存在局部有效性，需要结合具体模型与网络条件来评估其适用性。并且说明了其在极端恶劣的网络条件下可能无法完全解决通信瓶颈问题，此时需要结合本文中之前所述的优化策略来进一步提升性能。

5.9 本章小结

本章针对跨域训练条件下 DP+PP 训练的同步尾部问题，提出并验证了 POLAR-SGD 的通信掩盖方案。WAN 高 RTT 使得迭代末 All-Reduce 成为关键路径瓶颈，常规分桶重叠难以覆盖尾部阻塞。于是，本章采用“跨域 DP、域内 PP”的层次化并行骨架，将跨域通信限定为梯度同步；在每迭代只触发一次 All-Reduce 的约束下，引入截断点 τ 与独立通信流提前触发异步同步，并通过前缀缩放与误差反馈机制补齐后缀梯度信息。最终，在 30 Gb/s、50 ms RTT 的跨域训练约束下，方法获得约 1.87 $\times$ 的吞吐提升，并保持与基线接近的收敛轨迹；消融结果进一步证明梯度缩放与误差反馈均为稳定性与效果的关键组成。综上，本章在不改变每步一次梯度同步的训练语义的前提下，通过同步时机重排与误差修正有效削减尾部阻塞。

第六章 面向云边端协同的跨域分布式训练通信优化系统

本章将前文提出的三个通信优化模块统一为一个可部署的系统视角：模块 1（低比特量化压缩）面向通信数据量，模块 2（跨域集合通信流水化）面向通信路径与调度，模块 3（计算-通信掩盖）面向关键路径重叠。通过统一控制与运行时联动，系统可在不同网络状态下动态选择更合适的优化组合。

与前三章叙述逻辑不同，本章旨在构建一个完整的系统闭环：不再局限于讨论单一模块的局部最优，而是重点剖析这三大模块在同一训练迭代循环中的接口契约、触发时序、冲突消解机制以及协同验证方法。本章的核心目标在于：验证三者复杂的实际系统运行环境中能否稳定共存，以及在云边端跨域受限的网络条件下，所取得的通信性能增益能否稳定地转化为端到端模型训练的整体加速。

6.1 引言

从云边端协同训练的部署现实出发，系统设计遵循以下目标与约束：

一方面必须保持每迭代一次全局同步的语义节奏，避免以彻底异步换取短期吞吐；另一方面，性能收益需要体现在多轮训练的端到端时间而非局部算子峰值。与此同时，系统尽量复用 PyTorch 与 NCCL 既有执行栈，将改动集中在训练循环和通信调度层，以控制工程侵入性^[16,56]；当网络状态或模型规模变化使收益收敛时，策略层还应具备可解释、可验证的保守回退路径。

上述约束决定了系统采取“模块 2 常驻 + 模块 3 主导触发 + 模块 1 按需介入”的分层启用模式，而非静态启用全部优化。该模式一方面保留了跨域调度优化的基础收益，另一方面避免在通信不再主导时引入不必要的量化误差与额外管理开销。

6.2 问题分析与研究思路

为降低工程复杂度并提升可维护性，协同模块被放入四层执行架构：

其中训练控制层负责训练迭代周期、并行策略制定和优化器更新时机，策略决策层维护并不断更新当前的状态并据此输出模块组合动作，通信执行层承担分块、流水化和跨域集合通信的启动与句柄管理，误差与状态层则维护误差缓冲、滑动统计量、迟滞变量与回退标志。四层分工使策略决策和通信执行解耦，便于定位瓶颈与演化系统实现。对应到各模块的职责可总结为：模块 1 的职责是进行通信数据量变换，但不改写训练循环时序；模块 2 负责通信路径与并行度组织，但不改变梯度语义；模块 3 负责通信触发时

机重排，在语义约束内调整关键路径重叠关系。通过这种模块职能划分，系统能够在保持语义可控的同时获得组合优化收益。

该职责划分减少了功能交叉，使每个模块可以独立演进，同时保证在系统集成时具备清晰的故障定位路径。

6.2.1 统一符号与状态记号

为避免跨章节符号复用造成歧义，本章将系统集成阶段高频使用的变量统一如下：

表 6.1 第六章系统集成的统一符号与状态记号

符号	含义
t	全局训练 step 索引
m	micro-batch 索引
τ	前缀触发截断点
T_{comm}	本步跨域通信耗时统计量
$T_{\text{comp}(\tau)}$	从 $m = \tau$ 到步末的可重叠计算窗口估计
$s_{r,t}$	rank r 在 step t 的发送缓冲
h_t	本步异步集合通信句柄
$e_{r,t}$	误差反馈缓冲
θ_t	策略状态向量（含阈值、迟滞与回退标志）
a_t	系统动作：模块组合与参数配置

其中， a_t 可取“仅模块 2+3”“模块 1+2+3”“保守回退路径”等离散动作， θ_t 由滑动统计量与阈值控制变量组成。该表示方式使系统策略可被统一建模为“状态到动作”的受约束映射，便于后续实现与测试复现。

6.3 详细方案设计与实现

图 6.1 给出了各通信优化模块的运行时协同机制：模块 2 作为常驻基础能力持续提供可重叠窗口；模块 3 依据在线 profiling 与阈值判断决定是否执行前缀触发异步 All-Reduce；当监测到跨域通信时延上升、掩盖窗口不足时，模块 1 按需触发以进一步压缩通信量并扩大重叠空间。

该闭环可抽象为“监测-判定-执行-反馈”四个阶段：

系统先周期采样跨域通信时延与可重叠计算窗口，再依据阈值关系完成是否充分掩盖的判定；随后在“仅模块 2+3”与“模块 1+2+3”两类策略间做运行时选择，并将本轮执行效果回写到 profiling 与阈值更新逻辑，作为下一轮决策的输入。

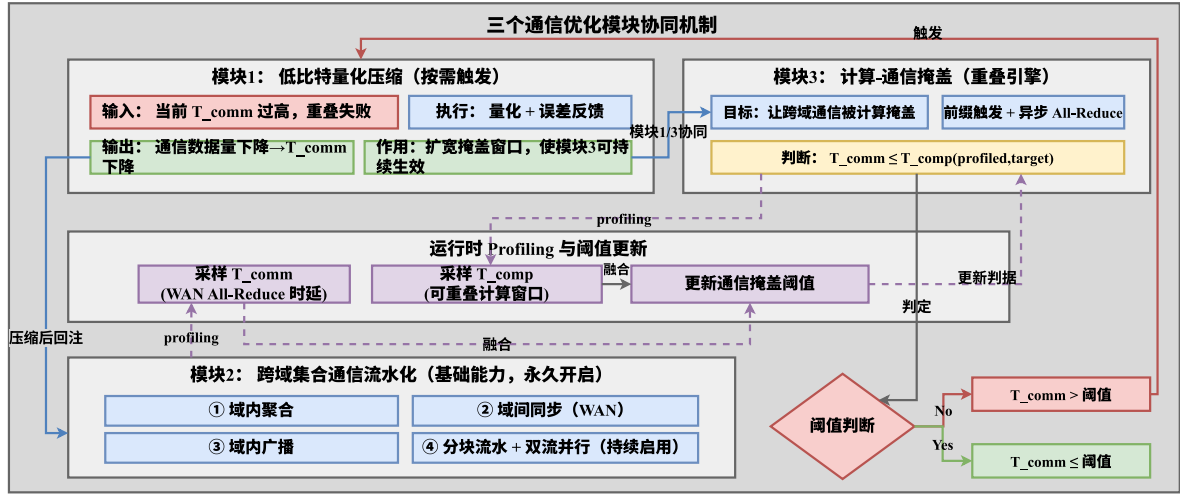


图 6.1 三个通信优化模块协同机制与运行时决策闭环

具体执行时，训练循环先按 DP+PP (1F1B) 完成常规前反向与梯度累积，再在 $m = \tau$ 的截断点由模块 3 触发异步 All-Reduce；模块 2 随后在独立通信 stream 上执行分块流水化集合通信，而模块 1 仅在判定为通信受限时介入发送缓冲压缩。到步末阶段，系统统一等待通信句柄、完成参数更新，并回写误差缓冲与 profiling 状态。

该流程保持每迭代一次全局同步的训练语义，同时把跨域通信尽量前移到可重叠窗口中，以降低尾部阻塞。

6.3.1 模块接口与状态变量约定

为了确保模块可插拔且便于复现实验，系统在实现上约定了统一的输入输出与状态变量：

表 6.2 协同系统集成中的接口与状态变量约定

模块	核心输入	核心输出/状态
模块 1：低比特量化	发送缓冲 $s_{r,t}$ 、量化配置	量化缓冲 $\hat{s}_{r,t}$ 、量化残差统计
模块 2：流水化集合通信	分块张量、通信组信息、stream/event	异步句柄 h_t 、分块完成时间序列
模块 3：计算-通信掩盖	micro-batch 进度、 T_{comm} 、 T_{comp}	截断点 τ 、触发决策、回退标志
统一状态层	本步完整梯度、预测梯度	误差缓冲 $e_{r,t+1}$ 、阈值更新量

该约定的意义在于将算法策略与执行机制解耦：策略层只决定何时触发、是否压缩，执行层只负责正确完成通信与同步，从而降低跨模块耦合度。

6.3.2 运行时决策与回退机制

系统在每个迭代结束时更新一次策略状态，并在下一迭代生效。决策逻辑可概括为：

算法 6.1: 协同通信优化系统控制器主循环

输入状态: θ_t , $e_{r,t}$, 网络与计算 profiling 缓冲
输出: 更新后的参数 w_{t+1} 与状态 θ_{t+1}

```

1: for 每个 step  $t$  do
2:   采样  $T_{\text{comm}}$ 、 $T_{\text{comp}(\tau)}$  并更新滑动统计
3:    $a_t \leftarrow \text{PolicyDecision}(\theta_t, T_{\text{comm}}, T_{\text{comp}})$ 
4:   for 每个 micro-batch  $m = 1, \dots, M$  do
5:     执行前向/反向并累积梯度
6:     if  $m = \tau$  then
7:       根据  $a_t$  选择是否启用模块 1 压缩
8:       在通信 stream 上调用模块 2 启动异步 All-Reduce, 记录  $h_t$ 
9:     end
10:  end
11:  wait ( $h_t$ ) 并执行参数更新
12:  更新误差缓冲  $e_{r,t+1}$  与回退标志
13:   $\theta_{t+1} \leftarrow \text{PolicyUpdate}(\theta_t, \text{观测指标}, a_t)$ 
14: end

```

当 $T_{\text{comm}} \leq T_{\text{comp}(\tau)}$ 时, 系统维持当前策略以优先保证稳定性; 若连续多步出现 $T_{\text{comm}} > T_{\text{comp}(\tau)}$, 则先尝试提前 τ 扩大重叠窗口。若该调整仍不足以覆盖通信尾部, 系统才启用模块 1 进行压缩; 当检测到通信瓶颈缓解后, 策略会逐步降低压缩强度并回退到更保守路径。

该机制能够优先利用不改变梯度数据本身的优化手段, 即第五章通信受限下的混合并行通信掩盖策略, 在必要时再引入数据压缩方法, 即第三章所述跨域通信低比特量化方法, 保证在尽最大可能提升训练系统效率的前提下保证训练过程的稳定。

为将上述策略流程落到可执行实现, 算法 6.1 给出每个迭代的统一控制器主循环。

6.3.3 工程实现细节与开销讨论

在工程实现中, 系统主要新增三类开销:

新增开销主要来自三个方面: 一是时延采样与滑动统计带来的 profiling 成本, 通常显著低于一次跨域 All-Reduce; 二是误差缓冲、策略变量与事件对象管理引入的状态维护成本, 表现为额外显存占用与少量主机侧调度负担; 三是压缩路径开关与分块参数变更带来的短时重配置成本。实验中通过降低 profiling 频率、限制策略更新步长与使用迟滞切换, 能够将上述开销控制在可接受范围内, 使其不抵消多模块协同收益。

6.4 实验与验证

为验证该系统的有效性，本节从系统级视角给出测试项与结果汇总。前文分章实验分别验证了单模块与局部组合能力，本节关注端到端可交付性：性能收益、收敛稳定性与工程一致性是否可同时成立。

6.4.1 系统级实验设置

系统级实验沿用前文的跨域 DP、域内 PP 训练骨架，并将第 3-5 章的单模块实验统一到同一组运行时观测口径下。与单章实验相比，本章不再只验证某个算子或某个通信阶段，而是同时记录完整训练 step 中的计算、通信、策略决策和收敛结果。

表 6.3 第六章系统级实验设置

组件	系统级配置与统计口径
模型与任务	以 LLaMA-7B 语言建模任务为主要系统级负载；固定模型与 batch 实验、带宽扫描实验和 batch size 敏感性实验均在该负载口径下统计吞吐
实验平台	以天数智芯（Iluvatar CoreX）国产 GPU 物理集群为主要验证平台；n210/n62 双节点分别对应 Domain-1/Domain-2，节点内 PCIe 互联，节点间 Ethernet 互联
并行骨架	跨域数据并行 + 域内流水线并行；跨域链路只承担梯度同步，域内负责模型切分与 micro-batch 流水执行
软件栈	复用 PyTorch、NCCL 与 CUDA 分布式训练栈；系统改动集中在训练循环、通信调度、量化压缩与 profiling 控制层
网络条件	通过网络整形注入跨域带宽与 RTT；弱受限为 10 – 30 Gb/s、5 – 30 ms，中度受限为 5 – 10 Gb/s、30 – 100 ms，强受限为 1 – 5 Gb/s、30 – 100 ms
对比方法	现有同步训练系统、DiLoCo、Streaming DiLoCo，以及启用模块 1+2+3 的本文系统
评价指标	端到端吞吐、单步迭代时间、通信时间、time-to-target、最终验证损失或 PPL、退化测试中的回退行为
日志变量	T_{comm} 、 $T_{\text{comp}}(\tau)$ 、 τ 、模块启用状态、异步通信句柄完成时间和误差缓冲状态

需要说明的是，本章系统级实验主要在天数智芯（Iluvatar CoreX）国产 GPU 物理集群上完成，用于验证前述通信优化机制在非 NVIDIA 生态和真实物理跨节点环境中的可部署性。实验平台由两个节点组成，每节点配置 Intel Xeon Gold 6330 CPU 与 Iluvatar BI-V150 GPU；节点内通过 PCIe 互联，节点间通过 Ethernet 互联。网络受限条件在该物理集群上通过链路整形方式注入，从而形成不同跨域带宽与 RTT 分组。相比单模块章节中部分 A100 环境或仿真化微基准，本章更关注完整系统在国产集群上的端到端运行稳定性与综合收益。表 6.3 设置使第六章的结果能够与前文形成对应关系：第 3 章的低比特

压缩影响通信数据量，第 4 章的流水化调度影响通信路径执行时间，第 5 章的前缀触发影响通信是否暴露在关键路径上；而本章统一观察三者叠加后在完整训练 step 中的端到端结果。

6.4.2 测试目标与判据

系统级测试覆盖以下四类判据：

判据体系同时覆盖性能、优化与工程三条主线：性能侧关注端到端 tokens/s 或 samples/s 相对基线是否提升，优化侧关注 step 对齐 loss 曲线是否与基线保持一致并避免明显漂移，同时通过评估达到目标 loss 时间是否实质缩短；工程侧则验证训练循环、通信句柄和误差缓冲更新链路能否长期稳定运行且不破坏训练语义。

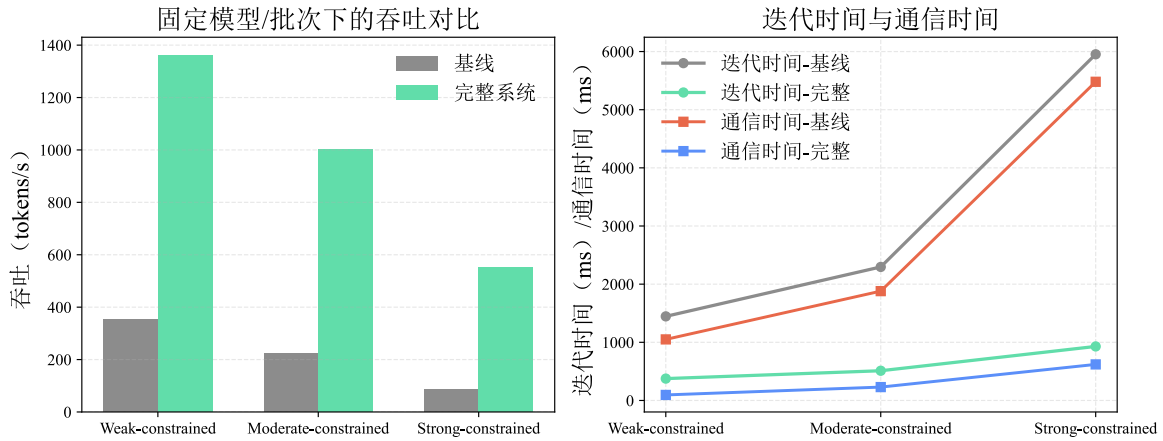


图 6.2 固定模型与 batch 条件下，本系统与基线系统的吞吐、迭代时间与通信时间对比

6.4.3 测试流程

为保证测试结论可复核，系统级验证按照功能正确性，性能收益，稳定性，退化行为顺序执行：

在具体实验组织上，本节补充五组系统级验证：第一组，在固定模型与 batch 条件下开展性能测试，对比基线与协同系统的吞吐、迭代时间和通信占比；第二组，在 LLaMA-7B 训练任务上改变跨域带宽，对比本方法、DiLoCo 与 Streaming DiLoCo 的吞吐适应性；第三组，在固定 5 Gb/s 跨域带宽下改变 batch size，分析 micro-batch 数量与吞吐、最终 PPL 之间的关系；第四组，通过按迭代次数对齐与按墙钟时间对齐的 loss 曲线评估稳定性；第五组，在更高带宽或更低 RTT 条件下执行退化测试，确认系统能够自动回退并保持与基线一致行为。测试矩阵覆盖三类网络区间：弱受限网络区间内通信绝大部分可被计算掩盖，重点观察回退机制是否合理；中度受限网络区间中通信与计算接

近,重点考察阈值策略稳定性及抗抖动能力;强受限网络区间中通信显著主导,重点验证多模块叠加后是否仍具可持续收益。

6.4.4 测试结果汇总

表 6.4 给出了系统级对比结果:在弱受限、中度受限、强受限三类网络环境中,统一启用第 3-5 章全部优化方法(模块 1+2+3)的本系统,均优于现有训练系统。

如图 6.2 所示,在固定模型与 batch 设置下,本系统在弱受限/中度受限/强受限网络区间均获得吞吐提升,同时迭代时间下降、通信时间降低。

表 6.4 不同网络环境下本系统与现有训练系统的最终对比结果

网络分组	链路条件	现有训练系统吞吐 (tokens/s)	本系统吞吐 (tokens/s)	加速比	TTT: 现有/本系统 (min)	最终验证损失
弱受限	10 - 30 Gb/s; 5 - 30 ms	354	1362	3.85×	302/79	0.02
中度受限	5 - 10 Gb/s; 30 - 100 ms	223	1003	4.50×	480/107	0.03
强受限	1 - 5 Gb/s; 30 - 100 ms	86	551	6.41×	1242/194	0.04

从吞吐看,网络约束越强,组合优化收益越明显:弱受限为 3.85×,中度受限为 4.50×,强受限为 6.41×。强受限场景下绝对吞吐低于中度受限区间,但相对基线的提升幅度更大,说明跨域瓶颈越突出,多模块协同对通信关键路径的削弱作用越明显。

从端到端训练效率看,到达目标损失时间在三类网络中均明显缩短,分别由 302/480/1242 min 降至 79/107/194 min,表明吞吐提升能够稳定转化为训练周期收益,而非仅停留在局部算子层面。尤其在强受限网络下,训练效率达到约 6.41× 的提升,说明在极端带宽受限条件中,多模块叠加能够显著缩短达到目标精度所需的墙钟时间。

进一步观察图 6.2 中吞吐与迭代时间的对应关系可以发现,三类网络下吞吐-迭代时间基本保持固定 batch 训练中的反比关系。

基线系统从弱受限到强受限时,吞吐由 354 tokens/s 降至 86 tokens/s,单步迭代时间由 1446 ms 增至 5953 ms;完整系统吞吐由 1362 tokens/s 降至 551 tokens/s,单步迭代时间则由 376 ms 增至 929 ms。也就是说,右图中的迭代时间变化能够解释左图吞吐变化,二者共同说明系统收益来自每个训练 step 的关键路径缩短,而非统计口径变化。从通信时间拆解看,跨域约束增强时通信占比持续上升。基线系统的通信时间从 1050 ms 增至

5480 ms，占单步迭代时间的比例由约 72.6% 增至 92.1%，说明在强受限链路下训练几乎退化为通信等待主导。完整系统中通信时间也随网络受限程度由 95 ms 增至 620 ms，占比由约 25.3% 升至 66.7%，但其绝对通信等待仍显著低于基线。这一现象符合本文各模块的作用边界：优化策略无法消除链路物理带宽下降带来的全部影响，但能够通过压缩、流水化和重叠将通信增长斜率显著压低。因此，强受限网络中完整系统的通信占比仍会上升，但上升后的迭代时间仍远低于基线。

上述结果还表明，各模块收益有叠加效应。弱受限场景中，模块 3 的通信掩盖和模块 2 的流水调度已经能够覆盖大部分通信尾部，因此模块 1 的压缩收益相对有限；中度受限场景中，通信与计算窗口接近，策略层需要在提前触发和压缩开销之间取得平衡，收益主要来自三者协同；强受限场景中，跨域通信成为绝对主导，模块 1 的通信量削减开始发挥更关键的作用，同时模块 2 与模块 3 保证压缩后的数据传输能够尽可能与剩余计算重叠。由此可见，网络越受限，系统越依赖完整模块组合，而不是单一优化手段。

6.4.5 不同带宽下的横向方法对比

为进一步验证本系统并非仅相对传统同步基线有效，本节在 LLaMA-7B 训练任务上补充与 DiLoCo 和 Streaming DiLoCo 的横向对比。DiLoCo 通过减少全局同步频率降低跨域通信压力，Streaming DiLoCo 在此基础上进一步引入流式通信组织；二者代表了跨域训练中较典型的降低同步频率/流式化同步路径。本文系统仍保持每轮迭代一次全局同步的语义，通过通信数据量压缩、层次化流水通信和计算-通信掩盖协同降低同步代价。

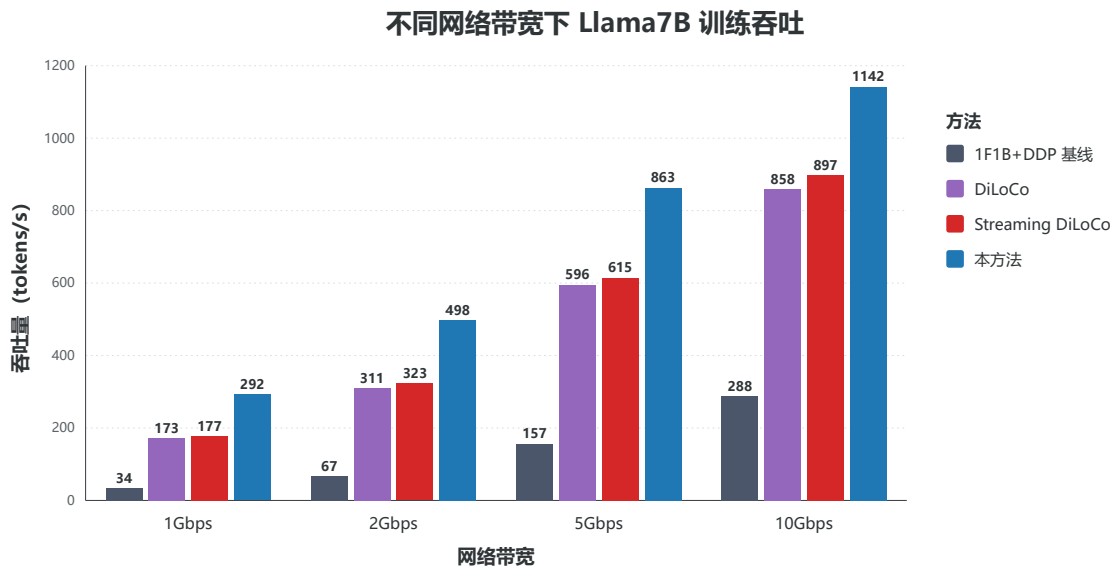


图 6.3 不同跨域带宽下 LLaMA-7B 训练吞吐的横向方法对比

如图 6.3 所示，在 1 Gb/s、2 Gb/s、5 Gb/s 与 10 Gb/s 四种网络带宽下，本方法均取得最高吞吐，分别达到 292、498、863 与 1142 tokens/s。相比 1F1B+DDP 基线，本方法在四种带宽下分别获得约 8.59 $\times$ 、7.43 $\times$ 、5.50 $\times$ 与 3.97 $\times$ 的吞吐提升；相比 Streaming DiLoCo，本方法仍分别提升约 1.65 $\times$ 、1.54 $\times$ 、1.40 $\times$ 与 1.27 $\times$ 。

该结果体现出两个趋势。第一，当跨域带宽较低时，传统同步训练受到 All-Reduce 尾部阻塞影响最为明显，DiLoCo 类方法虽然能够通过降低同步频率缓解通信压力，但吞吐仍受局部更新与跨域同步阶段的交替影响；本文系统则同时削减通信载荷、改善通信路径并提前隐藏通信，因此在 1 Gb/s 与 2 Gb/s 下优势最明显。第二，随着带宽提升至 10 Gb/s，各方法的通信瓶颈均有所缓解，吞吐差距自然收敛，但本方法仍保持领先，说明系统收益并非只来自极端低带宽下的单点压缩，而是来自三类模块对通信关键路径的联合优化。

从方法特性看，DiLoCo 和 Streaming DiLoCo 的优势主要来自降低跨域同步频率或将同步过程流式化，因此它们在低带宽环境中能够避免每步全量同步的直接代价。但这种路线会把部分全局一致性压力推迟到周期性同步阶段，并且对 local step、学习率和数据异质性更敏感。本文系统的优势在于不改变每步全局同步语义，而是在每步内部重排通信执行路径。因而在同等带宽下，本文系统能够在保持同步训练语义的同时获得接近或超过低同步频率方法的吞吐，这对需要严格控制训练轨迹、调试误差和复现实验的跨域训练任务更有意义。

带宽从 1 Gb/s 提升至 10 Gb/s 时，所有方法吞吐均上升，但上升斜率不同。基线方法受全量同步限制最明显，低带宽下几乎被通信完全压制；DiLoCo 与 Streaming DiLoCo 的曲线更平滑，说明降低同步频率确实能减轻带宽敏感性；本文系统在低带宽下提升最明显，在高带宽下则逐渐接近计算侧上限。该趋势说明，当跨域链路从强瓶颈逐渐变为次要瓶颈时，系统的主要收益来源也会从“减少通信量”转向“减少调度尾部和等待空窗”，这与运行时策略中模块 1 按需介入、模块 2/3 常驻的设计一致。

6.4.6 Batch Size 敏感性分析

模块 3 的核心收益依赖可重叠计算窗口，而该窗口与 micro-batch 划分直接相关。为分析该因素对系统行为的影响，本节在 5 Gb/s 跨域链路下固定模型与通信配置，改变 batch size，并同时观察训练吞吐与最终 PPL。

该实验用于回答两个问题：一是更大的批次是否能为通信掩盖提供更充分窗口；二是吞吐提升是否伴随明显的模型质量退化。

如图 6.4 所示, batch size 从 16 增大至 64 时, 各方法吞吐均随可并行计算量增大而提升, 但本文方法的提升幅度更显著: 吞吐由 863 tokens/s 提升至 1933 tokens/s, 始终高于 DiLoCo 与 Streaming DiLoCo。在 batch size 为 32、48、64 时, 本方法相对 Streaming DiLoCo 的吞吐提升分别约为 1.30 $\times$ 、1.17 $\times$ 与 1.24 $\times$ 。

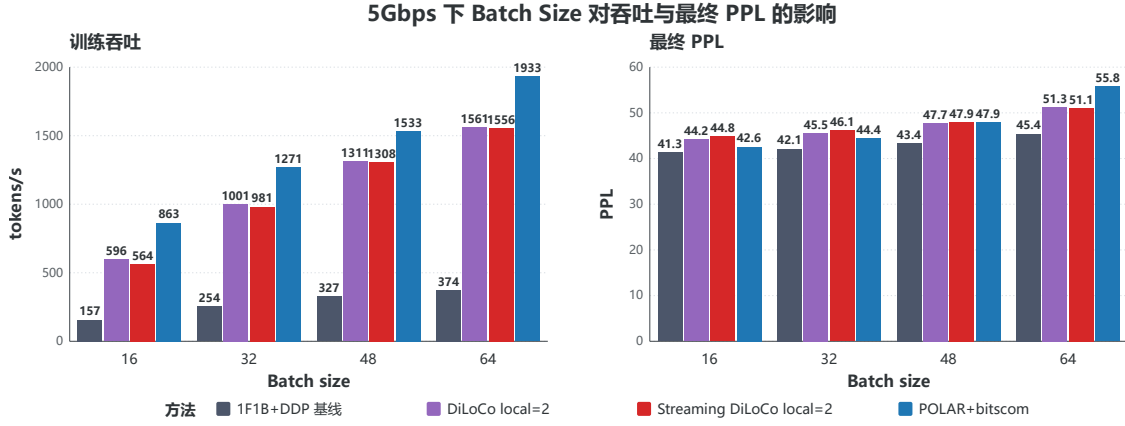


图 6.4 5 Gb/s 跨域带宽下 batch size 对吞吐与最终 PPL 的影响

从机制上看, batch size 增大通常对应更多 micro-batch 或更长的局部反向计算窗口, 使 $T_{\text{comp}(\tau)}$ 更容易覆盖跨域通信耗时 T_{comm} 。因此, 模块 3 能够更充分地把跨域同步前移至可重叠区间, 模块 2 的分块流水通信也拥有更稳定的并发执行窗口。与此同时, 最终 PPL 随 batch size 增大而上升, 说明吞吐收益与模型质量之间仍存在常见的大 batch 训练权衡。本文方法在 batch size 为 16 和 32 时最终 PPL 分别为 42.6 和 44.4, 与基线差距较小; 当 batch size 增至 64 时 PPL 升至 55.8, 表明在追求最高吞吐时仍需配合学习率、warmup 或局部 batch 策略进行调参。该结果也说明, 系统默认策略不应单纯追求最大 batch, 而应将吞吐、目标精度和可重叠窗口共同纳入部署配置。

batch size 从 16 增至 32 时, 本文方法吞吐由 863 tokens/s 提升到 1271 tokens/s, 属于较高收益区间; 继续增至 48 和 64 时, 吞吐进一步提升至 1533 和 1933 tokens/s, 但最终 PPL 也逐步上升。这说明在 5 Gb/s 链路下, 较小 batch 时系统仍受可重叠窗口不足限制, 而中等 batch 能显著改善计算-通信重叠; 当 batch 继续增大后, 吞吐收益仍存在, 但训练质量代价开始更明显。换言之, batch size 存在由通信掩盖收益和优化质量共同决定的折中区间。

该实验也补充解释了在线策略的必要性。不同带宽下 T_{comm} 变化明显, 不同 batch size 下 $T_{\text{comp}(\tau)}$ 也会变化; 若固定提前触发点 τ , 在小 batch 下可能过晚触发、留下通信

尾部，在大 batch 下又可能过早触发、增加预测和误差反馈压力。因此，策略层必须持续根据通信耗时和可重叠窗口更新触发位置，并在收益不足时回退到更保守的同步路径。

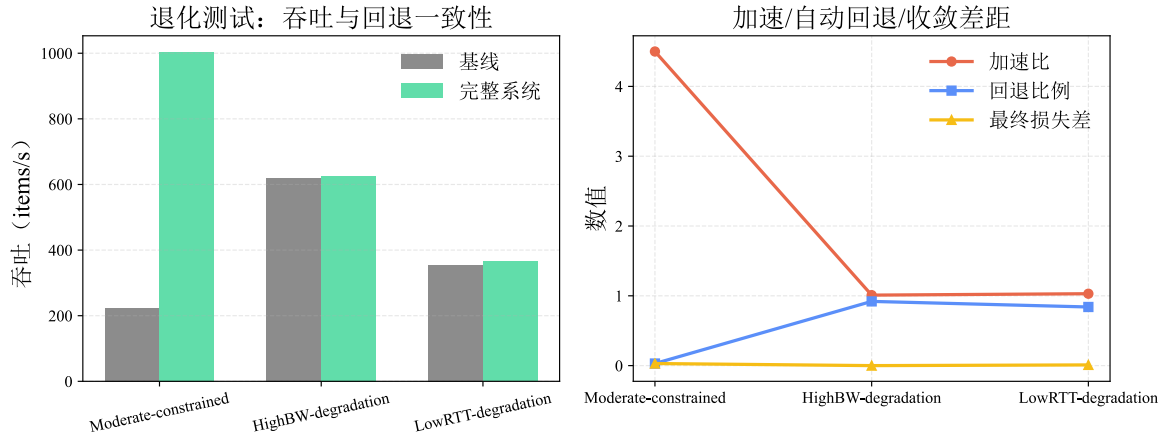


图 6.5 高带宽/低 RTT 退化测试下的自动回退与一致性验证

如图 6.5 所示，当网络条件表现为高带宽与低 RTT 时，通信不再是迭代的限速瓶颈。此时，系统策略决策层通过在线 Profiling 监测到通信时间 T_{comm} 持续小于等于可掩盖窗口 $T_{\text{comp}}(\tau)$ 。在这种状态下，系统主动关闭了模块 1 的低比特量化流程，避免了不必要的量化开销与精度扰动，使加速比自然收敛至接近 1。

退化测试的意义在于验证系统不会在不需要优化时强行引入额外复杂度。在高带宽或低 RTT 环境中，通信时间本身已经可以被计算阶段自然吸收，此时继续执行低比特压缩会带来编码、解码、残差维护和误差反馈等额外开销，且可能引入不必要的数值扰动。因此，加速比收敛到 1 并不是方法失效，而是策略层正确识别通信已不成为瓶颈后的预期行为。该结果说明本文系统具备可解释的保守性：在受限链路下积极优化，在非受限链路下主动减少干预。

综上，第 6 章在不同网络环境下同条件基线对比、与 DiLoCo 类方法的横向对比、batch size 敏感性分析以及退化测试中，均能够支撑模块间协同方案在真实跨域受限场景下的有效性结论。

6.4.7 适用范围与失效模式

尽管系统在跨域受限网络下表现稳定，仍存在明确适用范围：当模型计算量较小且通信量本身不高时，策略层的可优化空间会显著缩小；当网络抖动表现为强突发且难预测时，短时收益可能出现波动；当 micro-batch 数过少时，可重叠窗口不足，也会限制模块 3 的作用上限。另一方面，过大的 batch size 虽可提高吞吐并增强通信掩盖效果，但也

可能带来最终 PPL 上升，因此实际部署中需要结合目标精度、学习率策略与训练预算选择合适的 batch 配置。对应地，系统提供两类失效回退：一是通过阈值驱动回退到保守同步路径，以避免持续误判；二是限制策略切换频率，以防止过度自适应引入额外不稳定性。该设计使系统在非理想网络下仍可维持“可运行、可解释、可回退”的工程属性。

6.4.8 可复现性与部署建议

为便于后续复现与部署，交付材料如下：固定版本的软件栈（PyTorch、CUDA、NCCL）及关键环境变量、统一的 profiling 日志格式（含 T_{comm} 、 T_{comp} 、 τ 与策略状态）、以及与表 6.4 对齐的最小测试脚本和验收阈值^[16,56]。

6.5 本章小结

本章聚焦多模块可协同的系统问题，将低比特量化、层次化流水线通信与计算-通信掩盖三类机制整合为统一运行时控制闭环。方案层面，构建了分层执行架构与清晰的接口契约，形成“监测—判定—执行—反馈”的决策流程，并提供了可解释的回退路径，确保在不同网络条件下保持训练语义一致性。效果层面，本章主要在天数智芯（Iluvatar CoreX）国产 GPU 物理集群上进行系统级验证；测试在弱受限、中度受限与强受限网络下均取得稳定的吞吐提升与达成目标损失时间缩短，同时 step 对齐曲线保持与基线一致；LLaMA-7B 横向实验表明，本方法在 1-10 Gb/s 带宽区间内均优于 DiLoCo 与 Streaming DiLoCo；batch size 敏感性实验进一步说明，micro-batch 带来的可重叠窗口是影响通信掩盖效果的重要因素。在高带宽/低 RTT 条件下，系统能够自动回退并保持与基线一致的行为。综合来看，本章证明了模块协同不仅具备叠加收益，而且能够在国产 GPU 物理集群上稳定部署，为跨域受限场景提供了端到端可交付的系统级解决方案。

总结与展望

论文工作总结

本论文面向云边端跨域分布式训练中的通信数据规模大，通信链路间异构，训练尾部同步延迟高等挑战，开展面向云边端跨域分布式训练的低比特优化器量化方法、基于流水线的层次化通信调度策略、混合并行通信掩盖策略等系统性研究，优化跨域训练通信中的三个关键环节（数据规模大小、通信算法优劣、计算通信掩盖程度）提出了一套高效的跨域协同训练通信优化方案，提升云边端跨域分布式训练效率。本文的主要工作和成果总结如下：

（1）针对跨域受限链路下数据交换载荷过大导致的训练吞吐受限问题，提出一种分布式优化器低比特量化方法。首先，通过 k -bit 随机舍入量化分布式压缩机制，将连续浮点梯度映射为低比特表示的有限的离散数值；其次，通过带有残差补偿的量化同步与低位聚合算法，以缓解位宽降低带来的累积噪声，并优化了单次同步的通信带宽开销。实验证明，该方案在低带宽场景下显著降低了梯度通信同步开销，且加速优势随网络受限程度逐渐增强。基于随机舍入量化，该方案在受限网络下可大幅压缩梯度通信量，并可保障收敛稳定性；通过低位宽全归约方法，该方案克服了低位宽数据和高效集合通信方法不兼容的问题。

（2）针对跨域链路中跨域链路异构且域间网络带宽低导致的训练中集合通信执行效率低的问题，提出一种基于流水线的层次化梯度通信调度策略。首先，通过引入分层集合通信算法，将全局各节点的全归约通信分解为域内归约、跨域传输与域内广播三个阶段，降低了域间通信量；其次，通过引入张量分块级的流水线机制，将域内规约、跨域传输与域内广播三阶段按切分的张量块流水执行，优化了跨域链路间通信时域内链路的利用率。实验证明，该方案优化了跨域异构集群间数据聚合的调度重叠率，充分利用了域内部的高速带宽。通过引入分层集合通信算法，降低了原始集合通信在云边端跨域场景下在域间的通信时间，大幅消除了通信启动延迟。通过引入张量块流水执行，并行了域间和域内通信，降低了原始分层集合通信算法的整体开销。

（3）针对跨域训练在单次迭代末尾极易产生不可掩盖的阻塞式同步尾延迟问题，提出一种面向通信受限场景下的混合并行通信掩盖策略。首先，通过在反向传播阶段提前触发非阻塞集合通信，为梯度同步的长尾部预留足够的计算掩盖窗口；其次，利用张量缩放策略，来预测推算尚未计算完成的梯度数据并将其纳入跨域梯度同步，降低由于提

前触发通信导致的梯度数据尺度不一致的问题；最后，利用轻量级误差反馈，将预测梯度值与剩余梯度的偏差延迟补偿至下一计算迭代，去除了由于使用仅部分数据的梯度信息产生的信息损失。实验证明，该方案优化了设备本地计算时间与跨域通信时长的掩盖比率，最终成功消除了由跨域 All-Reduce 造成的尾部纯等待空窗，且维持了每迭代一次全局同步的语义。

下一步工作展望

本文解决了在云边端跨域分布式训练通信优化中的三类关键问题，优化了跨域训练通信效率，提升了跨域训练的整体性能。但这些方法仍具有一定的改进空间，其中主要包含以下三方面：

(1) 在低比特优化器量化方法中，本文采用了基于块量化的随机舍入量化策略和量化误差补偿技术来保证训练精度，但是在实际应用中，针对不同模型结构、训练任务和数据分布，单一的量化策略并不总是最优解。固定的块大小可能无法始终保持着最优精度解，依据如下。对于不同的模型结构、训练任务和数据分布，梯度数值的分布和异常值频率很有可能不一样。对较大的异常值频率而言，较小的块大小可能更有利于捕捉异常值，从而保持训练精度，但提升了传输的数据量；而对于较小的异常值频率而言，较大的块大小可能更有利于提高压缩率，但提升了当某块出现异常值时对该块总体量化数值影响的规模。因此，未来的工作可以考虑引入动态块量化方法，根据模型结构、训练任务和数据分布的不同，动态调整块大小，得到一个最优块粒度取值。提升训练精度的同时，以进一步提升量化优化器的性能。其次，本文的低比特全归约方法还可进一步提升效率，通过重写 NCCL All-Reduce Kernel，将 All-to-all 和本地反量化规约操作融合，实现通算融合。由于 All-to-all 通信的特点是每个节点都会依次收到来自其他节点的部分数据，因此在接收数据的同时就可以进行反量化规约操作，避免了先完成 All-to-all 通信再进行反量化规约的两阶段过程，从而进一步提升通信效率。

(2) 在基于流水线的层次化通信调度策略中，本文提出了基于张量切分的流水线机制和双流并行执行来提升跨域链路的利用率，但在实际应用中，训练节点间的通信拓扑结构可能更加复杂，正像本文所述，不同的链路结构需要不同的通信算法来达成最好的性能表现。仅仅将云边端链路解构为域内和域间来分析，没有结合更细粒度的拓扑感知是这一方法的缺点。因此，未来的工作可以考虑引入更细粒度的拓扑感知机制，来适应更复杂的通信拓扑结构。通过对训练节点间的通信拓扑进行更细粒度的分析和建模，识

别出不同链路之间的性能差异和瓶颈，进行通信算法的微调。从而设计出更加适应实际通信拓扑结构的调度策略。提升跨域链路的利用率，进一步提升跨域训练的整体性能。

(3) 在混合并行通信掩盖策略中，本文提出了在每步一次全局同步的语义下，将 **All-Reduce** 触发点前移并引入预测误差修正来消减迭代尾部的等待。在实际应用中，大型模型的训练实现范式已经偏向于 **Megatron**、**DeepSpeed** 的整体架构，但我们的工作仅支持在 **PyTorch** 上进行训练，没有兼容更多的并行策略，例如张量并行、序列并行和上下文并行等。因此，未来的工作主要考虑将我们的方法适配进入 **Megatron**，从而支持各类大模型训练的并行策略，将本方法在业界真正落地，在更大规模的模型训练中验证其有效性，并扩展至真实预训练任务中验证其有效性。

参考文献

- [1] VASWANI A, SHAZEER N, PARMAR N, et al. Attention Is All You Need[A]. 2023(arXiv:1706.03762) [2024-03-22].
- [2] BOMMASANI R, HUDSON D A, ADELI E, et al. On the opportunities and risks of foundation models[J]. CoRR, 2021, abs/2108.07258.
- [3] DEVLIN J, CHANG M W, LEE K, et al. BERT: pre-training of deep bidirectional transformers for language understanding[C]//Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers). Association for Computational Linguistics, 2019: 4171 – 4186. DOI: 10.18653/V1/N19-1423.
- [4] BROWN T B, MANN B, RYDER N, et al. Language models are few-shot learners[C]//Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual. 2020.
- [5] CHOWDHURY A, NARANG S, DEVLIN J, et al. Palm: scaling language modeling with pathways[J/OL]. CoRR, 2022, abs/2204.02311. DOI: 10.48550/ARXIV.2204.02311.
- [6] TOUVRON H, LAVRIL T, IZACARD G, et al. Llama: open and efficient foundation language models[J/OL]. CoRR, 2023, abs/2302.13971. DOI: 10.48550/ARXIV.2302.13971.
- [7] DEEPSEEK-AI. Deepseek-r1: incentivizing reasoning capability in llms via reinforcement learning[A/OL]. 2025. <https://arxiv.org/abs/2501.12948>.
- [8] KAPLAN J, MCCANDLISH S, HENIGHAN T, et al. Scaling laws for neural language models[A/OL]. 2020. <https://arxiv.org/abs/2001.08361>.
- [9] HOFFMANN J, BORGEAUD S, MENSCH A, et al. Training compute-optimal large language models[A/OL]. 2022. <https://arxiv.org/abs/2203.15556>.
- [10] SHOEYBI M, PATWARY M, PURI R, et al. Megatron-lm: training multi-billion parameter language models using model parallelism[J]. CoRR, 2019, abs/1909.08053.
- [11] HUANG Y, CHENG Y, BAPNAA, et al. Gpipe: efficient training of giant neural networks using pipeline parallelism[C]//Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada. 2019: 103 – 112.
- [12] NARAYANAN D, HARLAP A, PHANISHAYEE A, et al. Pipedream: generalized pipeline parallelism for DNN training[C]//Proceedings of the 27th ACM Symposium on Operating Systems Principles, SOSP 2019, Huntsville, ON, Canada, October 27-30, 2019. ACM, 2019: 1 – 15. DOI: 10.1145/3341301.3359646.
- [13] LEPIKHIN D, LEE H, XU Y, et al. Gshard: scaling giant models with conditional computation and automatic sharding[C]//9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021. OpenReview.net, 2021.
- [14] RAJBHANDARI S, RASLEY J, RUWASE O, et al. Zero: memory optimizations toward training trillion parameter models[C]//Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2020, Virtual Event / Atlanta, Georgia, USA, November 9-19, 2020. IEEE/ACM, 2020: 20. DOI: 10.1109/SC41405.2020.00024.

-
- [15] NARAYANAN D, SHOEYBI M, CASPER J, et al. Efficient large-scale language model training on GPU clusters using megatron-lm[J/OL]. 2021: 58. DOI: 10.1145/3458817.3476209.
 - [16] LI S, ZHAO Y, VARMA R, et al. Pytorch distributed: experiences on accelerating data parallel training[J/OL]. Proc. VLDB Endow., 2020, 13(12): 3005 – 3018. DOI: 10.14778/3415478.3415530.
 - [17] ZHAO Y, GU A, VARMA R, et al. Pytorch FSDP: experiences on scaling fully sharded data parallel[J/OL]. Proc. VLDB Endow., 2023, 16(12): 3848 – 3860. DOI: 10.14778/3611540.3611569.
 - [18] EUROPEAN UNION. General data protection regulation[Z/OL]. 2018. <https://gdpr-info.eu/art-5-gdpr/>.
 - [19] U.S. DEPARTMENT OF HEALTH AND HUMAN SERVICES. The hipaa privacy rule[Z/OL]. 1999. <https://www.hhs.gov/hipaa/for-professionals/privacy/index.html>.
 - [20] Dylan PATEL Daniel Nishball, ONTIVEROS J E. Multi-datacenter training: openai's ambitious plan to beat google's infrastructure[Z/OL]. 2024. <https://newsletter.semianalysis.com/p/multi-datacenter-training-openais>.
 - [21] 国家发展改革委等部门. 国家发展改革委等部门关于深入实施“东数西算”工程加快构建全国一体化算力网的实施意见[Z/OL]. 2023. https://www.gov.cn/zhengce/zhengceku/202401/content_6924596.htm.
 - [22] NVIDIA CORPORATION. Nvidia nvlink[Z/OL]. 2021. <https://www.nvidia.com/en-us/data-center/nvlink/>.
 - [23] NVIDIA CORPORATION. Hdr infiniband[Z/OL]. 2019. <https://www.nvidia.com/en-us/networking/infiniband-switching/>.
 - [24] JAIN S, KUMAR A, MANDAL S, et al. B4: experience with a globally-deployed software defined wan[C]//ACM SIGCOMM 2013 Conference, SIGCOMM 2013, Hong Kong, August 12-16, 2013. ACM, 2013: 3 – 14. DOI: 10.1145/2486001.2486019.
 - [25] CHEN T, KUBICEK A, HUANG L, et al. Crosspipe: towards optimal pipeline schedules for cross-datacenter training[C]//Proceedings of the 2025 USENIX Annual Technical Conference, USENIX ATC 2025, Boston, MA, USA, July 7-9, 2025. USENIX Association, 2025: 1089 – 1108.
 - [26] DAI J, WANG X, FANG K, et al. Geopipe: a geo-distributed LLM training framework with enhanced pipeline parallelism in a lossless rdma-enabled datacenter optical transport network[J/OL]. CoRR, 2025, abs/2510.12064. DOI: 10.48550/ARXIV.2510.12064.
 - [27] HEMMINGER S. Network emulation with netem[Z/OL]. <https://api.semanticscholar.org/CorpusID:17786091>.
 - [28] RASLEY J, RAJBHANDARI S, RUWASE O, et al. Deepspeed: system optimizations enable training deep learning models with over 100 billion parameters[C]//Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. ACM, 2020. DOI: 10.1145/3394486.3406703.
 - [29] FANG J, ZHAO S. Usp: a unified sequence parallelism approach for long context generative ai[A/OL]. 2024. <https://arxiv.org/abs/2405.07719>.
 - [30] GHADIAR, ABRAHAM M, VOROBYOV S, et al. Untied ulysses: memory-efficient context parallelism via headwise chunking[A/OL]. 2026. <https://arxiv.org/abs/2602.21196>.
 - [31] QI P, WAN X, HUANG G, et al. Zero bubble (almost) pipeline parallelism[C]//The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024. OpenReview.net, 2024.

-
- [32] SHE R, PANG B, LI K, et al. Automatic operator-level parallelism planning for distributed deep learning – a mixed-integer programming approach[A/OL]. 2025. <https://arxiv.org/abs/2503.09357>.
 - [33] DENG H, SONG X, ZHANG M, et al. JEEVES: the valet who masters the art of cross-dc training scheduling[C]//Proceedings of the 24th ACM Workshop on Hot Topics in Networks, HotNets 2025, UMD Campus, College Park, MD, USA, November 17-18, 2025. ACM, 2025: 168 – 175. DOI: 10.1145/3772356.3772387.
 - [34] MCMAHAN H B, MOORE E, RAMAGE D, et al. Federated learning of deep networks using model averaging[J]. CoRR, 2016, abs/1602.05629.
 - [35] STICH S U. Local SGD converges fast and communicates little[C]//7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019. OpenReview.net, 2019.
 - [36] GU X, LYU K, HUANG L, et al. Why (and when) does local SGD generalize better than sgd?[C]//The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. OpenReview.net, 2023.
 - [37] KARIMIREDDY S P, KALE S, MOHRI M, et al. SCAFFOLD: stochastic controlled averaging for federated learning[C]//Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event. PMLR, 2020: 5132 – 5143.
 - [38] JAGHOUAR S, ONG J M, HAGEMANN J. Opendiloco: an open-source framework for globally distributed low-communication training[J/OL]. CoRR, 2024, abs/2407.07852. DOI: 10.48550/ARXIV.2407.07852.
 - [39] DOUILLARD A, DONCHEV Y, RUSH K, et al. Streaming diloco with overlapping communication: towards a distributed free lunch[J/OL]. CoRR, 2025, abs/2501.18512. DOI: 10.48550/ARXIV.2501.18512.
 - [40] KLOUDAS K, RODRIGUES R, PREGUIÇA N M, et al. PIXIDA: optimizing data parallel jobs in wide-area data analytics[J/OL]. Proc. VLDB Endow., 2015, 9(2): 72 – 83. DOI: 10.14778/2850578.2850582.
 - [41] GUPTA S, AGRAWAL A, GOPALAKRISHNAN K, et al. Deep learning with limited numerical precision[C]//Proceedings of the 32nd International Conference on Machine Learning, ICML 2015, Lille, France, 6-11 July 2015. 2015: 1737 – 1746.
 - [42] SEIDE F, FU H, DROPPA J, et al. 1-bit stochastic gradient descent and its application to data-parallel distributed training of speech dnns[C]//15th Annual Conference of the International Speech Communication Association, INTERSPEECH 2014, Singapore, September 14-18, 2014. ISCA, 2014: 1058 – 1062. DOI: 10.21437/INTERSPEECH.2014-274.
 - [43] ALISTARH D, LI J, TOMIOKA R, et al. QSGD: randomized quantization for communication-optimal stochastic gradient descent[C]//. 2016.
 - [44] WEN W, XU C, YAN F, et al. Terngrad: ternary gradients to reduce communication in distributed deep learning[C]//Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA. 2017: 1509 – 1519.
 - [45] AJI A F, HEAFIELD K. Sparse communication for distributed gradient descent[C]//Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing, EMNLP 2017, Copenhagen, Denmark, September 9-11, 2017. Association for Computational Linguistics, 2017: 440 – 445. DOI: 10.18653/V1/D17-1045.

-
- [46] STICH S U, CORDONNIER J B, JAGGI M. Sparsified SGD with memory[C]//Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada. 2018: 4452 – 4463.
- [47] LIN Y, HAN S, MAO H, et al. Deep gradient compression: reducing the communication bandwidth for distributed training[C]//6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018. OpenReview.net, 2018.
- [48] KARIMIREDDY S P, REBJOCK Q, STICH S U, et al. Error feedback fixes signsgd and other gradient compression schemes[C]//Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA. PMLR, 2019: 3252 – 3261.
- [49] XU A, HUO Z, HUANG H. Step-ahead error feedback for distributed training with compressed gradient[C]//Proceedings of the AAAI Conference on Artificial Intelligence, 35(12). AAAI Press, 2021: 10478 – 10486. DOI: 10.1609/AAAI.V35I12.17254.
- [50] XU A, HUANG H. Detached error feedback for distributed SGD with random sparsification[C]//International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA. PMLR, 2022: 24550 – 24575.
- [51] WANG Y, WU R, LI X, et al. Pactrain: pruning and adaptive sparse gradient compression for efficient collective communication in distributed deep learning[C]//62nd ACM/IEEE Design Automation Conference, DAC 2025, San Francisco, CA, USA, June 22-25, 2025. IEEE, 2025: 1 – 7. DOI: 10.1109/DAC63849.2025.11133419.
- [52] MODORANU I V, KALINOV A, KURTIC E, et al. Error feedback can accurately compress preconditioners[C]//Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024. OpenReview.net, 2024.
- [53] VOGELS T, KARIMIREDDY S P, JAGGI M. Powersgd: practical low-rank gradient compression for distributed optimization[C]//Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada. 2019.
- [54] RABENSEIFNER R. Optimization of collective reduction operations[C]//Computational Science - ICCS 2004, 4th International Conference, Kraków, Poland, June 6-9, 2004, Proceedings, Part I. Springer, 2004: 1 – 9. DOI: 10.1007/978-3-540-24685-5_1.
- [55] SERGEEV A, DEL BALSIO M. Horovod: fast and easy distributed deep learning in tensorflow[A/OL]. 2018. <https://arxiv.org/abs/1802.05799>.
- [56] NVIDIA CORPORATION. Nvidia NCCL[Z/OL]. 2024. <https://developer.nvidia.com/nccl>.
- [57] WANG G, VENKATARAMAN S, PHANISHAYEE A, et al. Blink: fast and generic collectives for distributed ml[A/OL]. 2019. <https://arxiv.org/abs/1910.04940>.
- [58] SHAH A, CHIDAMBARAM V, COWAN M, et al. Taccl: guiding collective algorithm synthesis using communication sketches[A/OL]. 2021. <https://arxiv.org/abs/2111.04867>.
- [59] WARRAICH E, SHABTAI O, MANAA K, et al. Optireduce: resilient and tail-optimal allreduce for distributed deep learning in the cloud[A/OL]. 2023. <https://arxiv.org/abs/2310.06993>.
- [60] ZHONG Y, XIE C, ZHENG S, et al. Compressed communication for distributed training: adaptive methods and system[A/OL]. 2021. <https://arxiv.org/abs/2105.07829>.

- [61] ZHOU Y, YE Q, LV J. Communication-efficient federated learning with compensated overlap-fedavg[J/OL]. IEEE Trans. Parallel Distributed Syst., 2022, 33(1): 192 – 205. DOI: 10.1109/TPDS.2021.3090331.
- [62] HE K, ZHANG X, REN S, et al. Deep residual learning for image recognition[C]//2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016. IEEE Computer Society, 2016: 770 – 778. DOI: 10.1109/CVPR.2016.90.
- [63] DENG J, DONG W, SOCHER R, et al. Imagenet: A large-scale hierarchical image database[C]//2009 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR 2009), 20-25 June 2009, Miami, Florida, USA. IEEE Computer Society, 2009: 248 – 255. DOI: 10.1109/CVPR.2009.5206848.

攻读硕士学位期间取得的成果

攻读硕士学位期间取得的创新成果

- [1] **Yuntong Li**, Limin Xiao, Zhisheng Huo, Jinquan Wang, Zhibin Jia, Li Ruan, Lun Li. FusionTransmit: A Compression-Aware Distributed Training Scheduler for PowerSGD[J]. CCF Transactions on High Performance Computing, 2026-03-03. (CCF C, 已录用)
- [2] Runnan Shen, Jinquan Wang, Zhisheng Huo, Limin Xiao, Shengyang Tan, **Yuntong Li**, Lun Li, Xiangrong Xu, Liang Wang. A two-stage data placement strategy for cloud-edge-device collaborative environment[J]. COMPUTER COMMUNICATIONS, 2026-04-27
- [3] 肖利民, 沈润楠, 王锦权, 霍志胜, 谭升阳, **李云潼**, 阮利. 面向云边端协同环境的数据布局方法[P]. 中国专利: CN119544721A, 2025-02-28.

攻读硕士学位期间参与的主要科研工作

参与国家重点研发计划共性关键技术类专项“面向云边端协同的芯粒结构与系统软件”(项目编号: 2023YFB4503100, 周期: 2023.11 - 2026.10)。作为项目/课题骨干, 全面参与了项目的调研、申报及实施工作。期间, 围绕云边端跨域分布式训练场景下的通信瓶颈问题开展系统性研究, 主要负责通信优化技术方案的设计、实现及相关实验验证。

致谢

时光荏苒，从我收到北航录取通知书的那刻起，至今已过了三年。回首这一段时光，我深感自己无论是学术探索亦或是个人成长方面都取得了显著的进步，这都得益于在这段时光中遇到的每一个人给予我的帮助和鼓励。在此，我想向所有陪伴我走过这段旅程的人表达最诚挚的感谢。

首先，我由衷感谢我的导师肖利民教授。肖老师深厚的学术造诣、独到的见解和严谨的治学态度，深深地感染了我。对于每个科研项目，大到整体架构设计，小到每个技术细节，肖老师都亲力亲为，与我们展开深入的细致探讨，力求完美，使我们能够高质量地完成科研工作。非常感谢肖老师对我在科研道路上的指导，让我得到了成长。同时，也要感谢课题组的王良老师、霍志胜老师、阮利老师、刘磊、廖晓坚老师。各位老师尽职尽责地完成了实验室的组织管理工作，同时为我们的学术研究提供了充足的指导性意见。

其次，我要感谢实验室的同学们对我的关心和帮助。非常感谢王锦权博士对我在科研工作和工程实践上的指导和讨论，锦权学术能力比起我都要强上不少，让我十分钦佩。在和锦权一同完成的各项科研工作中，我们相互之间的讨论和分析给我提供了许多帮助，教会了我许多工程技术上的学习技巧，也获取了很多学术上的灵感。同时也要感谢我的同门谭升阳同学，和他在研究生伊始共同求学时便是我学习的榜样，他的代码能力和工程经验在我的开发过程和求职过程中给予了很大帮助。同样也要感谢贾志斌师兄和沈润楠师兄，两位师兄为我的学术探索提供了宝贵的思路。感谢各位同窗好友们，与你们共度的时光充满欢乐和收获。

此外，我要特别感谢我的家人。感谢我的父母对我的关心和支持，让我能够心无旁骛地完成学业。在我求学的道路上，你们始终是我坚实的后盾。

最后，我要感谢参与本论文评审和答辩的各位专家教授，各位的宝贵意见对我论文质量的提高起到了至关重要的作用。我将认真对待每一条建议，不断完善自己的学术成果。

再次向所有给予我帮助的人致以最诚挚的谢意！